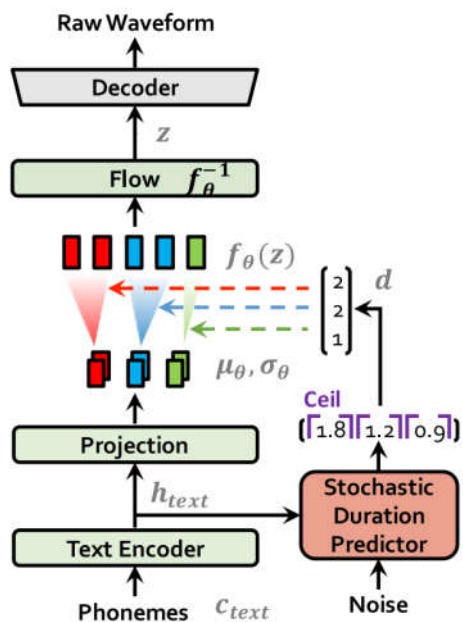
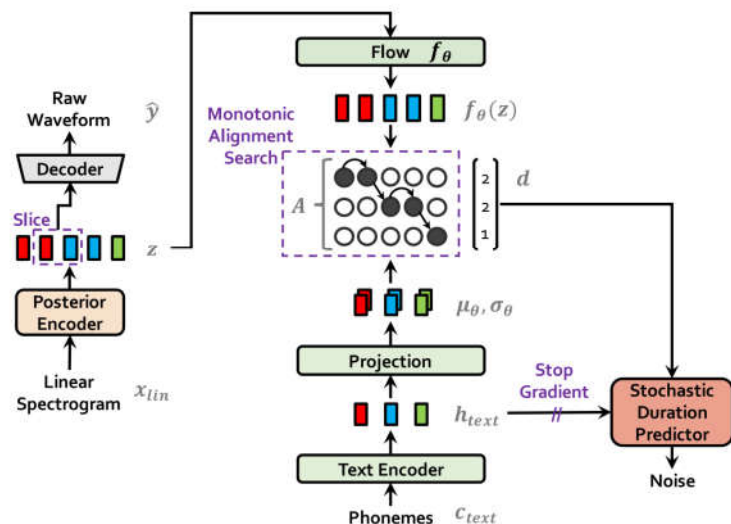


# 智能语音

## VITS Variational Inference Text-to-Speech



于泓  
鲁东大学  
信息与电气工程学院

# 一个例子

## (1) 下载代码及模型

链接:

<https://pan.baidu.com/s/1PArk1czW6ugvwwxdZ9wu0w>

提取码: 23t9

支持中日双语, 模拟了“马娘”以及“原神”中十多个不同人物的声音

```
#coding:utf-8
from text import text_to_sequence
import utils
from models import SynthesizerTrn
import torch
import commons
from torch import no_grad, LongTensor
import os
import soundfile as sf

limitation = os.getenv("SYSTEM") == "spaces"

hps_ms = utils.get_hparams_from_file('./configs/uma_trilingual.json')

device = torch.device("cpu")

net_g_ms = SynthesizerTrn(
    len(hps_ms.symbols),
    hps_ms.data.filter_length // 2 + 1,
    hps_ms.train.segment_size // hps_ms.data.hop_length,
    n_speakers=hps_ms.data.n_speakers,
    **hps_ms.model)
_ = net_g_ms.eval().to(device)
speakers = hps_ms.speakers
model, optimizer, learning_rate, epochs = utils.load_checkpoint("./pretrained_models/G_trilingual.pth", net_g_ms,
```

```

def get_text(text, hps):
    text_norm = text_to_sequence(text, hps.symbols, hps.data.text_cleaners)
    if hps.data.add_blank:
        text_norm = commons.intersperse(text_norm, 0)
    text_norm = LongTensor(text_norm)
    return text_norm

def vits(text, language, speaker_id, noise_scale, noise_scale_w, length_scale):
    if not len(text):
        return "输入文本不能为空!", None, None
    text = text.replace('\n', ' ').replace('\r', '').replace(" ", "")
    if len(text) > 100 and limitation:
        return f"输入文字过长! {len(text)}>100", None, None
    if language == 0:
        text = f"[ZH]{text}[ZH]"
    elif language == 1:
        text = f"[JA]{text}[JA]"
    else:
        text = f"{text}"
    stn_tst = get_text(text, hps_ms)
    with no_grad():
        x_tst = stn_tst.unsqueeze(0).to(device)
        x_tst_lengths = LongTensor([stn_tst.size(0)]).to(device)
        speaker_id = LongTensor([speaker_id]).to(device)
        # tensor([132]) 0.1 0.668 1.1
        audio = net_g_ms.infer(x_tst, x_tst_lengths, sid=speaker_id, noise_scale=noise_scale,
                               noise_scale_w=noise_scale_w,
                               length_scale=length_scale)[0][0, 0].data.cpu().float().numpy()

    return 22050, audio
  
```

文本

语种

说话人ID

文本转符号序列

增加随机性

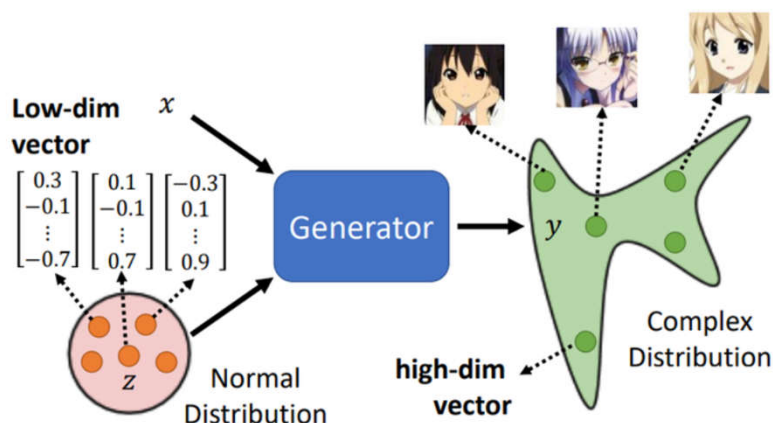
调速

```
if __name__ == "__main__":  
    speakers = {"特别周": 0,  
               "无声铃鹿": 1,  
               "东海帝王": 2,  
               "丸善斯基": 3,  
               "富士奇迹": 4,  
               "小栗帽": 5,  
               "黄金船": 6,  
               "伏特加": 7, "大和赤骥": 8, "大树快车": 9, "草上飞": 10, "菱亚马逊": 11, "目白麦昆": 12,  
               }  
  
    speed = 1  
    for k in speakers:  
        id = speakers[k]  
        print(k)  
        sr, audio = vits('你好呀,欢迎来到鲁东大学', 0, torch.tensor([id]), 0.1, 0.668, 1.0/speed)  
        sf.write('output_file'+k+'.wav', audio, samplerate=sr)
```

# 三种生成模型 (GAN、VAE, FLOW)

## • 什么是生成模型?

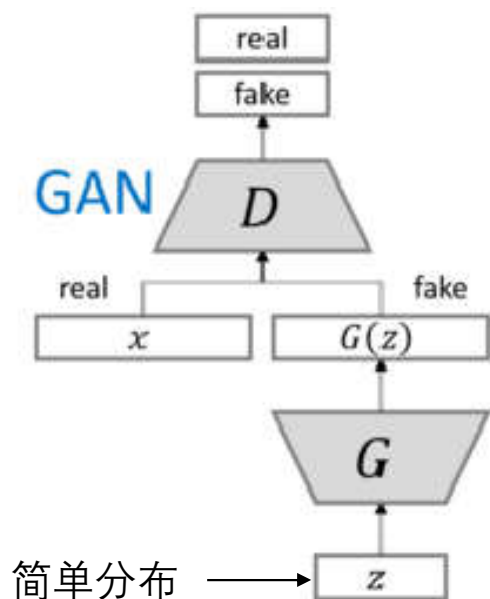
图像、文本、语音等数据，都可以看做是从一个复杂分布中采样得到的。



一个简单的分布随机分布，经过一系列复杂的变换 (Generator) 之后变成复杂分布

从简单分布中随机采样一个  $z$ ，经过  $G$  后生成一个复杂样本  $x$ ，这个过程就叫做生成。

# GAN(Generative Adversarial Networks) 生成竞争网络



$$L_D = E \left[ (D(x) - 1)^2 + (D(G(z)) - 0)^2 \right]$$

$$L_G = E \left[ (D(G(z)) - 1)^2 \right] + \lambda \|x - G(z)\|$$

有时还会  
加入重构损失  
令生成数据  
和真实数据  
尽可能相似

生成器 G 与鉴别器 D 依次交替更新。

竞争训练

- 更新 D 时，固定 G，令 D 能区别 输入的是真实/real 或者 虚假/fake 的数据
- 更新 G 时，固定 D，令 G 生的虚假数据能够欺骗鉴别器 D

GAN的作用： 使生成的采样数据  
和真实分布的采样数据尽可能相似



# VAE(Variational Auto-encoder) 变分自编码器

数据  $x$  符合复杂分布  $p_\theta(x)$ 。根据  $x$  直接建模  $p_\theta(x)$  比较困难，因此引入一个较为简单的先验分布  $p_\theta(z)$  先从简单分布中采样  $z$ ，再利用  $z$  生成  $x$

根据贝叶斯公式

$$p_\theta(x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(z|x)}$$

生成网络 (G)  $\nearrow$   $p_\theta(x|z)$

$\nearrow$  简单先验  $p_\theta(z)$

$\searrow$  难以求解  $p_\theta(z|x)$

设计一个  $q_\phi(z|x)$  尽可能的逼近  $p_\theta(z|x)$

Encoder 编码网络  $\nwarrow$   $q_\phi(z|x)$

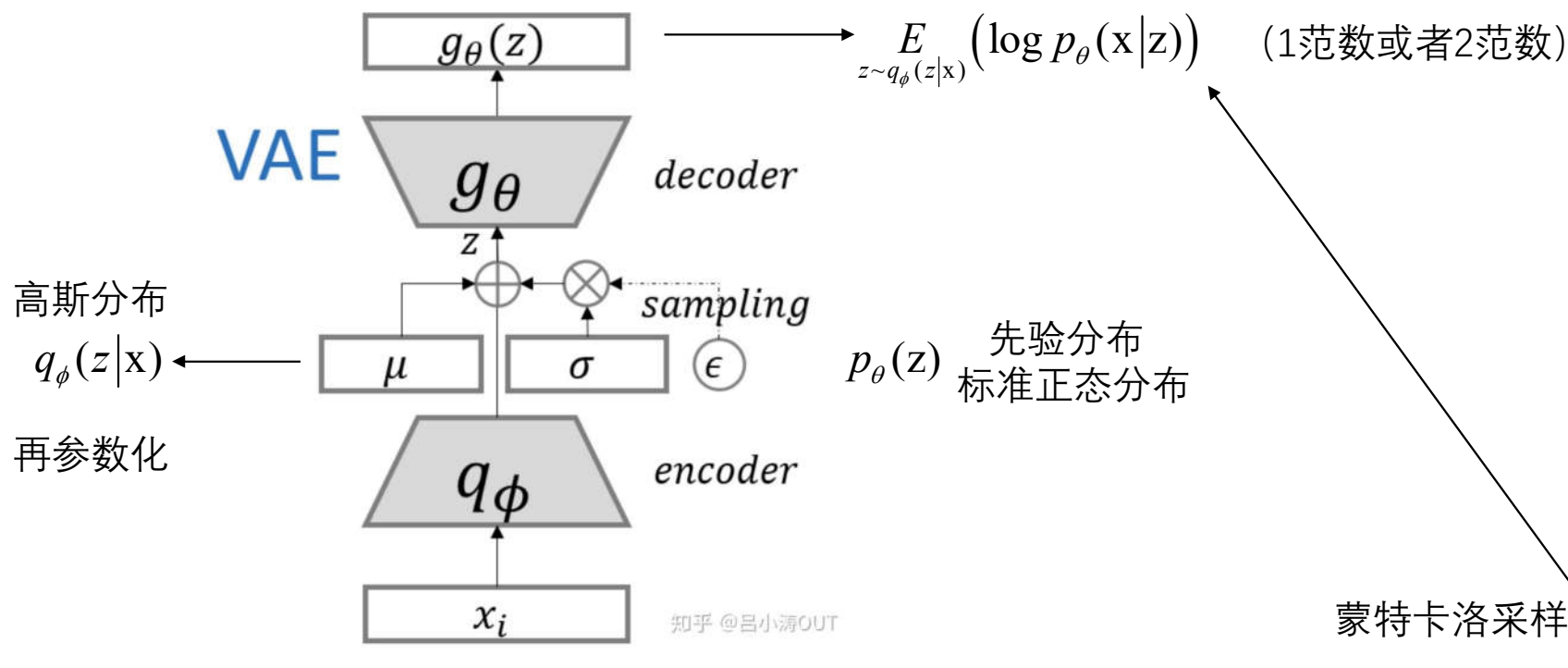
$$\begin{aligned}
 \text{KL}(q_\phi(z|x) \| p_\theta(z|x)) &= \sum_z q_\phi(z|x) \log \frac{q_\phi(z|x)}{p_\theta(z|x)} = \sum_z q_\phi(z|x) \log \frac{q_\phi(z|x) p_\theta(x)}{p_\theta(x|z) p_\theta(z)} \\
 &= \sum_z q_\phi(z|x) \log q_\phi(z|x) + \sum_z q_\phi(z|x) \log p_\theta(x) - \sum_z q_\phi(z|x) \log p_\theta(x|z) - \sum_z q_\phi(z|x) \log p_\theta(z) \\
 &= \log p_\theta(x) + \left( \sum_z q_\phi(z|x) \log q_\phi(z|x) - \sum_z q_\phi(z|x) \log p_\theta(z) \right) - \sum_z q_\phi(z|x) \log p_\theta(x|z) \\
 &= \log p_\theta(x) + \text{KL}(q_\phi(z|x) \| p_\theta(z)) - E_{z \sim q_\phi(z|x)} (\log p_\theta(x|z))
 \end{aligned}$$

$$\log p_\theta(x) \geq \underbrace{E_{z \sim q_\phi(z|x)} (\log p_\theta(x|z)) - \text{KL}(q_\phi(z|x) \| p_\theta(z))}_{\text{Evidence lower bound (ELBO) 证据下界}}$$

Evidence lower bound (ELBO) 证据下界 **最大化**

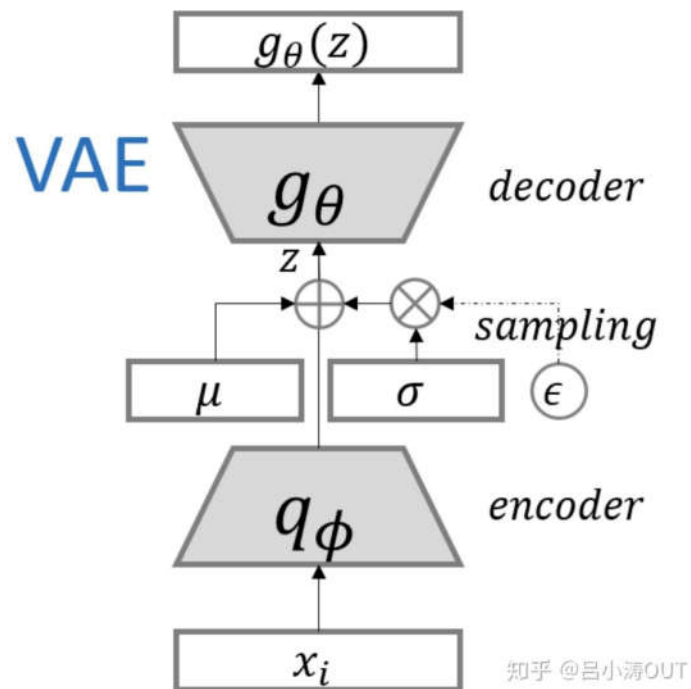
令encoder 生成的分布和先验分布尽可能相似

从encoder 生成的分布的分布中采样z, 再用z生成x期望最大



# 分布的逼近

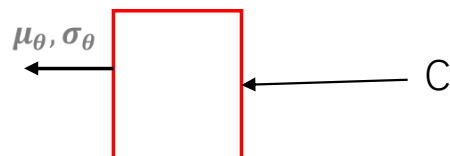
$$\tilde{\mathcal{L}}_{\text{VAE}}(\mathbf{x}; \theta, \phi) = -KL(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z})) + \frac{1}{L} \sum_{l=1}^L \log p_\theta(\mathbf{x}|\mathbf{z}^{(l)})$$



### VITS中的VAE

$$\log p_{\theta}(x|c) \geq \mathbb{E}_{q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) - \log \frac{q_{\phi}(z|x)}{p_{\theta}(z|c)} \right] \quad (1)$$

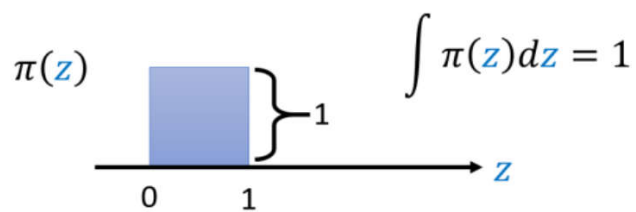
$p_{\theta}(z|c)$



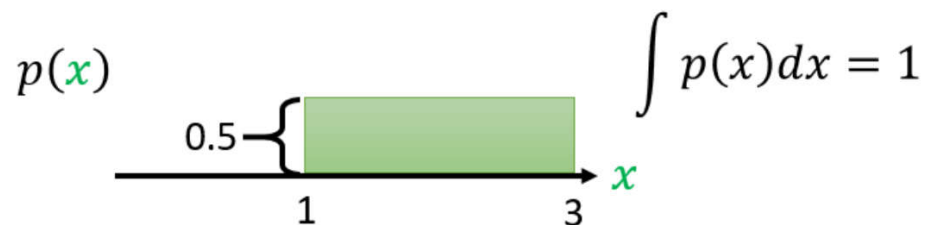
条件

# flow

直接构建 分布A到分布B的转换函数



$\pi(z)$  较为简单 0-1之间均匀分布



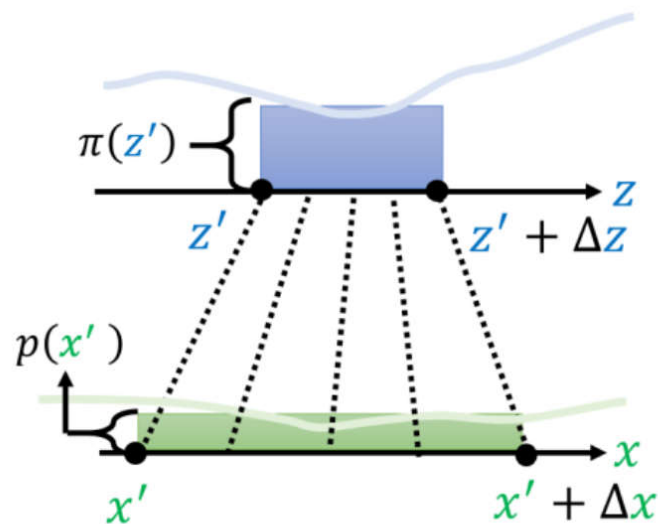
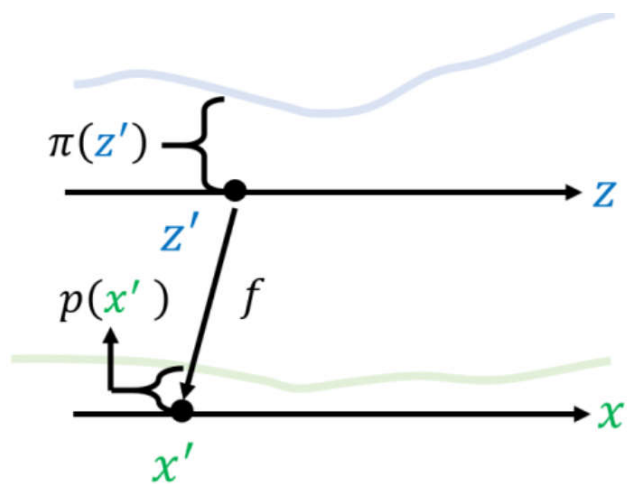
$p(x)$  较为复杂 1-3之间均匀分布

实现两个分布之间的转换?

$$\begin{aligned} x &= f(z) \\ &= 2z + 1 \end{aligned}$$

从 $z$ 中采样一个点  
乘一个数加一个数

考虑复杂分布，在一个小的区间内可以看成是均匀分布



蓝色方块和绿色方块  
需要有相同的面积

$$p(x')\Delta x = \pi(z')\Delta z$$

$$p(x') = \pi(z') \frac{\Delta z}{\Delta x}$$

$$= \pi(z') \frac{dz}{dx}$$

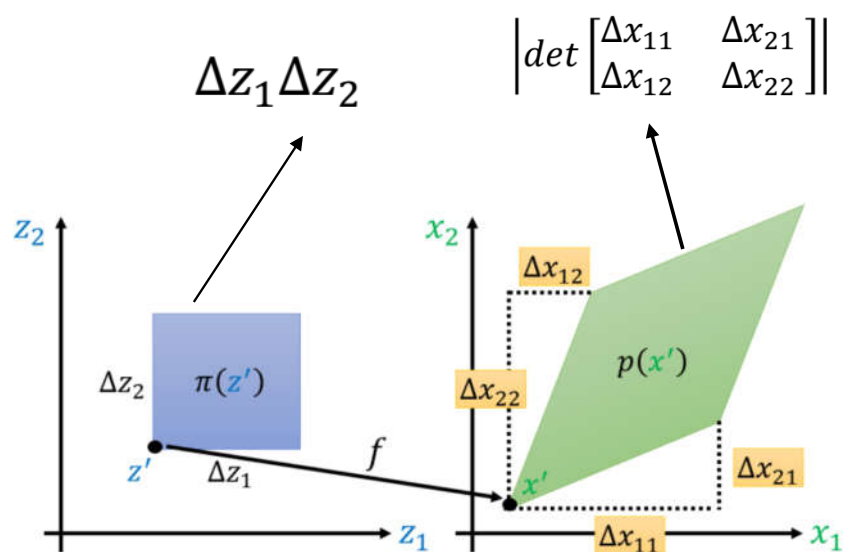
$$p(x') = \pi(z') \frac{dz}{dx}$$

$$\frac{dz}{dx}$$

有可能是负值

$$p(x') = \pi(z') \left| \frac{dz}{dx} \right|$$

考虑到多维的情况



$$p(x') \left| \det \begin{bmatrix} \Delta x_{11} & \Delta x_{21} \\ \Delta x_{12} & \Delta x_{22} \end{bmatrix} \right| = \pi(z') \Delta z_1 \Delta z_2$$

$$p(x') \left| \frac{1}{\Delta z_1 \Delta z_2} \det \begin{bmatrix} \Delta x_{11} & \Delta x_{21} \\ \Delta x_{12} & \Delta x_{22} \end{bmatrix} \right| = \pi(z')$$

$$p(x') \left| \det \begin{bmatrix} \Delta x_{11}/\Delta z_1 & \Delta x_{21}/\Delta z_1 \\ \Delta x_{12}/\Delta z_2 & \Delta x_{22}/\Delta z_2 \end{bmatrix} \right| = \pi(z')$$

$$p(x') \left| \det \begin{bmatrix} \partial x_1 / \partial z_1 & \partial x_2 / \partial z_1 \\ \partial x_1 / \partial z_2 & \partial x_2 / \partial z_2 \end{bmatrix} \right| = \pi(z')$$

正变换

$$p(x') | \det(J_f) | = \pi(z')$$

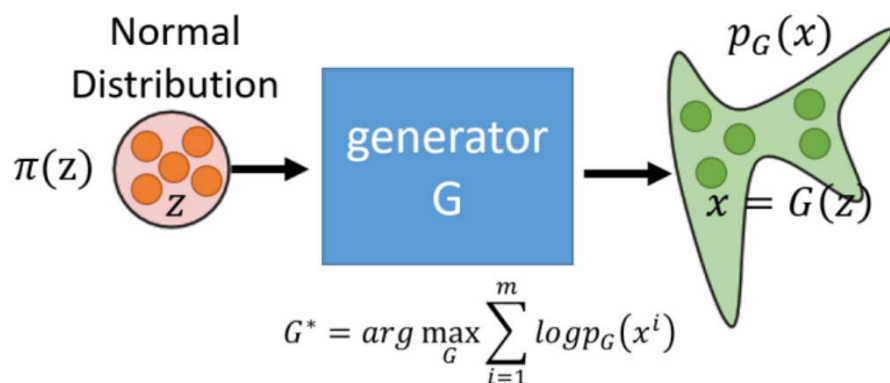
逆变换

$$p(x') = \pi(z') | \det(J_{f^{-1}}) |$$

至此，我们得到了一个比较重要的结论：如果  $z$  与  $x$  分别满足两种分布，并且  $z$  通过函数  $f$  能够转变为  $x$ ，那么  $z$  与  $x$  中的任意一组对应采样点  $z'$  与  $x'$  之间的关系为：

$$\begin{cases} \pi(z') = p(x') | \det(J_f) | \\ p(x') = \pi(z') | \det(J_{f^{-1}}) | \end{cases}$$

Flow模型的基本建模思路



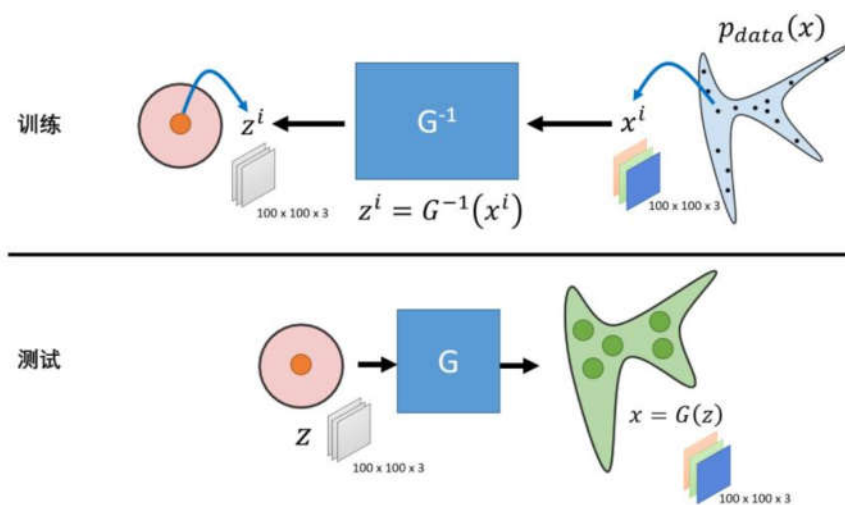
具体设计中的两个困难：

- 1、 $x=G(z)$ ， $G(z)$ 的表达式什么样子？
- 2、 $G^{-1}$ 的 Jacobian 矩阵 计算量过大

其中  $p_G(x^i) = \pi(z^i) (|\det(J_G)|)^{-1}$

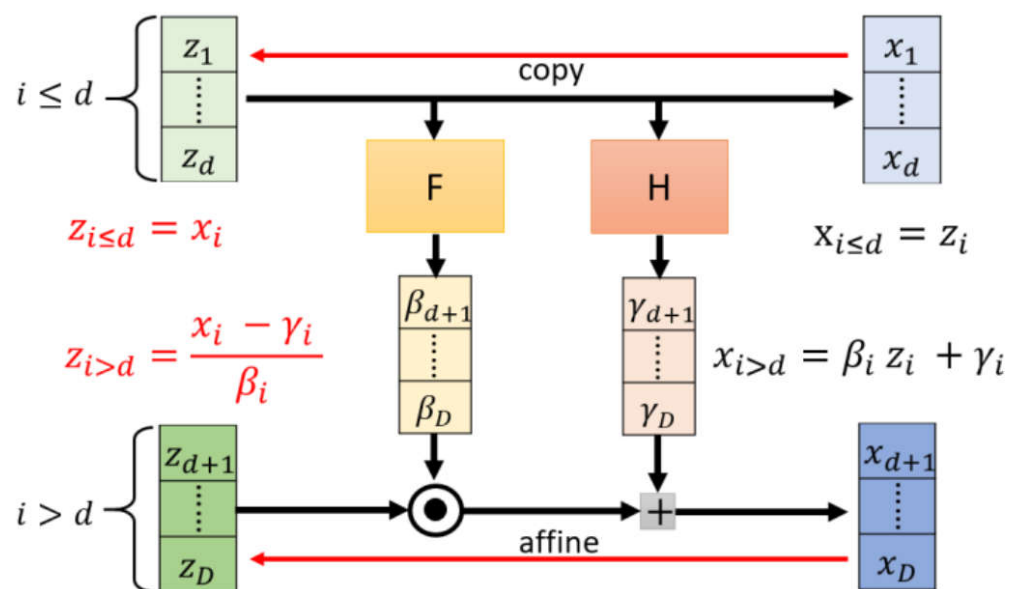
$$z^i = G^{-1}(x^i)$$

## 训练过程



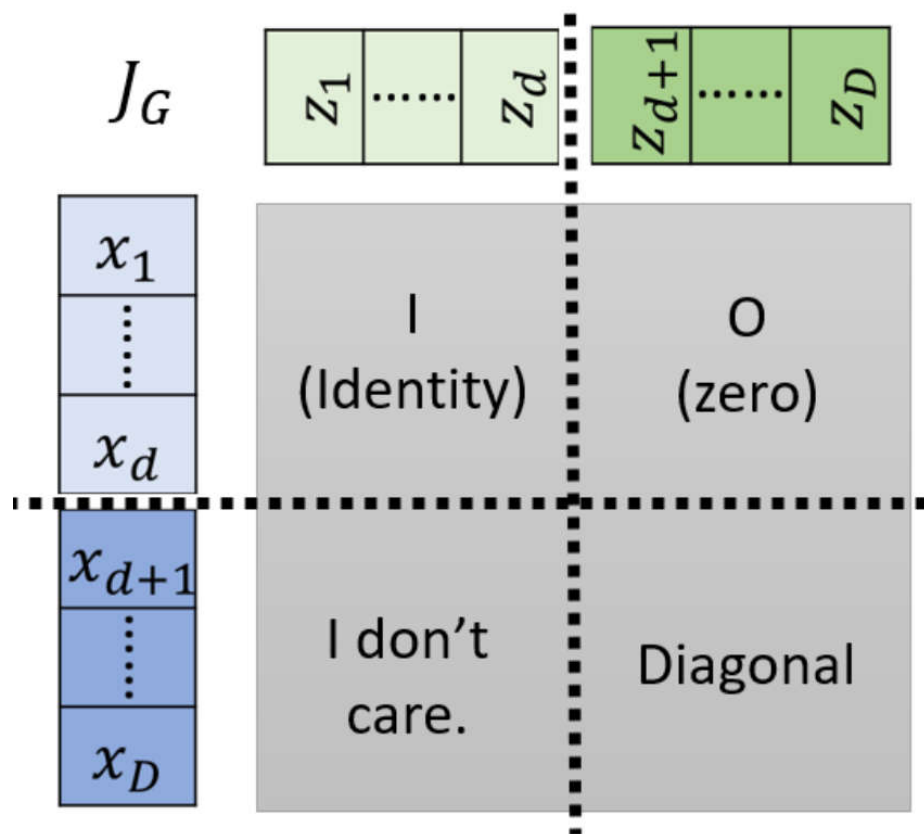
## G的内部结构

接下来开始具体考虑 G 的内部设计，为了让  $G^{-1}$  可以计算并且 G 的 Jacobian 行列式也易于计算，Flow-based Model 采用了一种称为耦合层（Coupling Layer）的设计来实现。

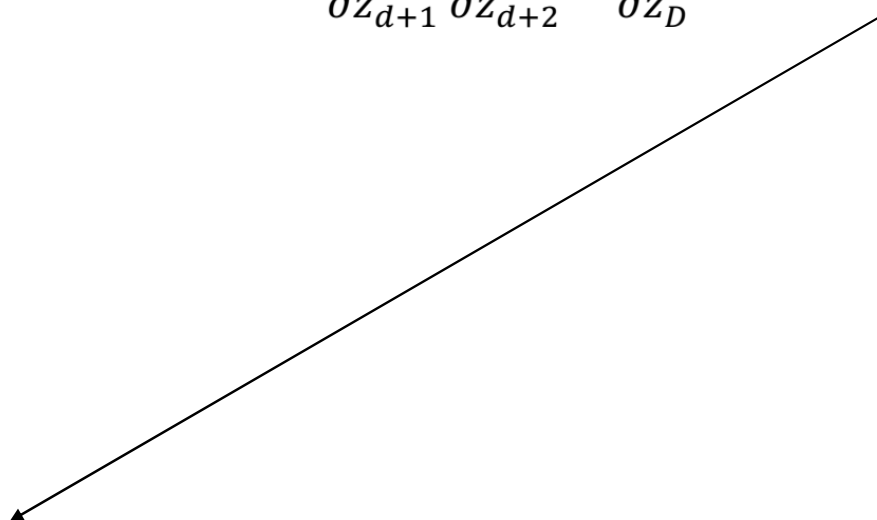


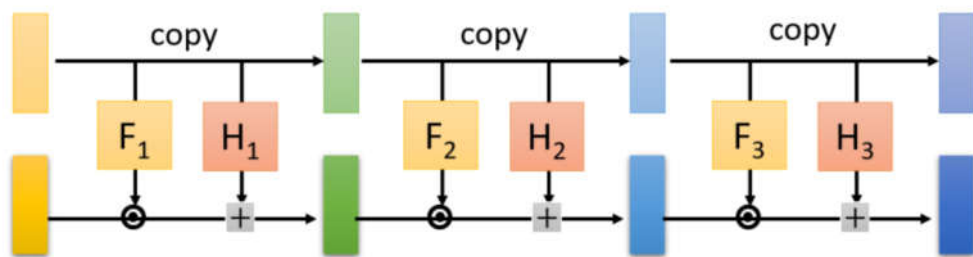
$$\begin{cases} x_i = z_i, i \leq d \\ x_i = \beta_i z_i + \gamma_i, i > d \end{cases}$$

$$\begin{cases} z_i = x_i, i \leq d \\ z_i = \frac{x_i - \gamma_i}{\beta_i}, i > d \end{cases}$$

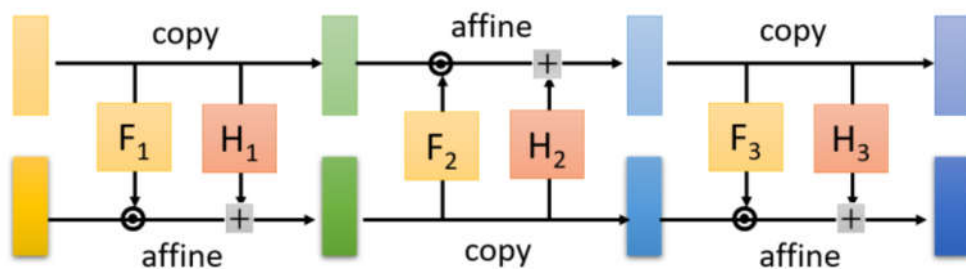


$$\det(J_G) = \frac{\partial x_{d+1}}{\partial z_{d+1}} \frac{\partial x_{d+2}}{\partial z_{d+2}} \cdots \frac{\partial x_D}{\partial z_D} = \beta_{d+1} \beta_{d+2} \cdots \beta_D$$





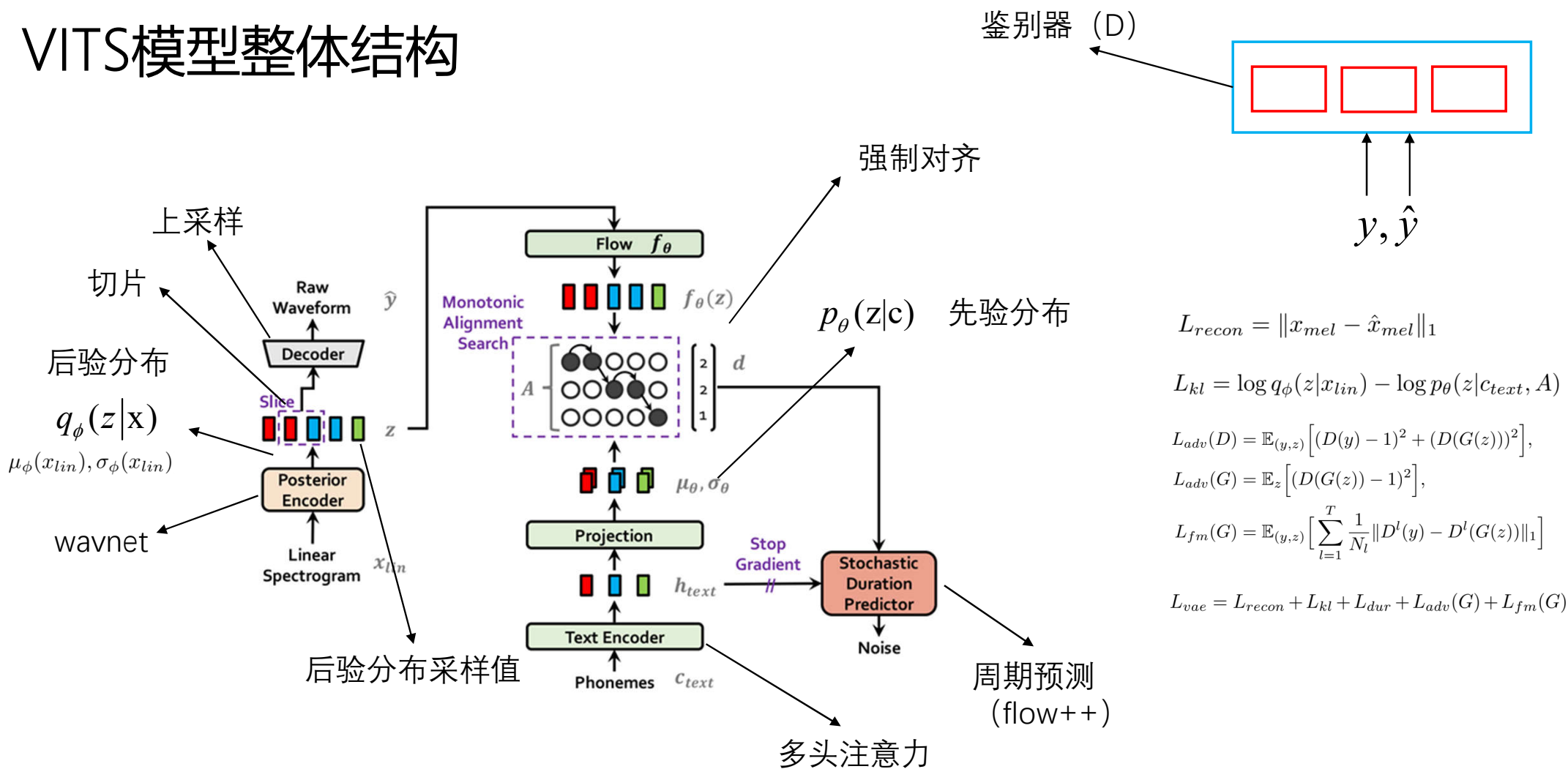
多模块耦合

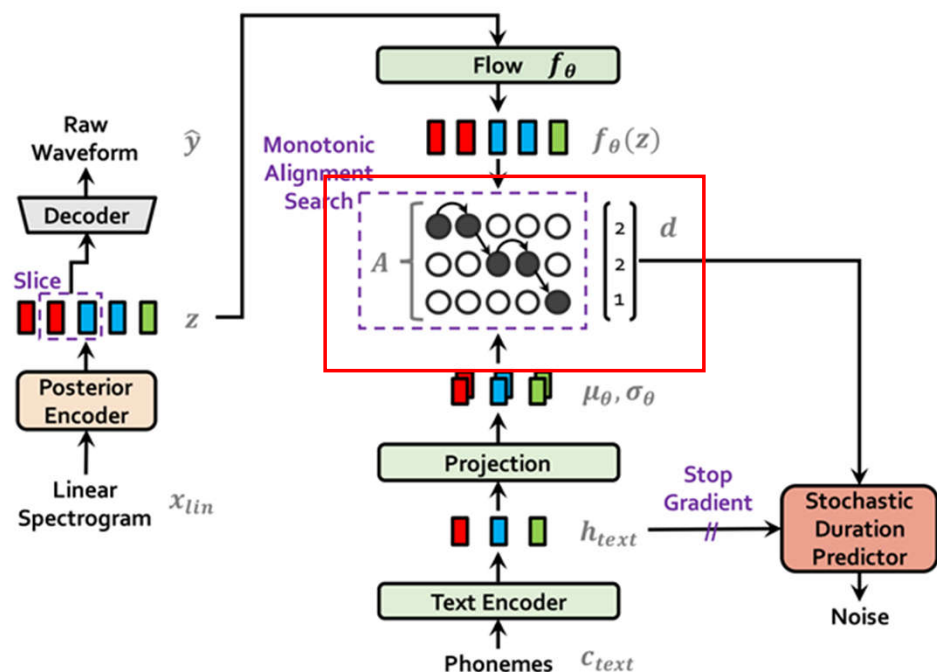


Copy与affine 交替



# VITS模型整体结构





此电脑 > Data (D:) > 工作相关 > 智能语音 > VITS > vits-main\_all > vits-main > monotonic\_align >

| 名称              | 修改日期            | 类型       |
|-----------------|-----------------|----------|
| __pycache__     | 2023/8/10 11:11 | 文件夹      |
| build           | 2023/8/10 11:02 | 文件夹      |
| monotonic_align | 2023/8/10 11:13 | 文件夹      |
| __init__.py     | 2023/8/10 11:10 | PY 文件    |
| core.c          | 2023/8/10 11:02 | C Source |
| core.pyx        | 2022/12/2 14:23 | PYX 文件   |
| setup.py        | 2022/12/2 14:23 | PY 文件    |

python setup.py build\_ext --inplace

2023/9/25

```
def monotonic_alignment_search(value):
    """Returns the most likely alignment for the given log-likelihood matrix.
    Args:
        value: the log-likelihood matrix. Its (i, j)-th entry contains
            the log-likelihood of the j-th latent variable
            for the given i-th prior mean and variance:
        .. math::
            value_{i,j} = \log N(f(z)_{j}; \mu_{i}, \sigma_{i})
            (dtype=float, shape=[text_length, latent_variable_length])
    Returns:
        path: the most likely alignment.
            (dtype=float, shape=[text_length, latent_variable_length])
    """
    t_x, t_y = value.shape # [text_length, latent_variable_length]
    path = zeros([t_x, t_y])

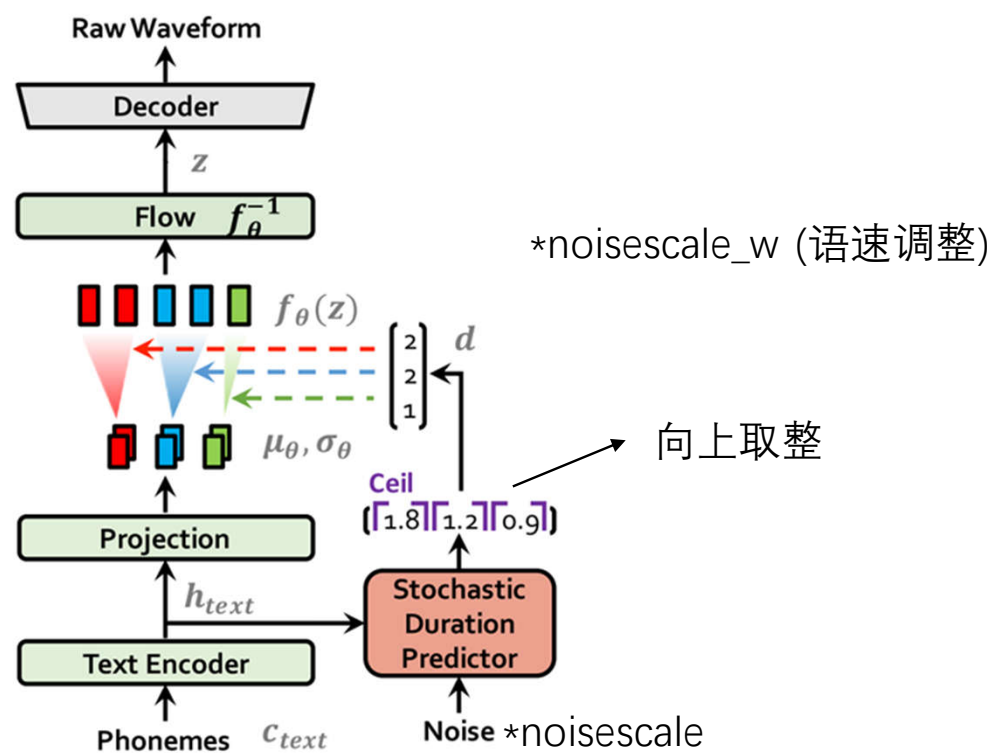
    # A cache to store the log-likelihood for the most likely alignment so far.
    Q = -INFINITY * ones([t_x, t_y])

    for y in range(t_y):
        for x in range(max(0, t_x + y - t_y), min(t_x, y + 1)):
            if y == 0: # Base case. If y is 0, the possible x value is only 0.
                Q[x, 0] = value[x, 0]
            else:
                if x == 0:
                    v_prev = -INFINITY
                else:
                    v_prev = Q[x-1, y-1]
                v_cur = Q[x, y-1]
                Q[x, y] = value[x, y] + max(v_prev, v_cur)

    # Backtrack from last observation.
    index = t_x - 1
    for y in range(t_y - 1, -1, -1):
        path[index, y] = 1
        if index != 0 and (index == y or Q[index, y-1] < Q[index-1, y-1]):
            index = index - 1

    return path
```

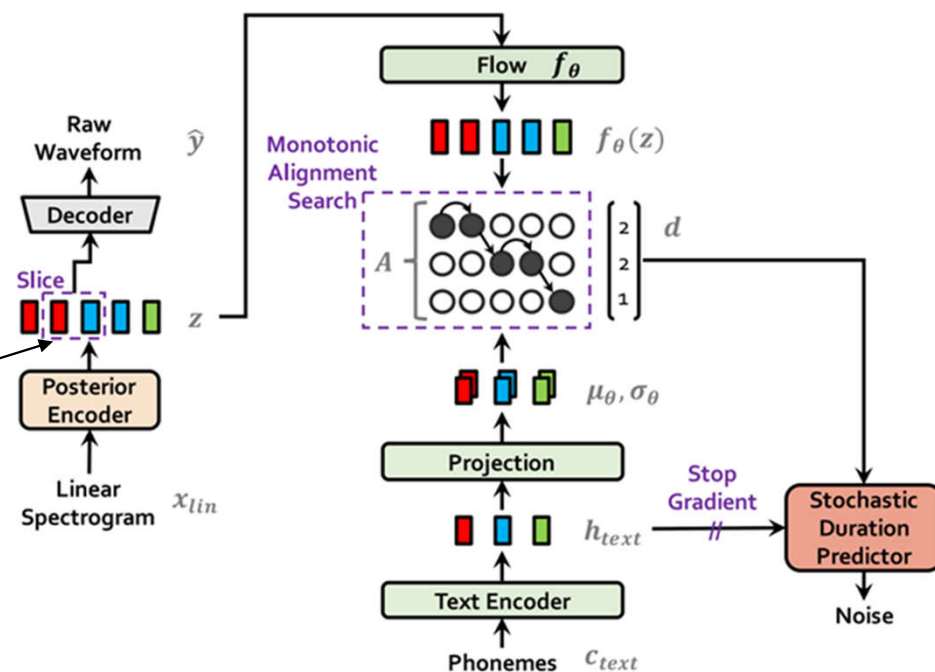
## 推理部分



&gt; 此电脑 &gt; Data (D:) &gt; 工作相关 &gt; 智能语音 &gt; VITS &gt; vits-main\_all &gt; vits-main &gt; configs

| 名称                  | 修改日期            | 类型   |
|---------------------|-----------------|------|
| 🎵 chinese_base.json | 2022/12/2 14:23 | JSON |

```
{
  "train": {
    "log_interval": 200,
    "eval_interval": 1000,
    "seed": 1234,
    "epochs": 10000,
    "learning_rate": 2e-4,
    "betas": [0.8, 0.99],
    "eps": 1e-9,
    "batch_size": 32,
    "fp16_run": true,
    "lr_decay": 0.999875,
    "segment_size": 8192,
    "init_lr_ratio": 1,
    "warmup_epochs": 0,
    "c_mel": 45,
    "c_kl": 1.0
  },
}
```



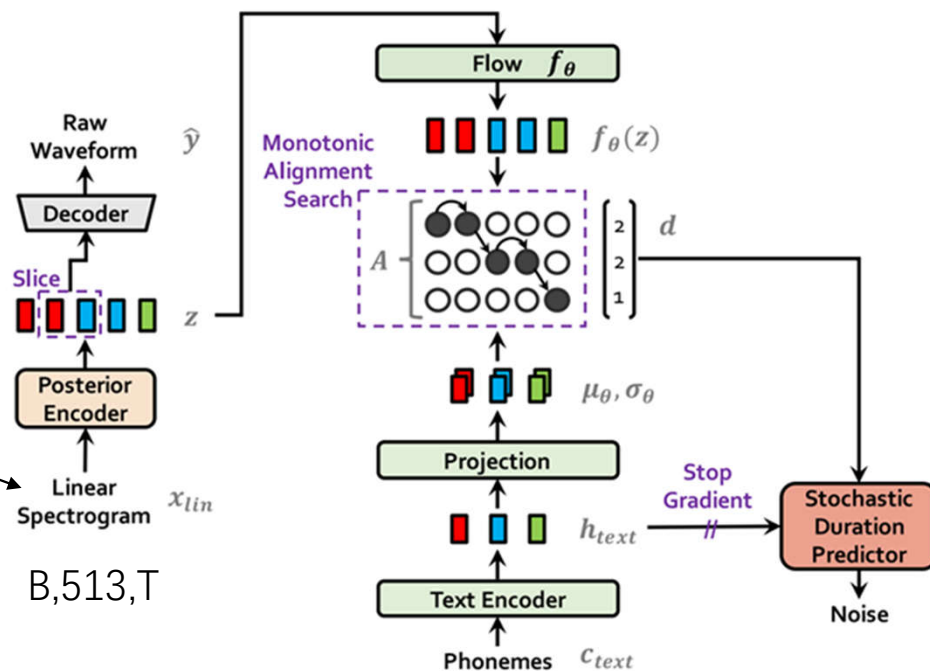
$$L_{vae} = L_{recon} + L_{kl} + L_{dur} + L_{adv}(G) + L_{fm}(G)$$

```

"data": {
  "training_files": "filelists/juzi_train_filelist.txt.cleaned",
  "validation_files": "filelists/juzi_val_filelist.txt.cleaned",
  "text_cleaners": ["chinese_cleaners"],
  "max_wav_value": 32768.0,
  "sampling_rate": 22050,
  "filter_length": 1024,
  "hop_length": 256,
  "win_length": 1024,
  "n_mel_channels": 80,
  "mel_fmin": 0.0,
  "mel_fmax": null,
  "add_blank": true,
  "n_speakers": 8,
  "cleaned_text": true
},

```

$$L_{recon} = \|x_{mel} - \hat{x}_{mel}\|_1$$



"model": {

"inter\_channels": 192,  
"hidden\_channels": 192,  
"filter\_channels": 768,  
"n\_heads": 2,  
"n\_layers": 6,  
"kernel\_size": 3,  
"p\_dropout": 0.1,

Textencoder

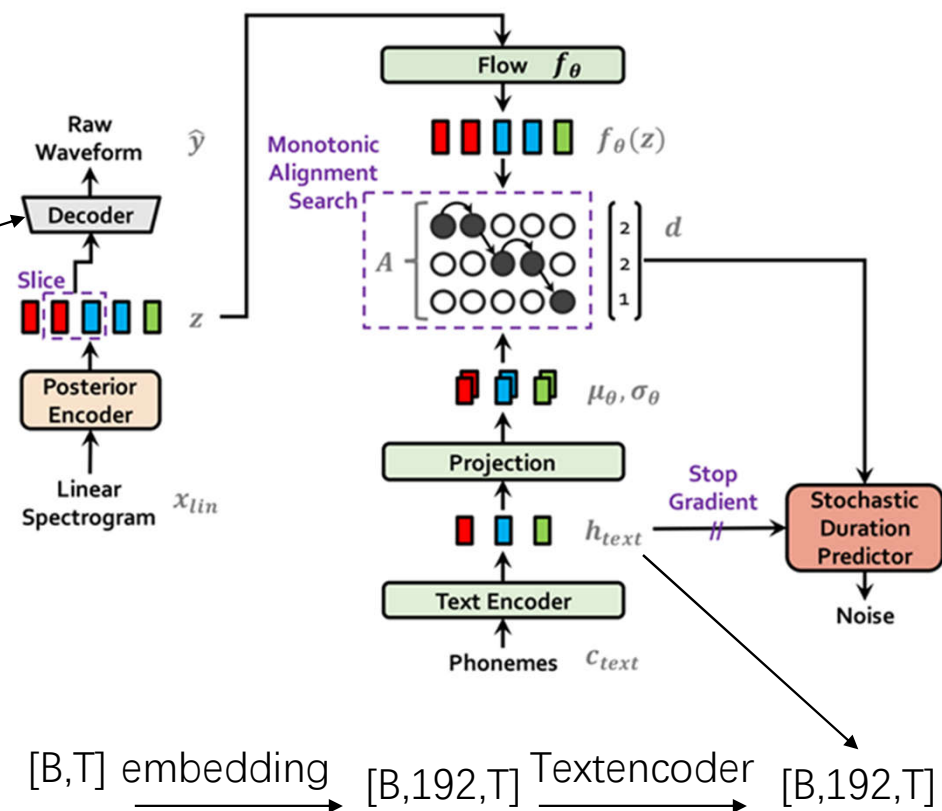
"resblock": "1",  
"resblock\_kernel\_sizes": [3, 7, 11],  
"resblock\_dilation\_sizes": [[1, 3, 5], [1, 3, 5], [1, 3, 5]],  
"upsample\_rates": [8, 8, 2, 2],  
"upsample\_initial\_channel": 512,  
"upsample\_kernel\_sizes": [16, 16, 4, 4],  
"n\_layers\_q": 3,  
"use\_spectral\_norm": false,  
"gin\_channels": 256

},

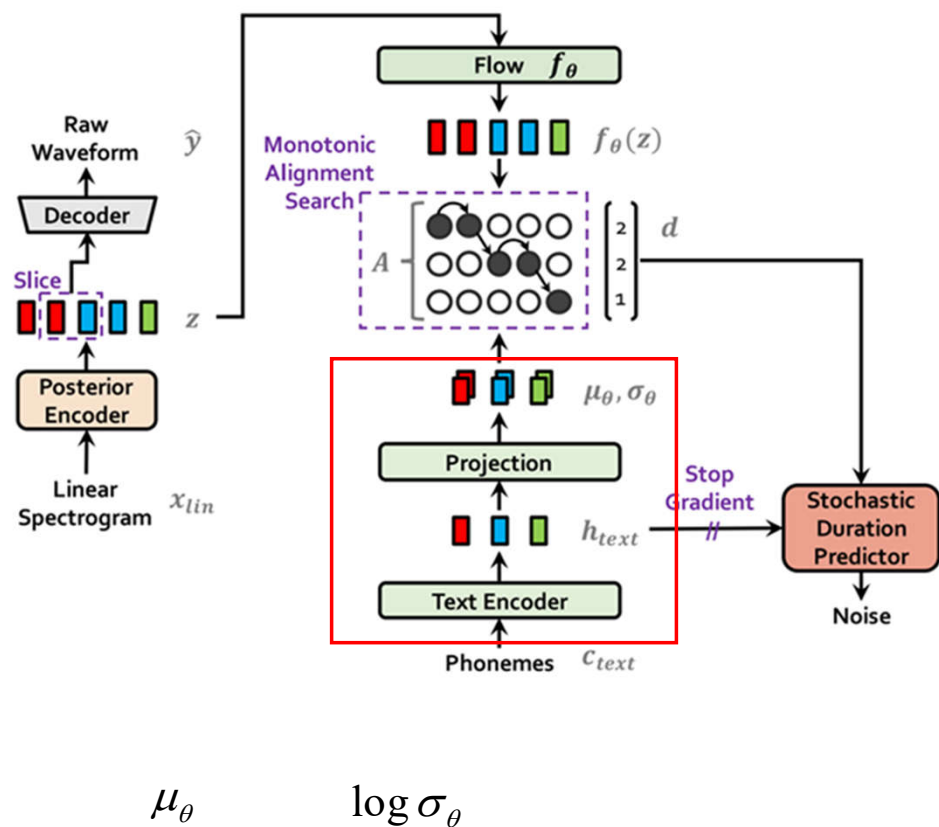
$8 \times 8 \times 2 \times 2 = 256$

Speaker\_id与symbols 列表

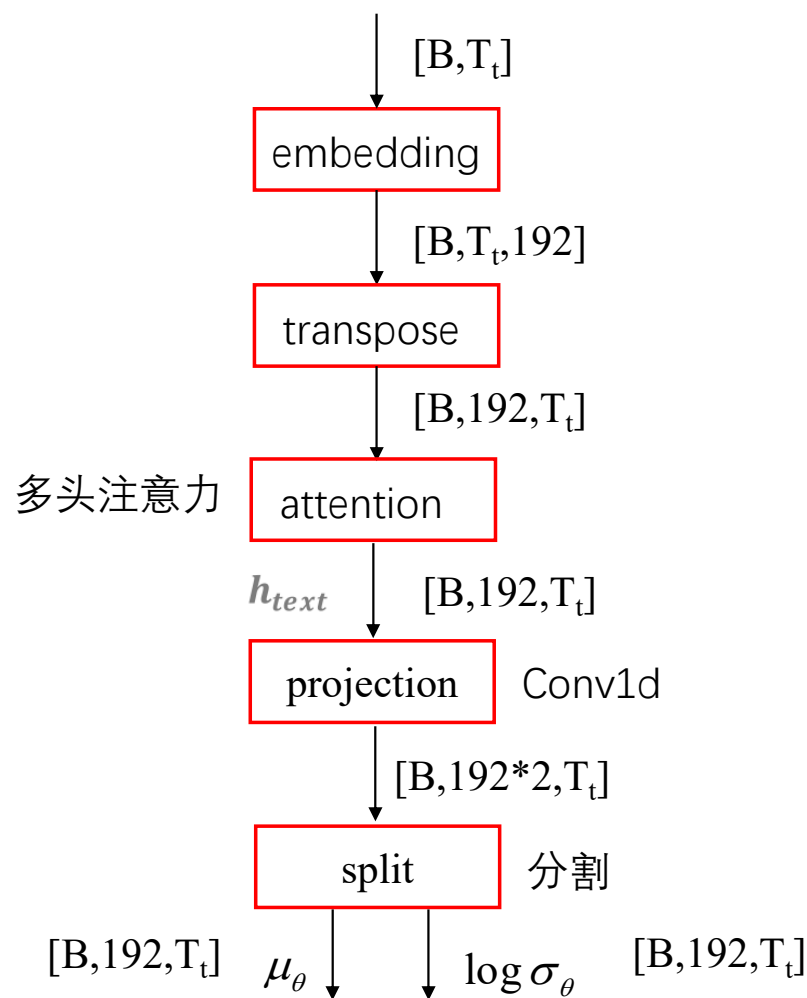
"speakers": ["\u5c0f\u8338", "\u5510\u4e50\u541f", "\u5c0f\u6bb7", "\u82b1\u73b2", "\u8bb8\u8001",  
"symbols": [" ", "\uff0c", "\u3002", "\uff01", "\uff1f", "\u2014", "\u2026", "\u3105", "\u3106",



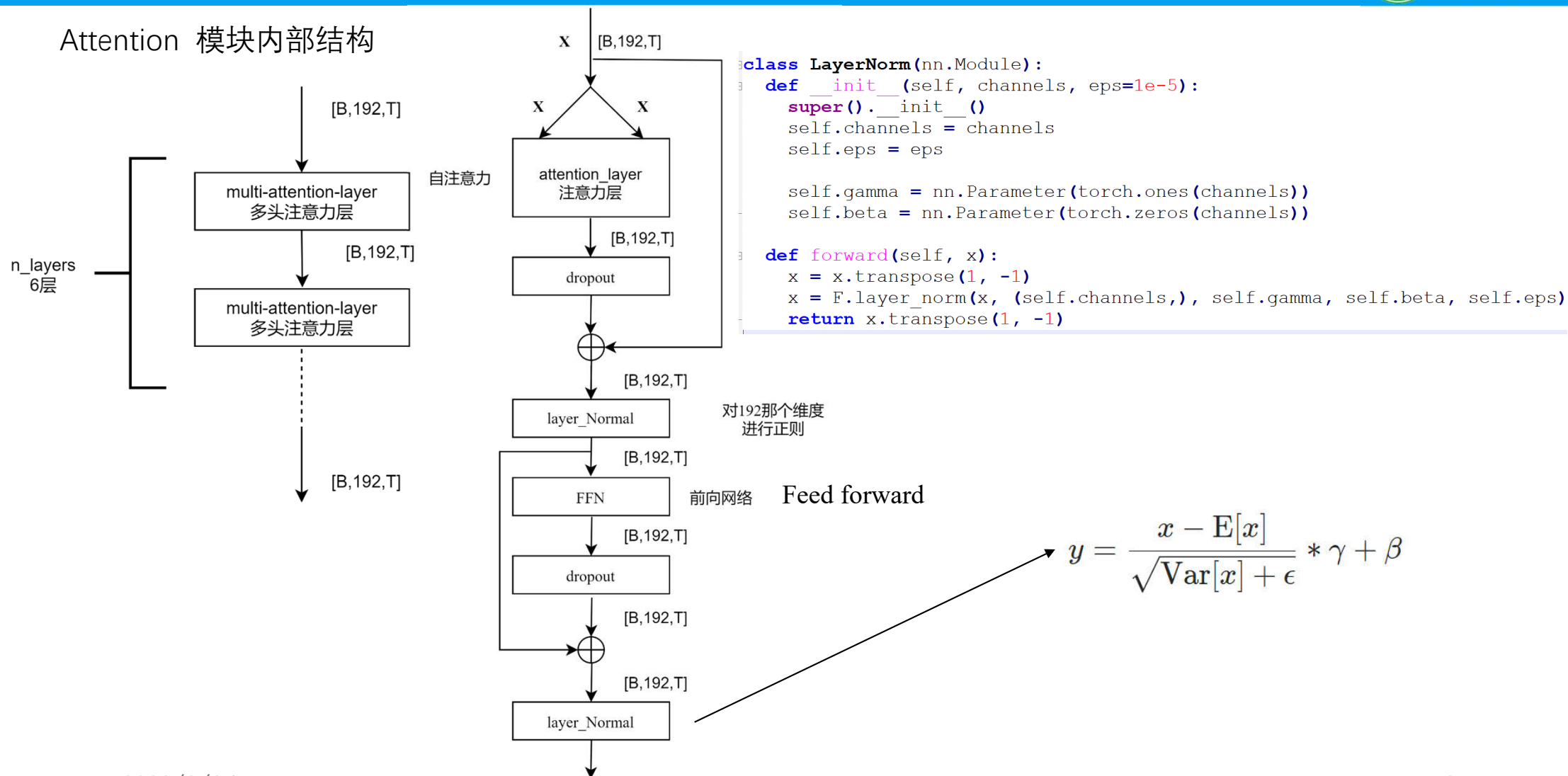
# 文本编码器

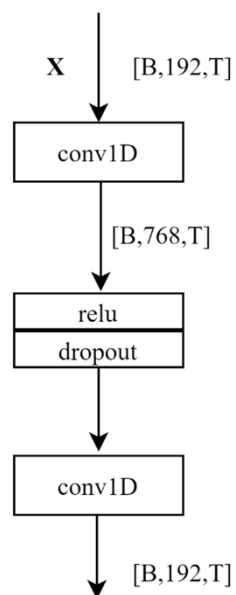
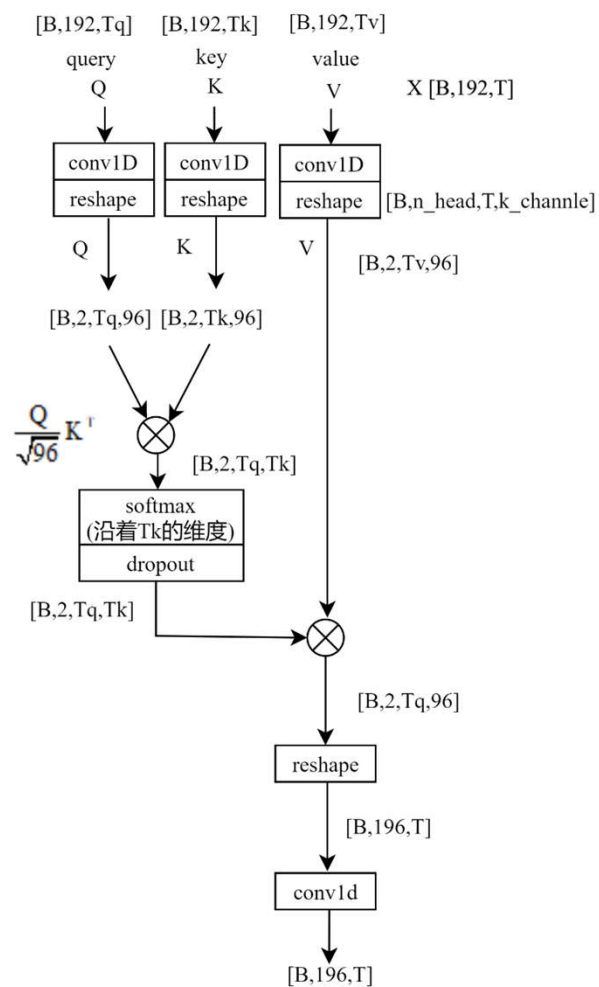


## TextEncoder (models.py 代码135行)



## Attention 模块内部结构

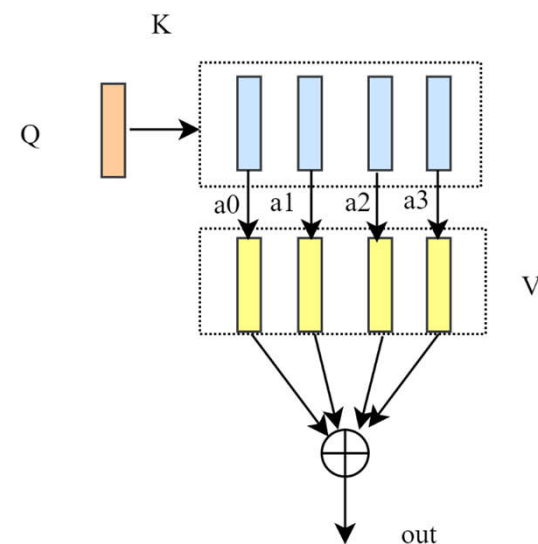


**FFN****多头注意力**

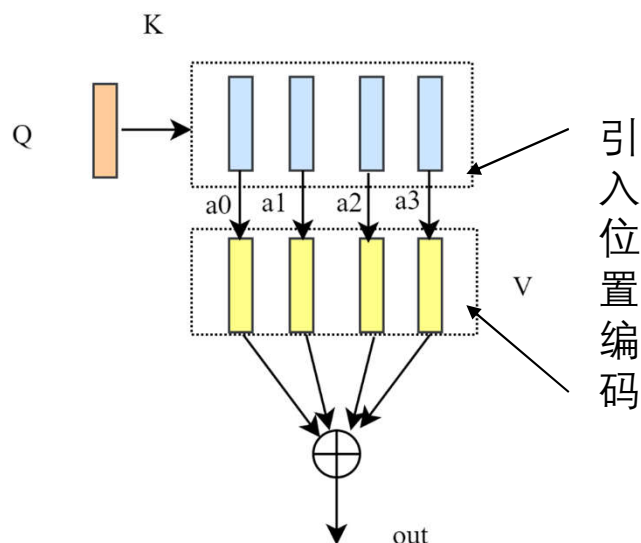
Q: 查询量

K: 特征池, 结合Q产生权重att

V: 输出特征, 维度与K相同, 由权重att加权后输出



## 相对位置编码 Self-Attention with Relative Position Representations



只要Q固定，那么不管Q出现在什么地方，输出 out 也是固定的

白天吃鸡，晚上吃烧鸡

游戏

食品

文字相同、意义不同，对应的隐变量输出也应当不一样

位置编码，只与位置有关，与内容无关。

$$PE_{t,2i} = \sin(t/10000^{2i/d}),$$

$$PE_{t,2i+1} = \cos(t/10000^{2i/d}),$$

早期的位置编码采用一个固定的公式来计算

假设输入序列的长度 $L=5$ ,  
那么就可以构建如下所示的  $L \times L \times d$  矩阵来描述,  
序列样本之间的**相对位置关系**

矩阵中的矢量, 有  $2 \times L - 1$  个

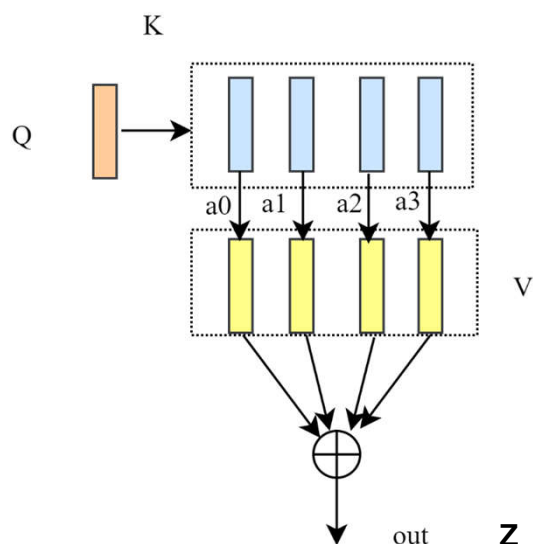
| x0    | x1       | x2       | x3       | x4       |
|-------|----------|----------|----------|----------|
| $w_0$ | $w_{-1}$ | $w_{-2}$ | $w_{-3}$ | $w_{-4}$ |
| $w_1$ | $w_0$    | $w_{-1}$ | $w_{-2}$ | $w_{-3}$ |
| $w_2$ | $w_1$    | $w_0$    | $w_{-1}$ | $w_{-2}$ |
| $w_3$ | $w_2$    | $w_1$    | $w_0$    | $w_{-1}$ |
| $w_4$ | $w_3$    | $w_2$    | $w_1$    | $w_0$    |

如果再设置一个边界值 **$K=2$**   
只考虑 **$K$** 邻域之内的相对关系 那么

矩阵中的矢量, 有  $2 \times K - 1$  个

| x0    | x1       | x2       | x3       | x4       |
|-------|----------|----------|----------|----------|
| $w_0$ | $w_{-1}$ | $w_{-2}$ | 0        | 0        |
| $w_1$ | $w_0$    | $w_{-1}$ | $w_{-2}$ | 0        |
| $w_2$ | $w_1$    | $w_0$    | $w_{-1}$ | $w_{-2}$ |
| 0     | $w_2$    | $w_1$    | $w_0$    | $w_{-1}$ |
| 0     | 0        | $w_2$    | $w_1$    | $w_0$    |

参考文献 Self-Attention with Relative Position Representations [1803.02155.pdf \(arxiv.org\)](https://arxiv.org/abs/1803.02155)



加入位置编码之前

$$Z_i = \sum_{j=1}^L \alpha_{ij} V_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^L \exp(e_{ik})}$$

$$e_{ij} = \frac{Q_i K_j^T}{\sqrt{d}}$$

加入位置编码之后

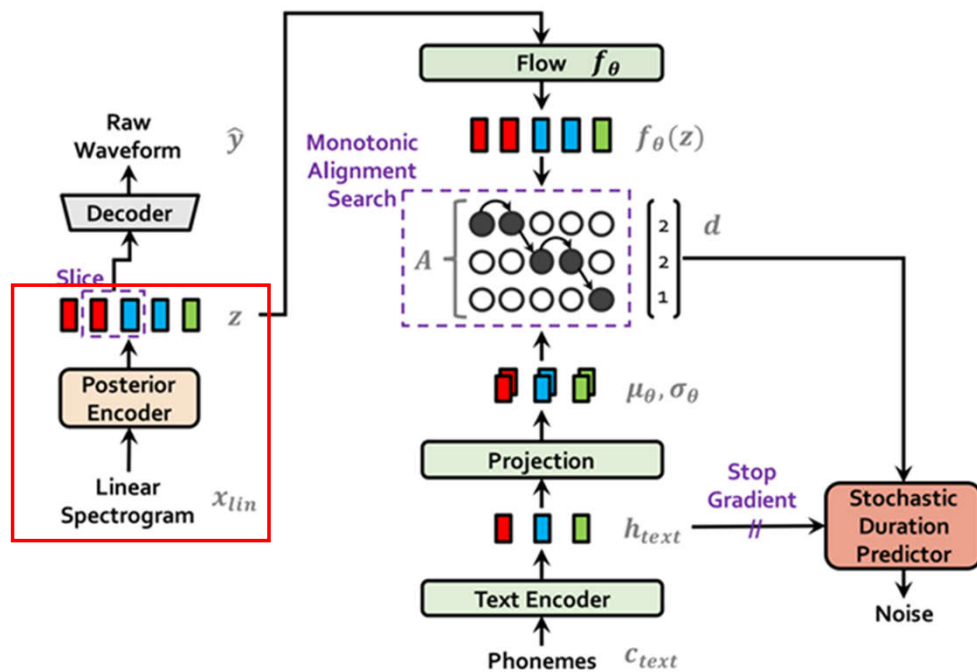
$$Z_i = \sum_{j=1}^L \alpha_{ij} (V_j + \alpha_{ij}^V)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^L \exp(e_{ik})}$$

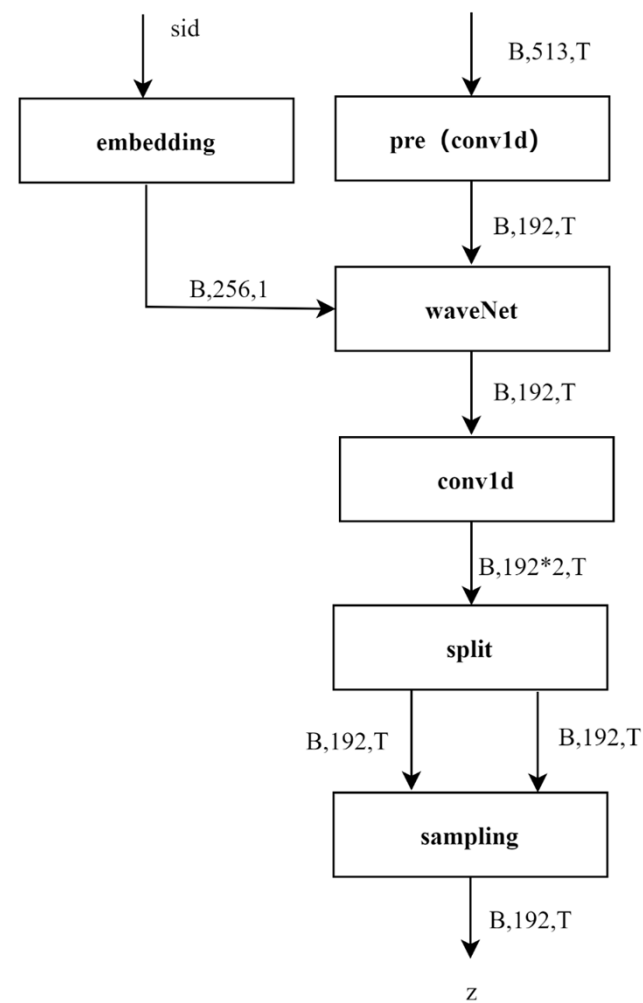
$$e_{ij} = \frac{Q_i (K_j^T + \alpha_{ij}^K)}{\sqrt{d}}$$

| x0             | x1              | x2              | x3              | x4              |
|----------------|-----------------|-----------------|-----------------|-----------------|
| w <sub>0</sub> | w <sub>-1</sub> | w <sub>-2</sub> | 0               | 0               |
| w <sub>1</sub> | w <sub>0</sub>  | w <sub>-1</sub> | w <sub>-2</sub> | 0               |
| w <sub>2</sub> | w <sub>1</sub>  | w <sub>0</sub>  | w <sub>-1</sub> | w <sub>-2</sub> |
| 0              | w <sub>2</sub>  | w <sub>1</sub>  | w <sub>0</sub>  | w <sub>-1</sub> |
| 0              | 0               | w <sub>2</sub>  | w <sub>1</sub>  | w <sub>0</sub>  |

$\alpha_{ij}^V, \alpha_{ij}^K$



PosteriorEncoder models.py 212 行



## 卷积中的dilation

|    |    |    |
|----|----|----|
| k0 | k1 | k2 |
|----|----|----|

|    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|
| x0 | x1 | x2 | x3 | x4 | x5 | x6 | x7 | x8 |
|----|----|----|----|----|----|----|----|----|

d=1

|    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|
| x0 | x1 | x2 | x3 | x4 | x5 | x6 | x7 | x8 |
|----|----|----|----|----|----|----|----|----|

$$y1 = x0 * k0 + x1 * k1 + x2 * k2$$

d=2

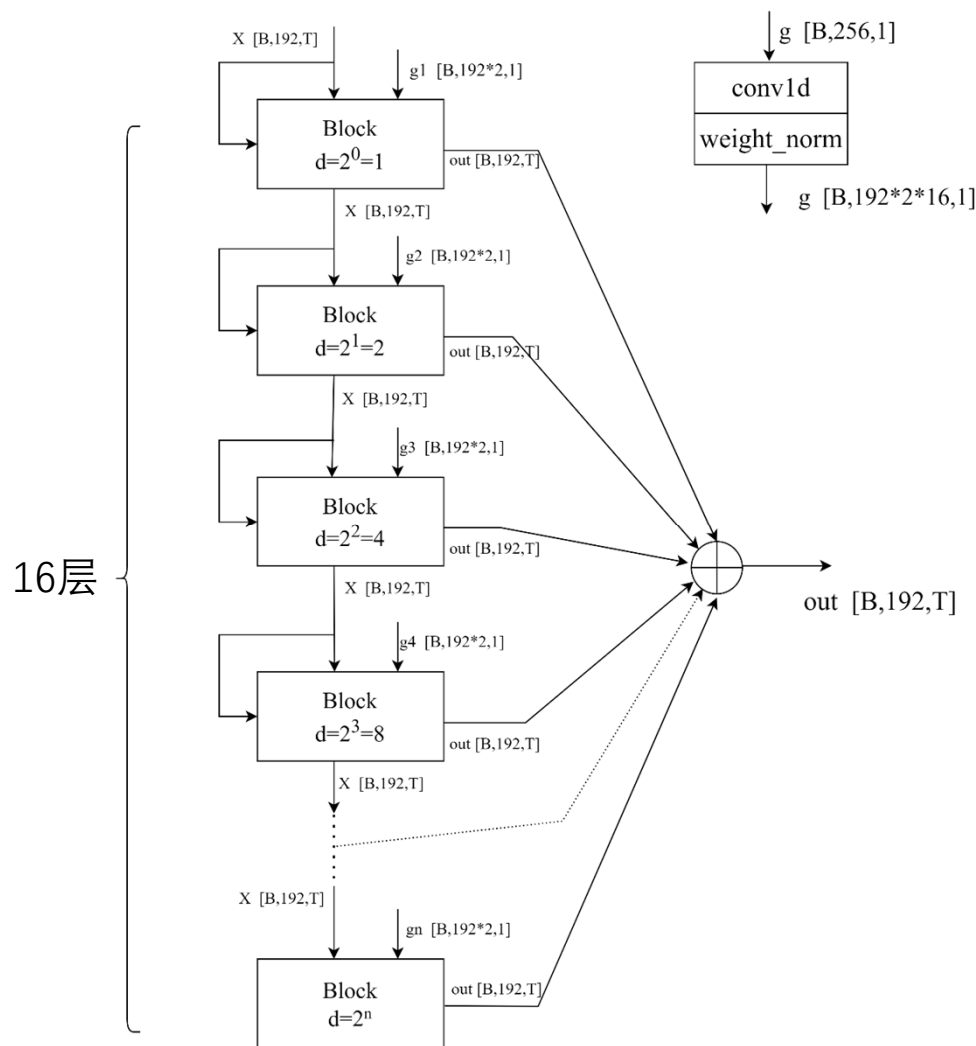
|    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|
| x0 | x1 | x2 | x3 | x4 | x5 | x6 | x7 | x8 |
|----|----|----|----|----|----|----|----|----|

$$y2 = x0 * k0 + x2 * k1 + x4 * k2$$

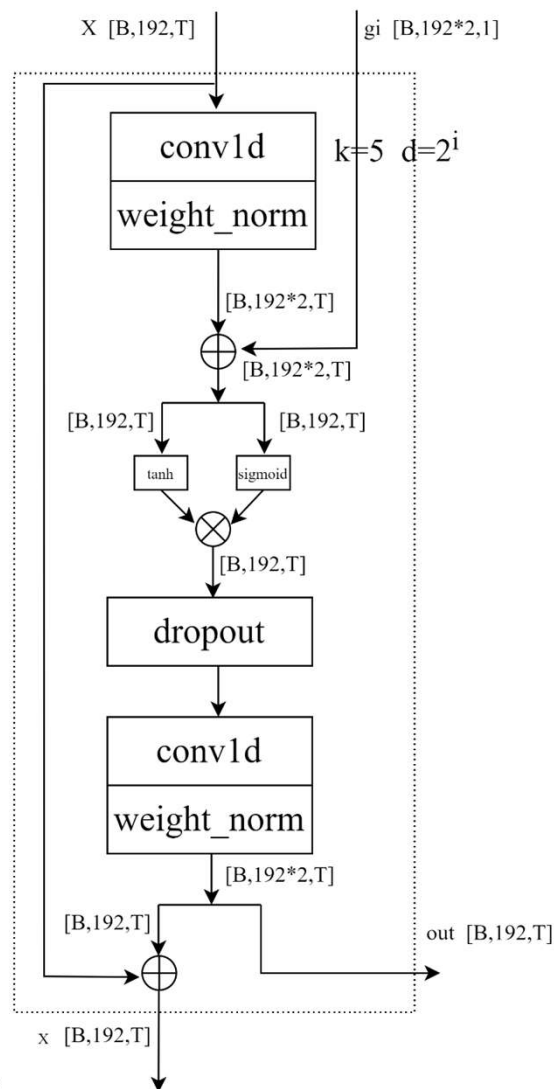
Dilation后kernel的长度为  
 $k + (k-1) * (d-1) = kd - d + 1$   
 为了保证卷积前后长度不变  
 $\text{Padding} = (kd - d) / 2$

2023/9/25

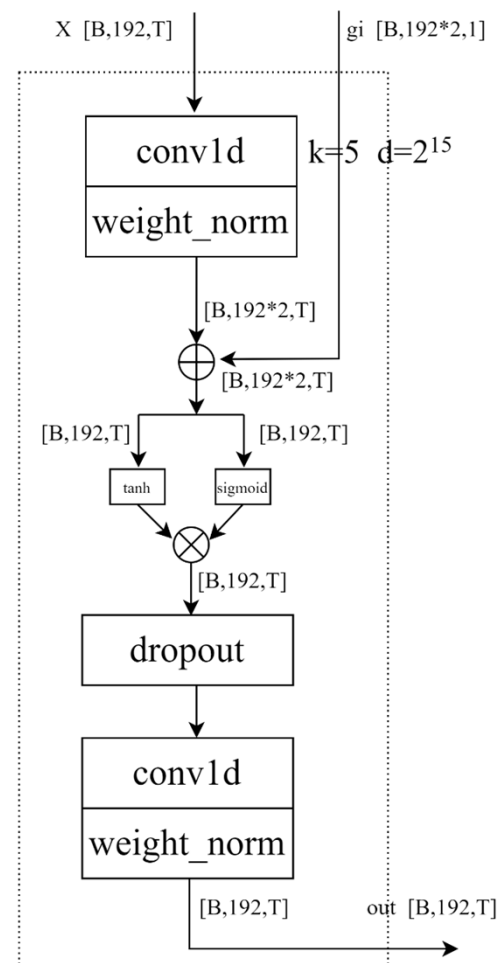
Wavnet 就是由若干个 dilation 指数增加的 卷积层堆叠而成



前面  
若干层

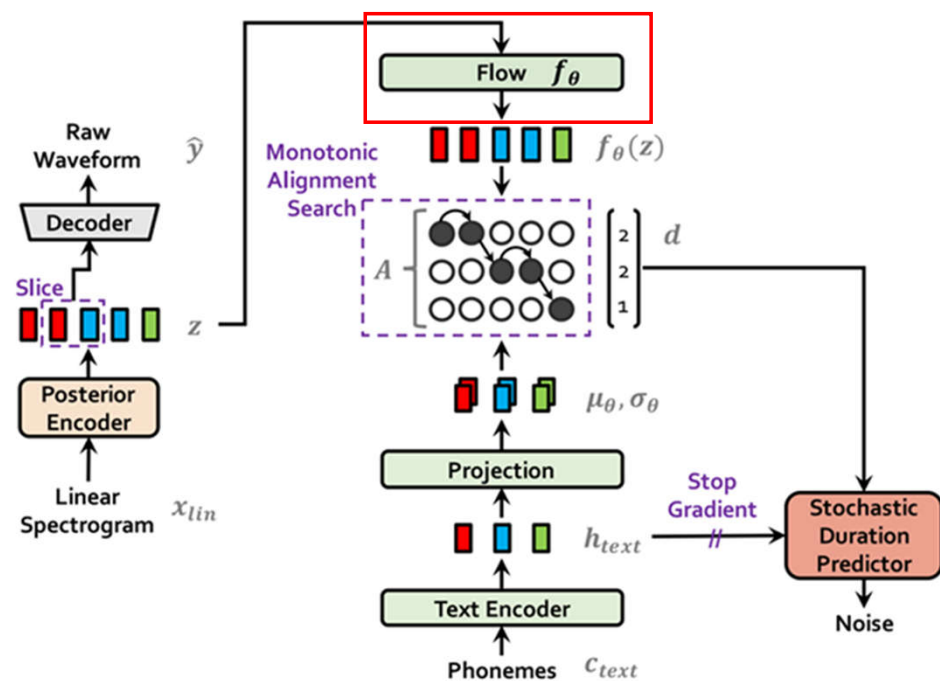


最后一层

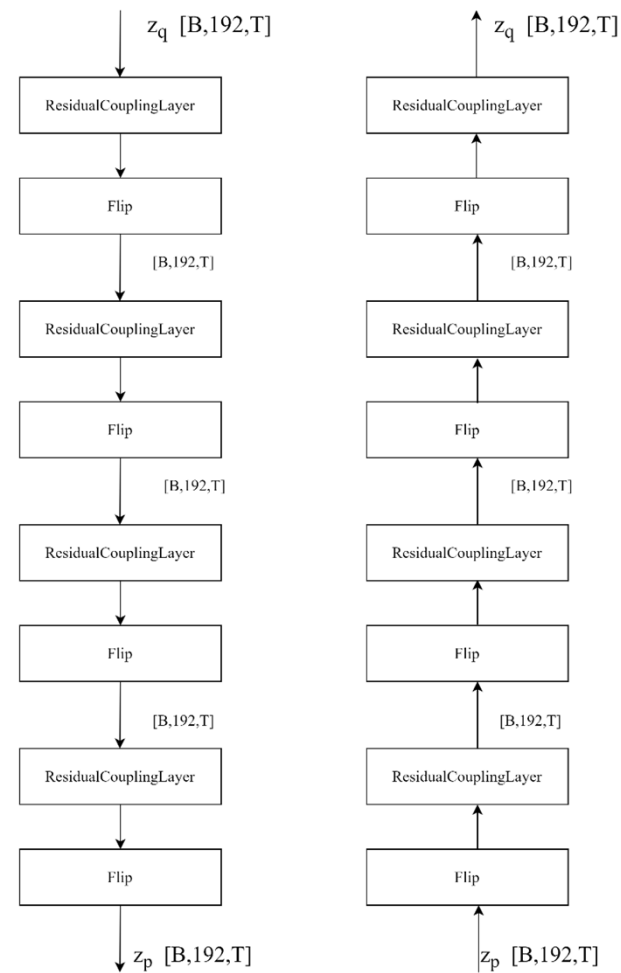


`torch.nn.utils.weight_norm(module, name='weight', dim=0)`

$$\mathbf{w} = g \frac{\mathbf{v}}{\|\mathbf{v}\|}$$



正向



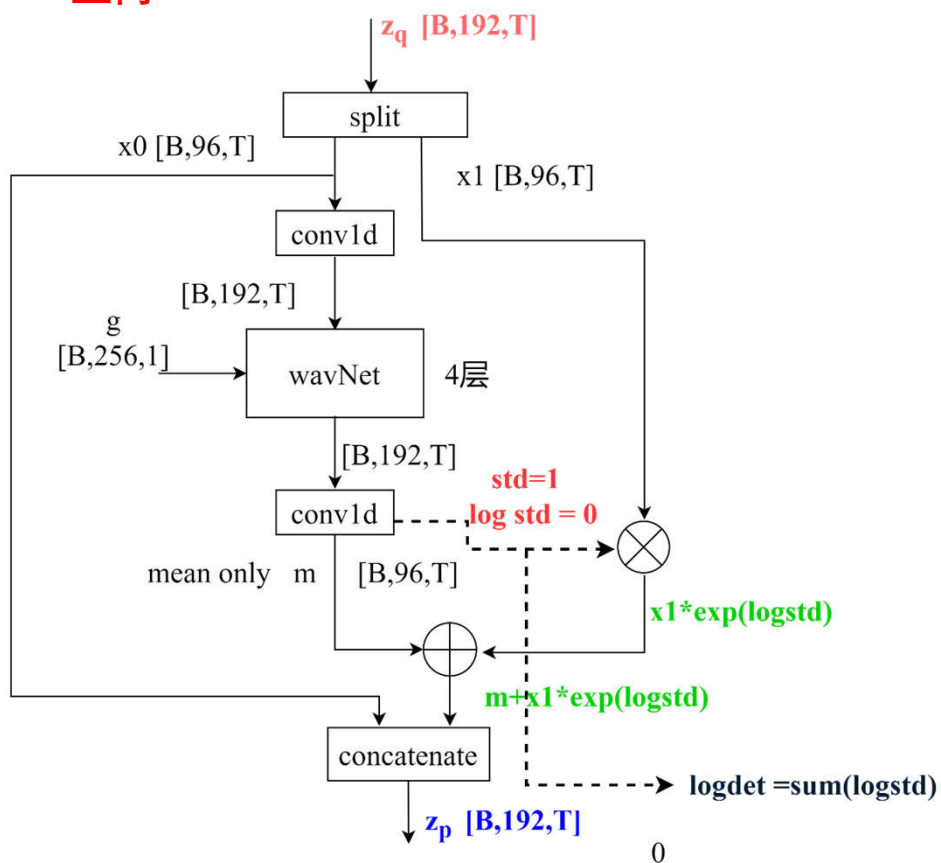
```
class ResidualCouplingBlock(nn.Module):
    def __init__(self,
        channels,
        hidden_channels,
        kernel_size,
        dilation_rate,
        n_layers,
        n_flows=4,
        gin_channels=0):
        super().__init__()
        self.channels = channels
        self.hidden_channels = hidden_channels
        self.kernel_size = kernel_size
        self.dilation_rate = dilation_rate
        self.n_layers = n_layers
        self.n_flows = n_flows
        self.gin_channels = gin_channels

        self.flows = nn.ModuleList()

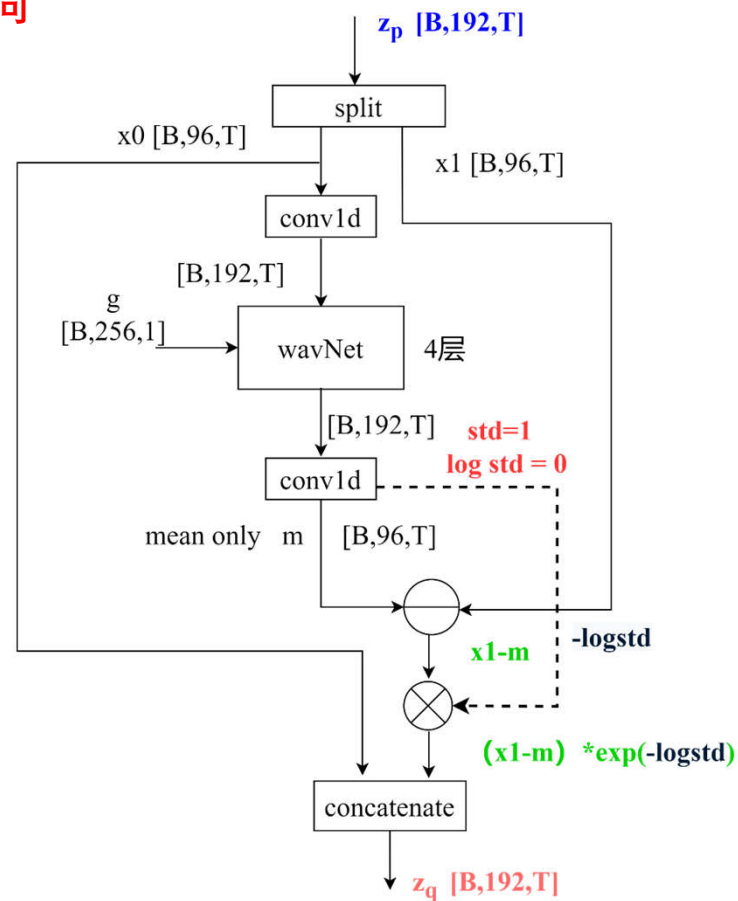
        for i in range(n_flows):
            self.flows.append(modules.ResidualCouplingLayer(channels, hidden_channels, kernel_size,
                self.flows.append(modules.Flip()))

    def forward(self, x, x_mask, g=None, reverse=False):
        if not reverse:
            for flow in self.flows:
                x, _ = flow(x, x_mask, g=g, reverse=reverse)
        else:
            for flow in reversed(self.flows):
                x = flow(x, x_mask, g=g, reverse=reverse)
        return x
```

## 正向



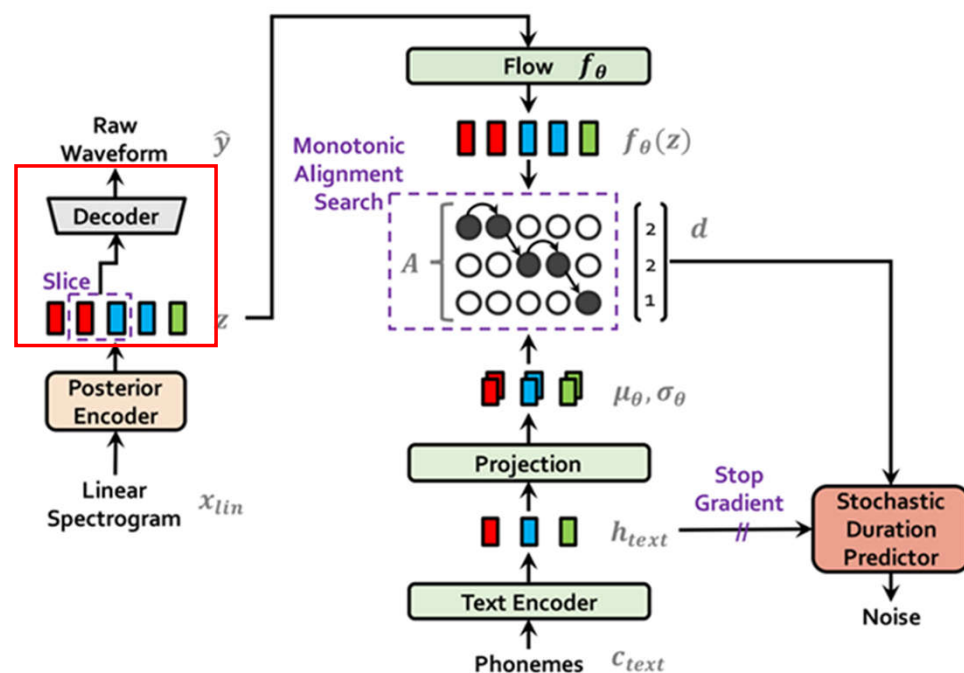
## 反向



B,192,T

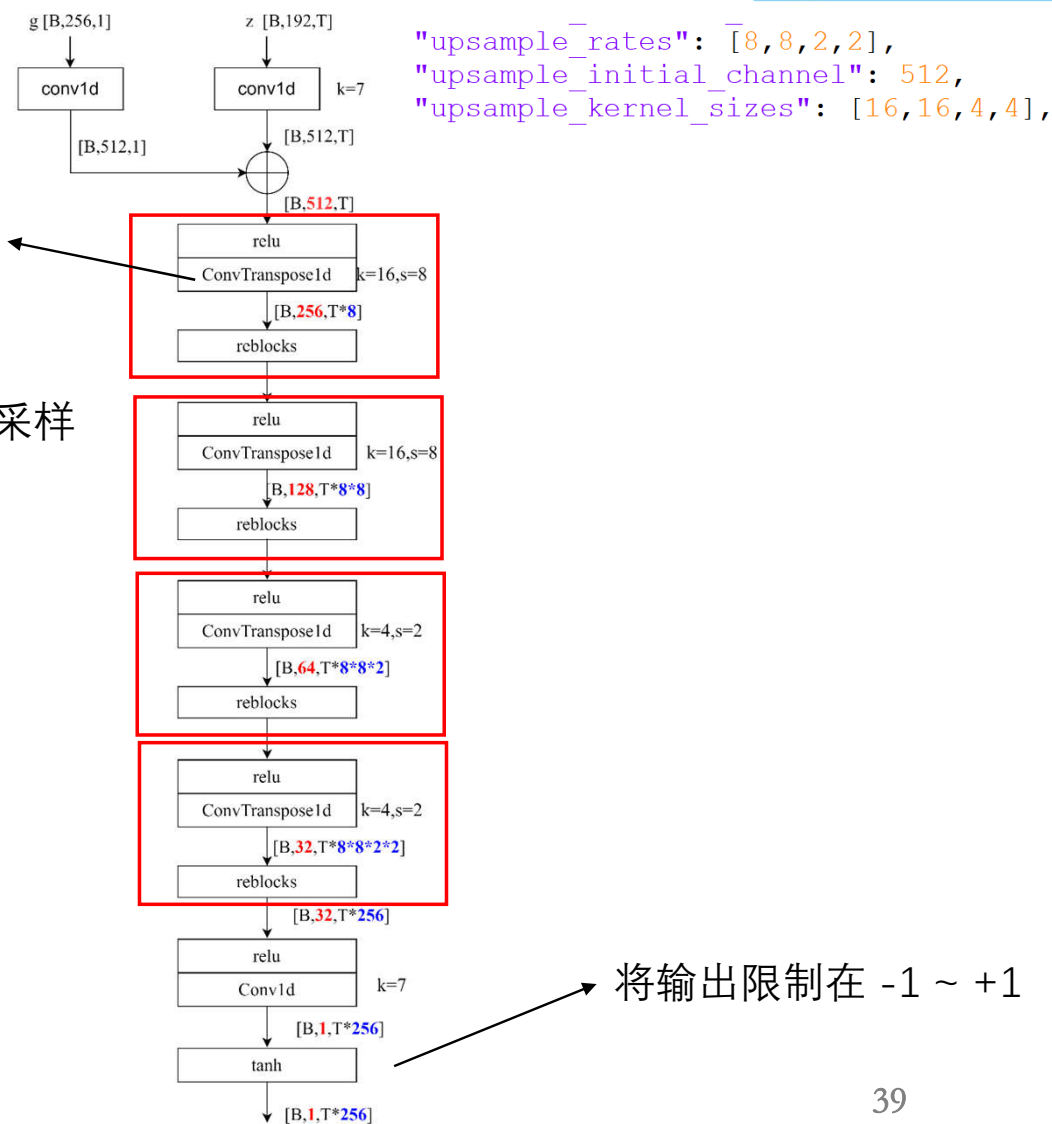
```
class Flip(nn.Module):  
    def forward(self, x, *args, reverse=False, **kwargs):  
        x = torch.flip(x, [1])  
        if not reverse:  
            logdet = torch.zeros(x.size(0)).to(dtype=x.dtype, device=x.device)  
            return x, logdet  
        else:  
            return x
```

训练时对 长度固定的切片进行解码  
测试时对全部的采样值进行解码。

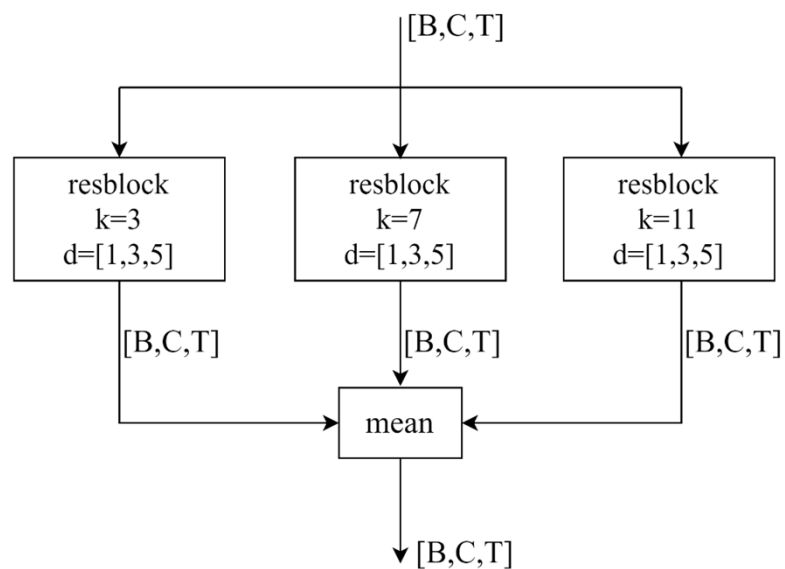


引入  
WeightNormal

上采样



## resblocks 内部

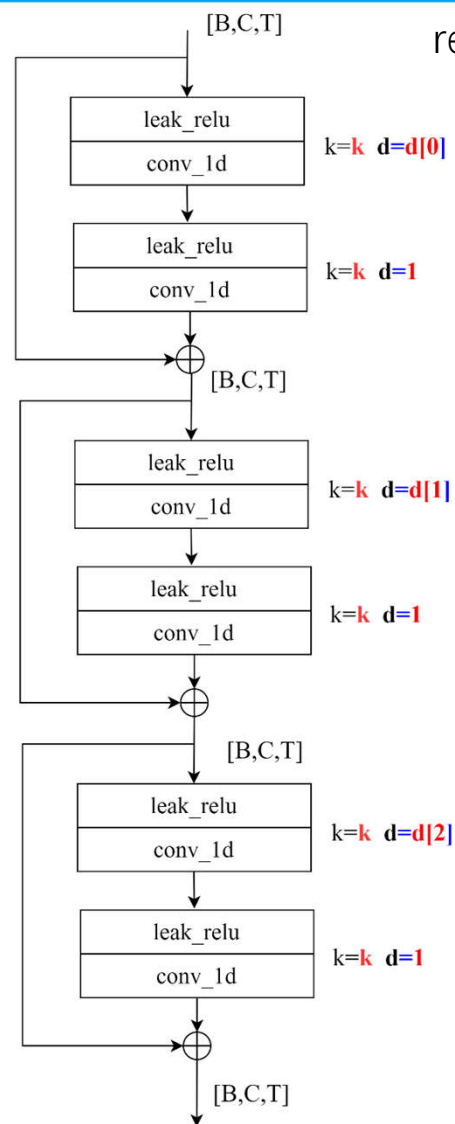


```

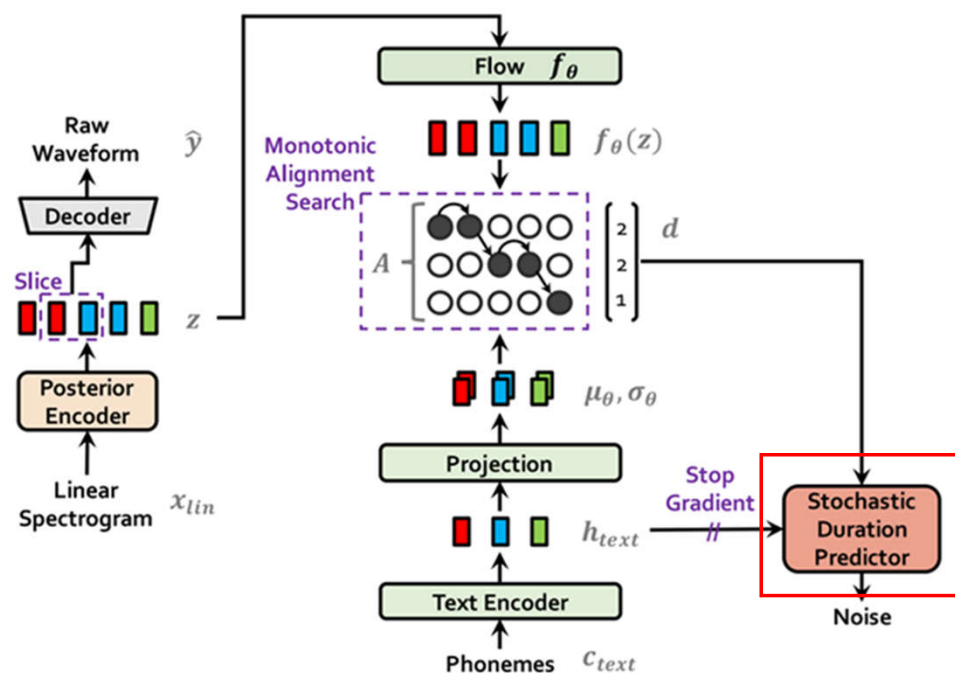
"resblock": "1",
"resblock_kernel_sizes": [3, 7, 11],
"resblock_dilation_sizes": [[1, 3, 5], [1, 3, 5], [1, 3, 5]],

```

## resblock内部



# 随机时长预测器



预测输入文本中的每个symbols的持续时长 $d$ （帧数）

这个时长本来是个整数，但是但是需要使用一个连续模型对其建模。

（离散模型在测试时遇到未知数据似然值趋近负无穷）  
因此设计一去量化的随机变量 $u$ ，对连续数据  $d-u$  进行建模

其中 $u$ 是一个  $(0,1)$  之间的随机分布

$u$ 的分布通过一个flow 来进行建模  $N(0,1) \sim u$

$d-u$ 的分布通过一个flow 来进行映射

Flow++

$d-u \sim N(0,1)$

由于预测的帧数 $d$ 是个1维的变量，所以添加了额外的维度  $v$  来增加建模能力

variational data augmentation

## Flow++: Improving Flow-Based Generative Models with Variational Dequantization and Architecture Design

<https://arxiv.org/abs/1902.00275>

利用离散数据  $x \sim P_{\text{data}}$  对连续模型  $P_{\text{model}}$  进行建模 似然值的数学期望为

$$\begin{aligned}
 & \mathbb{E}_{x \sim P_{\text{data}}} [\log P_{\text{model}}(x)] \\
 &= \mathbb{E}_{x \sim P_{\text{data}}} \left[ \log \int_{[0,1)} q(u|x) \frac{P_{\text{model}}(x-u)}{q(u|x)} du \right] \\
 &\geq \mathbb{E}_{x \sim P_{\text{data}}} \left[ \int_{[0,1)} q(u|x) \log \frac{P_{\text{model}}(x-u)}{q(u|x)} du \right] \\
 &= \mathbb{E}_{x \sim P_{\text{data}}} \mathbb{E}_{u \sim q(\cdot|x)} \left[ \log \frac{P_{\text{model}}(x-u)}{q(u|x)} \right]
 \end{aligned}$$

对应VITS论文

$$\log p_{\theta}(d|c_{\text{text}}) \geq \mathbb{E}_{q_{\phi}(u, \nu|d, c_{\text{text}})} \left[ \log \frac{p_{\theta}(d-u, \nu|c_{\text{text}})}{q_{\phi}(u, \nu|d, c_{\text{text}})} \right]$$



$$\log p_{\theta}(d|c_{text}) \geq$$

$$\mathbb{E}_{q_{\phi}(u, \nu|d, c_{text})} \left[ \log \frac{p_{\theta}(d - u, \nu|c_{text})}{q_{\phi}(u, \nu|d, c_{text})} \right]$$

这里只考虑u

利用flow建模

$$z \sim p(z) \quad p(z) = N(z; 0, 1) \quad u \sim q_{\phi}(u)$$

$$z \xrightarrow{G(\cdot)} u$$

$$u \xrightarrow{G^{-1}(\cdot)} z$$

取 log

$$p(z) = q_{\phi}(G(z)) \left| \frac{\partial(G(z))}{\partial(z)} \right|$$

$$q_{\phi}(u) = p(G^{-1}(u)) \left| \frac{\partial(G^{-1}(u))}{\partial(u)} \right|$$

$$= p(z) \left| \frac{\partial(G(z))}{\partial z} \right|^{-1}$$

$$\log(q_{\phi}(u)) = \log p(z) - \log \left( \left| \frac{\partial(G(z))}{\partial z} \right| \right)$$

$$\log p_{\theta}(d|c_{text}) \geq$$

$$\mathbb{E}_{q_{\phi}(u, \nu|d, c_{text})} \left[ \log \frac{p_{\theta}(d - u, \nu|c_{text})}{q_{\phi}(u, \nu|d, c_{text})} \right]$$

只考虑 d-u 设 d-u = y

$$y \sim p_{\theta}(y) \quad \varepsilon \sim Q(\varepsilon) \quad Q(\varepsilon) = N(\varepsilon; 0, 1)$$

$$y \xrightarrow{f(\cdot)} \varepsilon$$

$$p_{\theta}(y) = Q(f(y)) \left| \frac{\partial f(y)}{\partial y} \right| \quad \text{取log} \quad \log p_{\theta}(y) = \log Q(f(y)) + \log \left| \frac{\partial f(y)}{\partial y} \right|$$

$$\log p_{\theta}(d|c_{text}) \geq \mathbb{E}_{q_{\phi}(u, \nu|d, c_{text})} \left[ \log \frac{p_{\theta}(d - u, \nu|c_{text})}{q_{\phi}(u, \nu|d, c_{text})} \right]$$

$$\begin{aligned} &\rightarrow \log p_{\theta}(y) = \log Q(f(y)) + \log \left| \frac{\partial f(y)}{\partial y} \right| \\ &\rightarrow \log(q_{\phi}(u)) = \log p(z) - \log \left( \left| \frac{\partial(G(z))}{\partial z} \right| \right) \end{aligned}$$

整体  $-\text{loss} = \log Q(f(y)) + \log \left| \frac{\partial f(y)}{\partial y} \right| - \log p(z) + \log \left( \left| \frac{\partial(G(z))}{\partial z} \right| \right)$

$$N(x; 0, 1) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

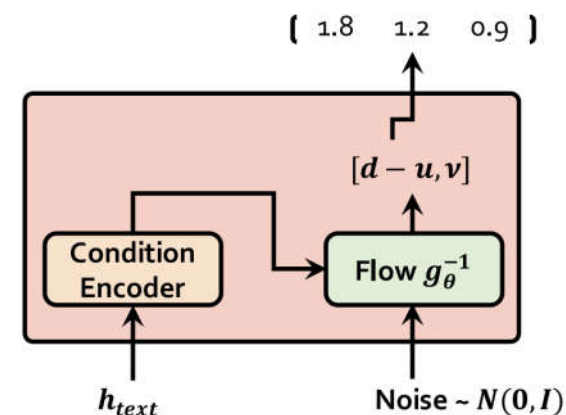
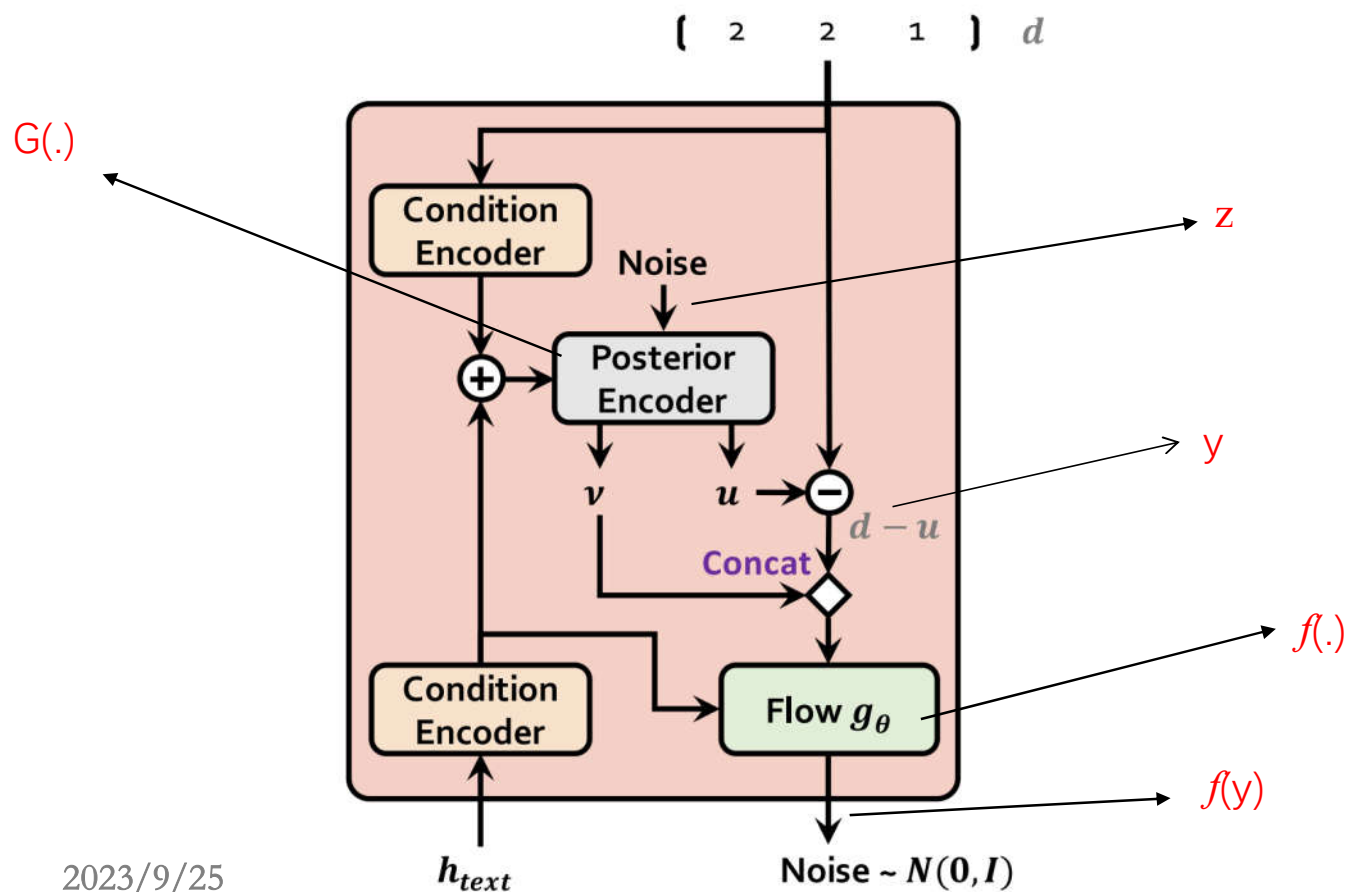
$$\log(x) = -0.5(\log 2\pi + x^2)$$

$$\text{loss} = \log p(z) - \log \left( \left| \frac{\partial(G(z))}{\partial z} \right| \right) - \log Q(f(y)) - \log \left| \frac{\partial f(y)}{\partial y} \right|$$

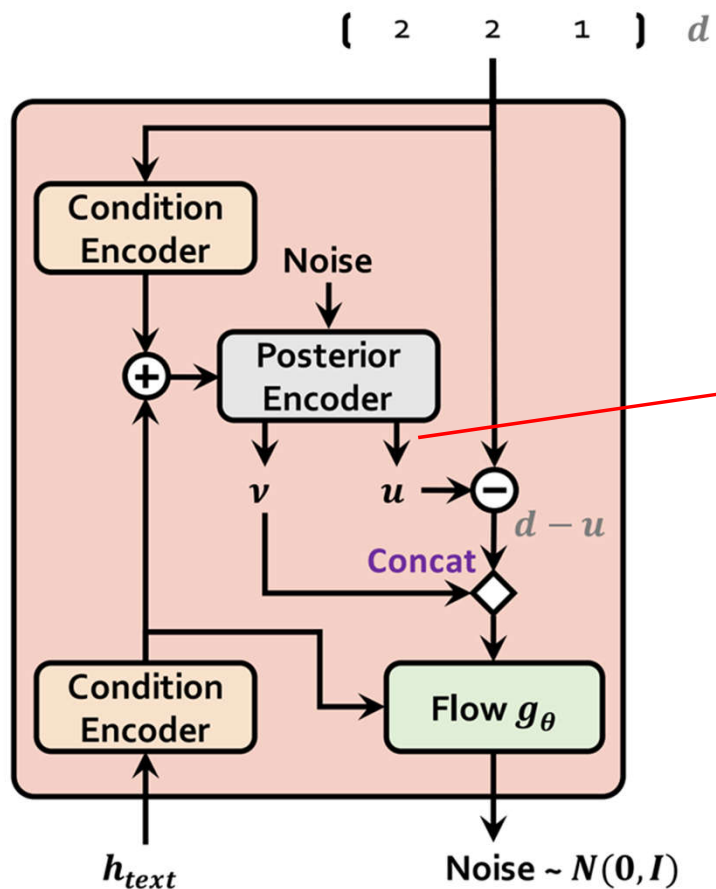
$$N(0, 1)$$

$$\text{loss} = -0.5(\log 2\pi + z^2) - \log \left( \left| \frac{\partial(G(z))}{\partial z} \right| \right) + 0.5(\log 2\pi + f(y)^2) - \log \left| \frac{\partial f(y)}{\partial y} \right|$$

$$\text{loss} = -0.5(\log 2\pi + z^2) - \log \left( \left| \frac{\partial(G(z))}{\partial z} \right| \right) + 0.5(\log 2\pi + f(y)^2) - \log \left| \frac{\partial f(y)}{\partial y} \right|$$



推理部分



引入了 sigmoid 函数  $u = \text{sigmoid}(z) = \frac{1}{1 + e^{-z}}$

$$\left| \frac{\partial u}{\partial z} \right| = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \frac{e^{-z}}{1 + e^{-z}} = \frac{1}{1 + e^{-z}} \frac{1}{e^z + 1}$$

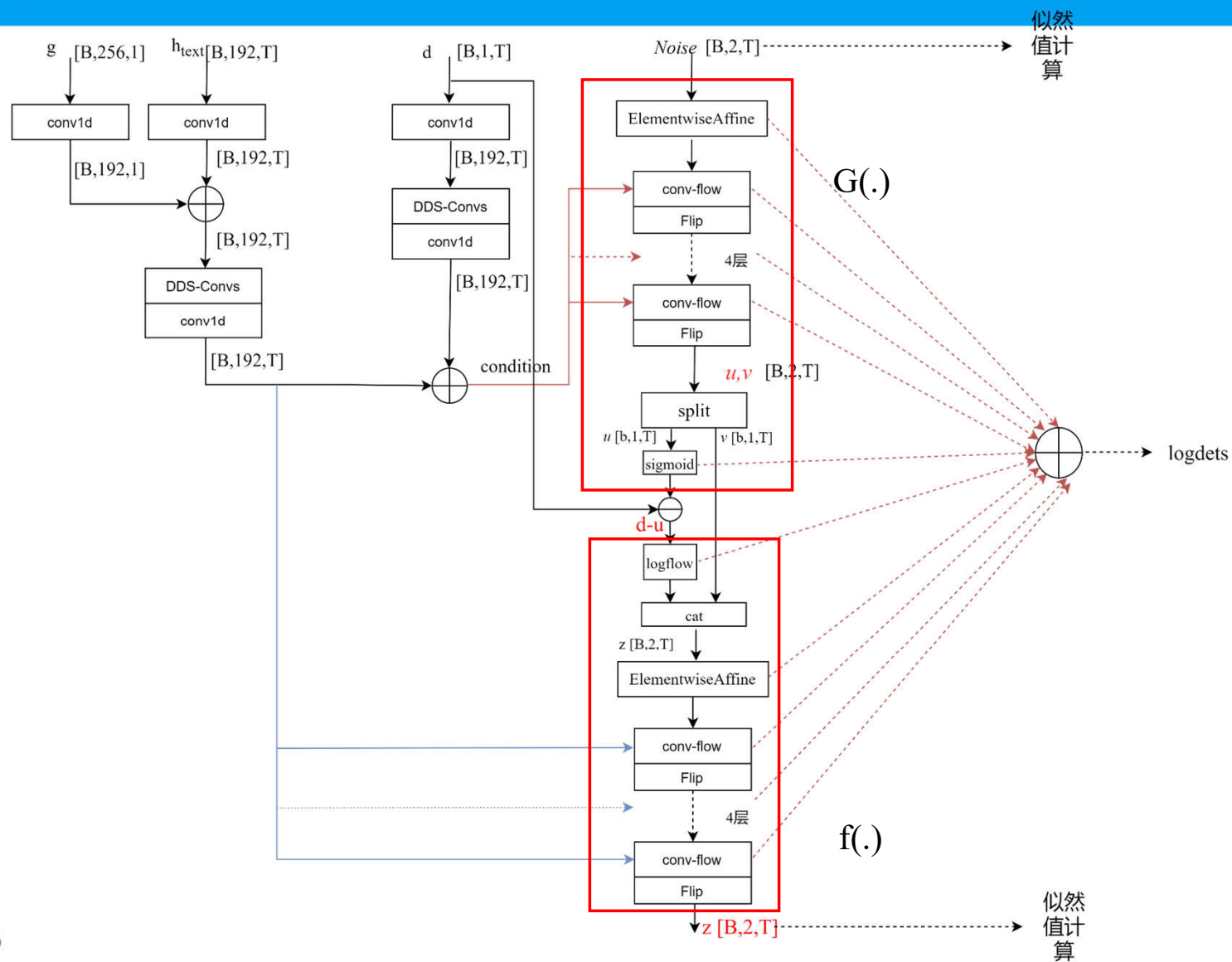
$$= \text{sigmoid}(z) \text{sigmoid}(-z)$$

$$\log \left| \frac{\partial u}{\partial z} \right| = \log \text{sigmoid}(z) + \log \text{sigmoid}(-z)$$

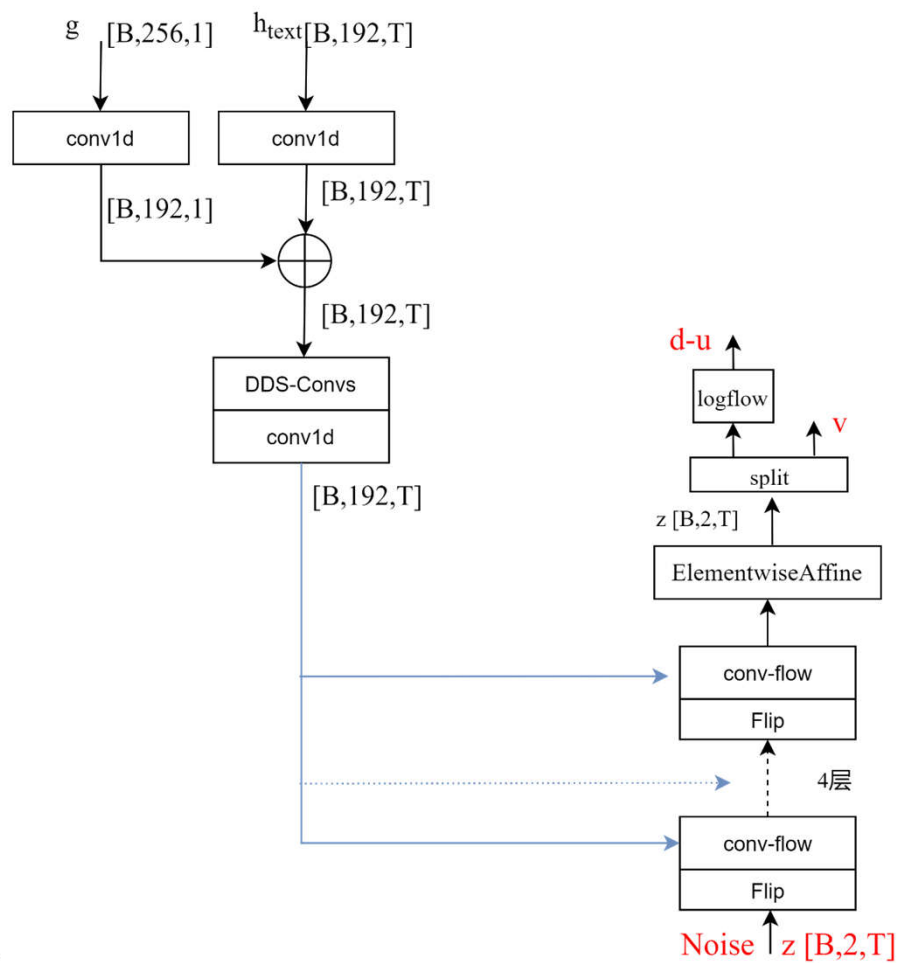
models.py 第75行

## 模型整体结构

models.py  
17行



## 反向推理过程

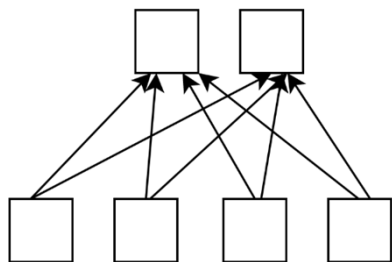


## DDScnv (Dilated and depth-wise separable convolutional residual block)

卷积中 group 的概念:  $\text{group} = k$   
 $\text{in\_channel}$  和  $\text{out\_channel}$  都能被  $k$  整除

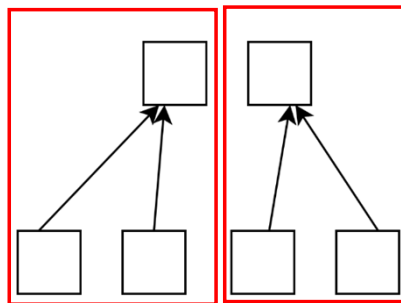
$\text{group} = 1$   $\text{in\_channel} = 4$   $\text{out\_channel} = 2$

一个卷积层 需要  $2 \times 4 = 8$  个 kernel



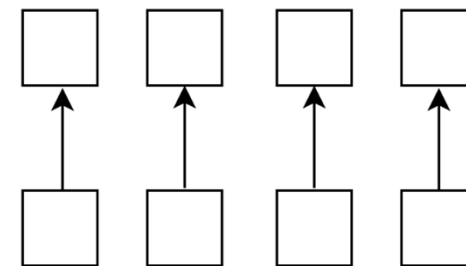
$\text{group} = 2$

一个卷积层 需要  $2 \times (2 \times 1) = 4$  个 kernel

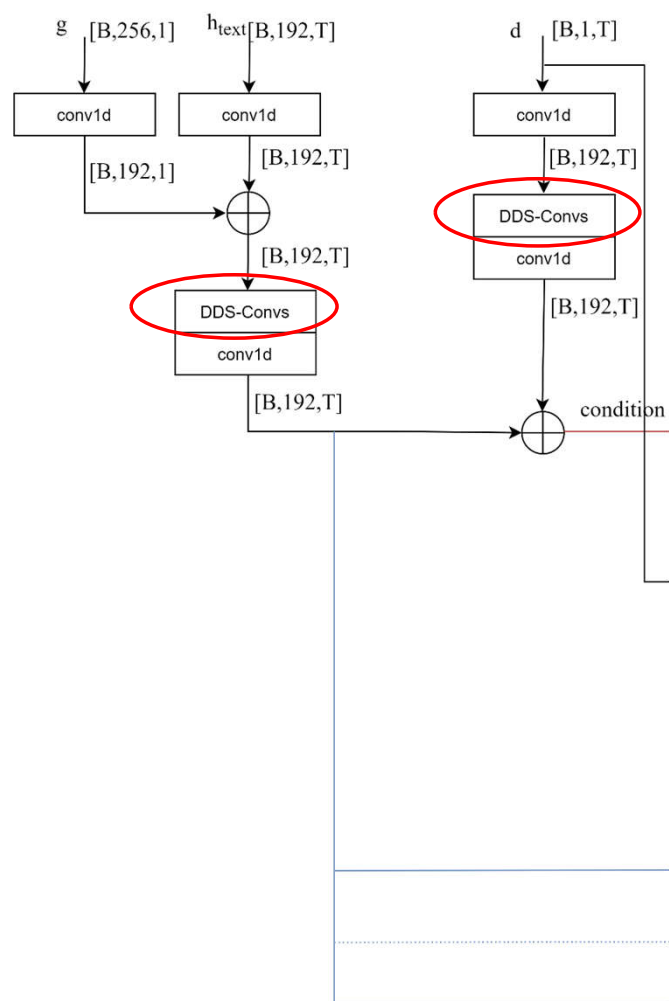
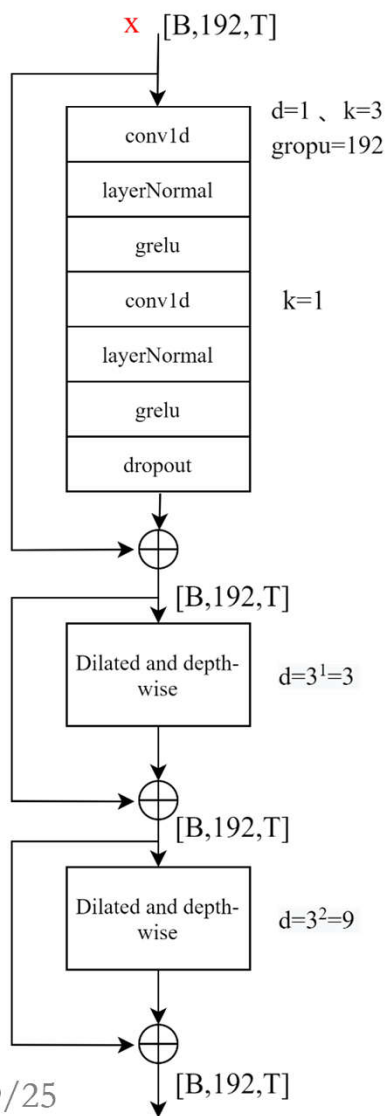


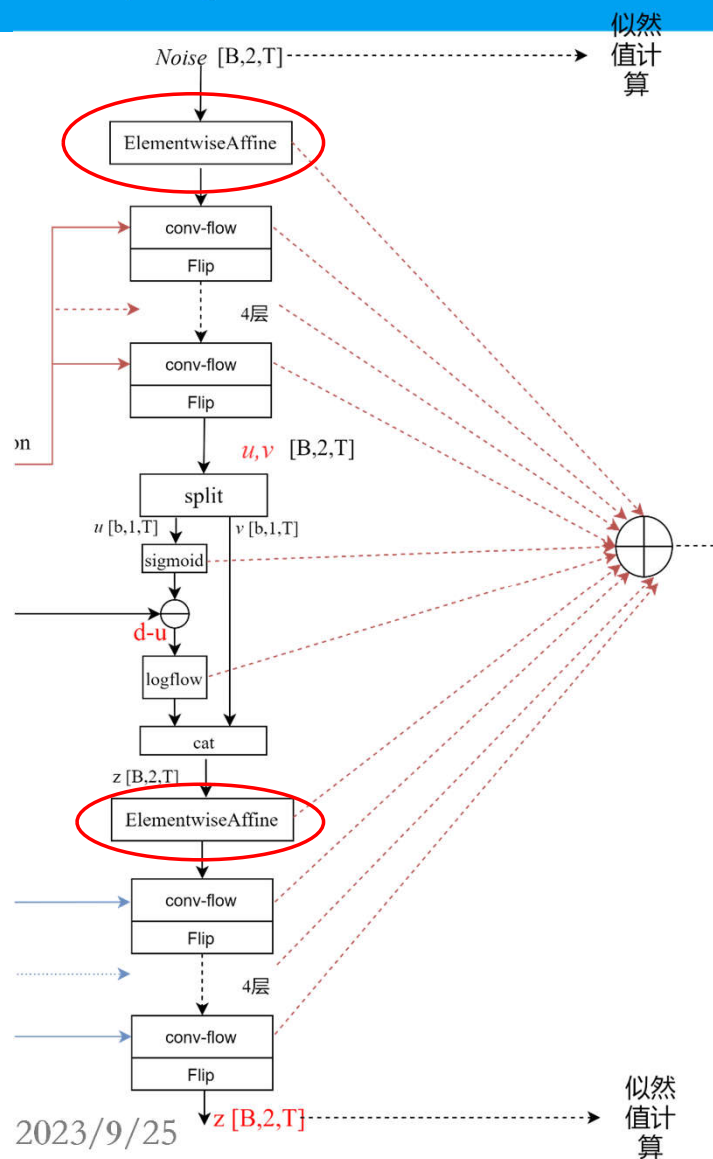
特殊情况

$\text{group} = \text{in\_channel}$   
**depth-wise convolution**



通过 group 可以减少模型参数、减少过拟合

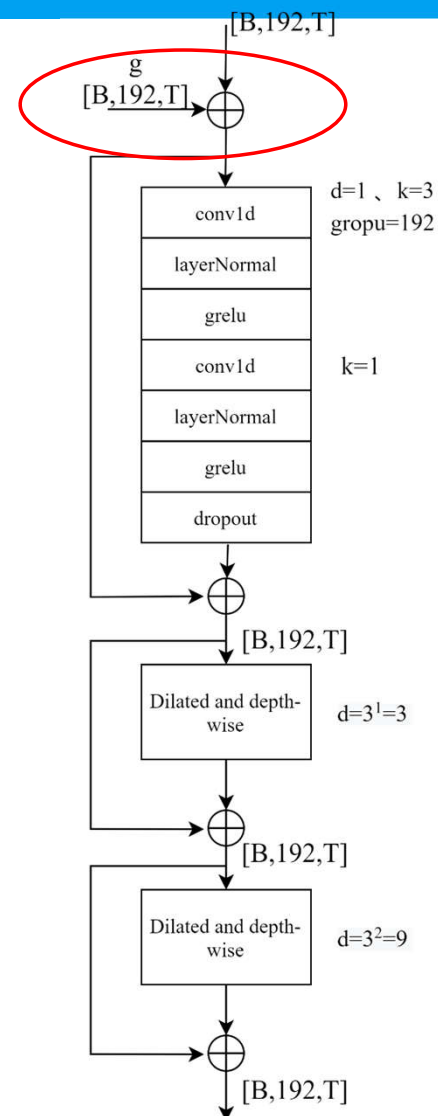
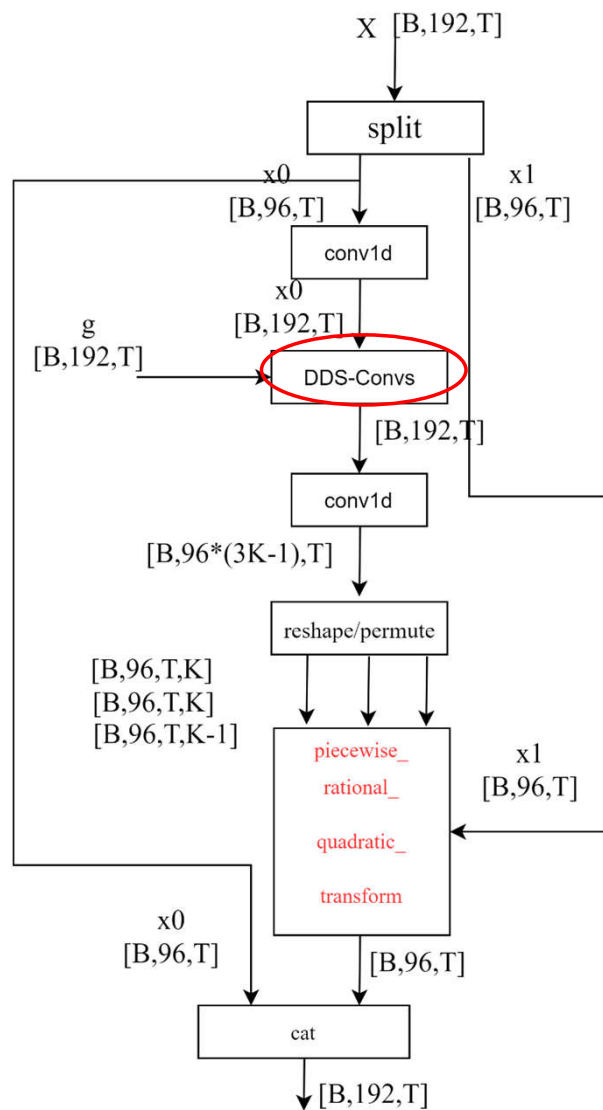
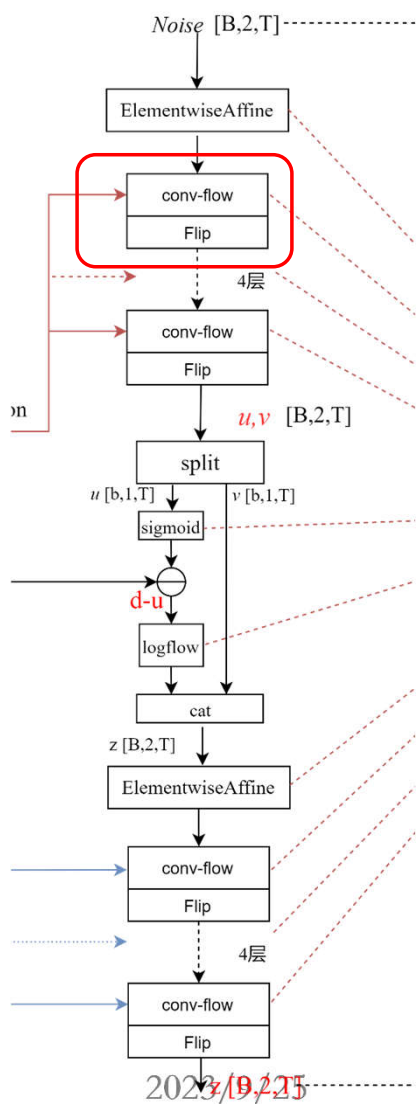


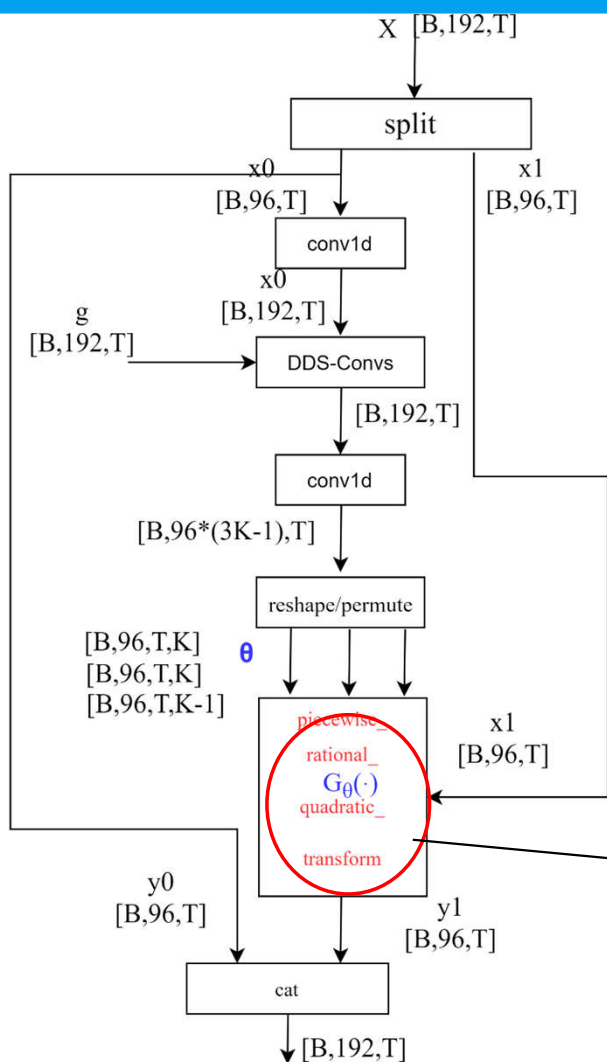


```
class ElementwiseAffine(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.channels = channels
        self.m = nn.Parameter(torch.zeros(channels,1))
        self.logs = nn.Parameter(torch.zeros(channels,1))

    def forward(self, x, x_mask, reverse=False, **kwargs):
        if not reverse:
            y = self.m + torch.exp(self.logs) * x
            y = y * x_mask
            logdet = torch.sum(self.logs * x_mask, [1,2])
            return y, logdet
        else:
            x = (x - self.m) * torch.exp(-self.logs) * x_mask
            return x
```

=2





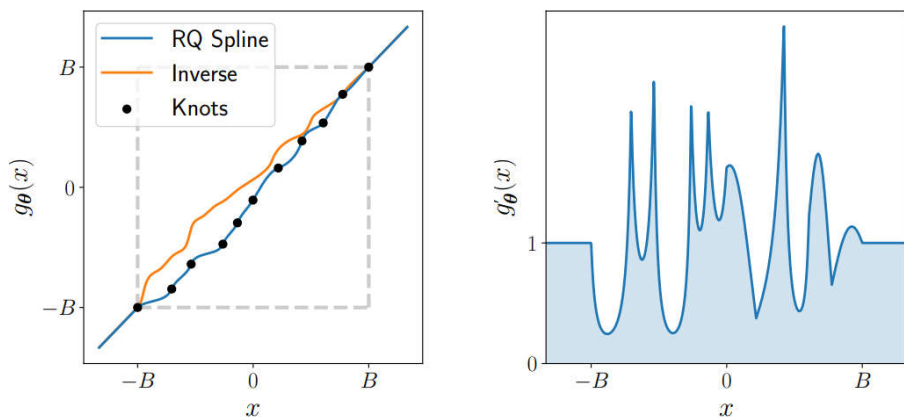
coupling transforms 的基本流程

- (1) 将输入数据  $X$  分割成  $X_0$   $X_1$  两部分
- (2) 利用神经网络从  $X_0$  中学取一组参数  $\theta$   
 $\theta = \text{NN}(X_0)$
- (3) 使用以  $\theta$  为参数的函数  $G_\theta(\cdot)$  对  $X_1$  进行处理得到  $Y_1$   
 $Y_1 = G_\theta(X_1)$
- (4)  $Y_0 = X_0$ , 将  $Y_0$   $Y_1$  进行拼接得到输出  $Y$

普通的  $G_\theta(\cdot)$  的形式为:

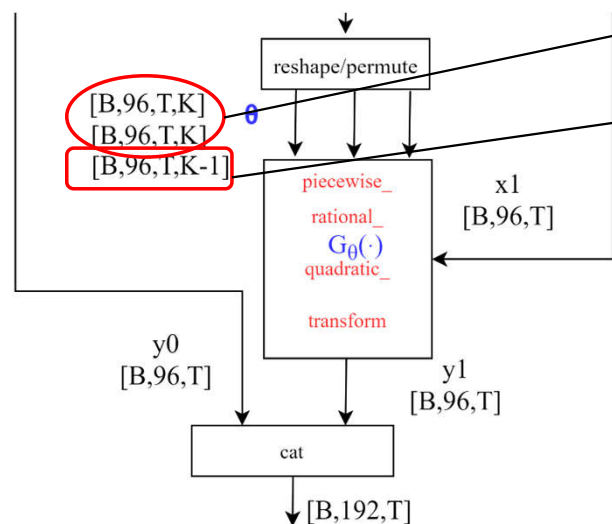
$$g_{\theta_i}(x_i) = \alpha_i x_i + \beta_i, \quad \text{where } \theta_i = \{\alpha_i, \beta_i\}$$

设计了一组分段函数，对不同的输入，采用不同的参数计算输出



- (1) 当 $x$ 取值 $[-B, B]$ 时 (VITS中设置 $B=5$ )  
输出 $y$ 的取值也是 $[-B, B]$ ; 当 $x$ 超出范围后  
 $y=x$
- (2)  $x \sim [-B, B]$ 分成 $K$ 段, 对应的输出 $y$ 也分成 $K$ 段  
当输入  $x \sim [x^k, x^{k+1}]$ 时 对应的输出  $y \sim [y^k, y^{k+1}]$
- (3)  $x^k$  与  $y^k$  是通过神经网络学习得到的

将输出经过 softmax后再进行累加就可获取



- (4) 每一段的转换函数中还需要用到一个非负参数  
 $\delta^k$  是通过 softplus 函数得到的

$$w^{(k)} = x^{(k+1)} - x^{(k)}$$

$$s^{(k)} = (y^{(k+1)} - y^{(k)})/w^{(k)}$$

$$\xi = (x - x^{(k)})/w^{(k)}$$

$$g(x) = \frac{\alpha^{(k)}(\xi)}{\beta^{(k)}(\xi)}$$

变换公式

$$\frac{\alpha^{(k)}(\xi)}{\beta^{(k)}(\xi)} = y^{(k)} + \frac{(y^{(k+1)} - y^{(k)})[s^{(k)}\xi^2 + \delta^{(k)}\xi(1 - \xi)]}{s^{(k)} + [\delta^{(k+1)} + \delta^{(k)} - 2s^{(k)}]\xi(1 - \xi)}$$

导数

$$\frac{d}{dx} \left[ \frac{\alpha^{(k)}(\xi)}{\beta^{(k)}(\xi)} \right] = \frac{(s^{(k)})^2 [\delta^{(k+1)}\xi^2 + 2s^{(k)}\xi(1 - \xi) + \delta^{(k)}(1 - \xi)^2]}{[s^{(k)} + [\delta^{(k+1)} + \delta^{(k)} - 2s^{(k)}]\xi(1 - \xi)]^2}$$

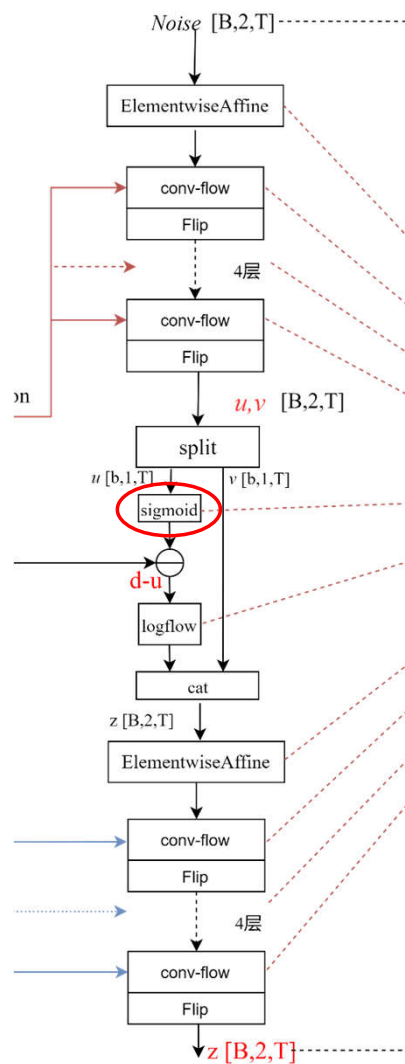
逆变换

$$\xi(x) = \frac{-b + \sqrt{b^2 - 4ac}}{2a} = \frac{2c}{-b - \sqrt{b^2 - 4ac}};$$

$$a = (y^{(k+1)} - y^{(k)})[s^{(k)} - \delta^{(k)}] + (y - y^{(k)})[\delta^{(k+1)} + \delta^{(k)} - 2s^{(k)}]$$

$$b = (y^{(k+1)} - y^{(k)})\delta^{(k)} - (y - y^{(k)})[\delta^{(k+1)} + \delta^{(k)} - 2s^{(k)}],$$

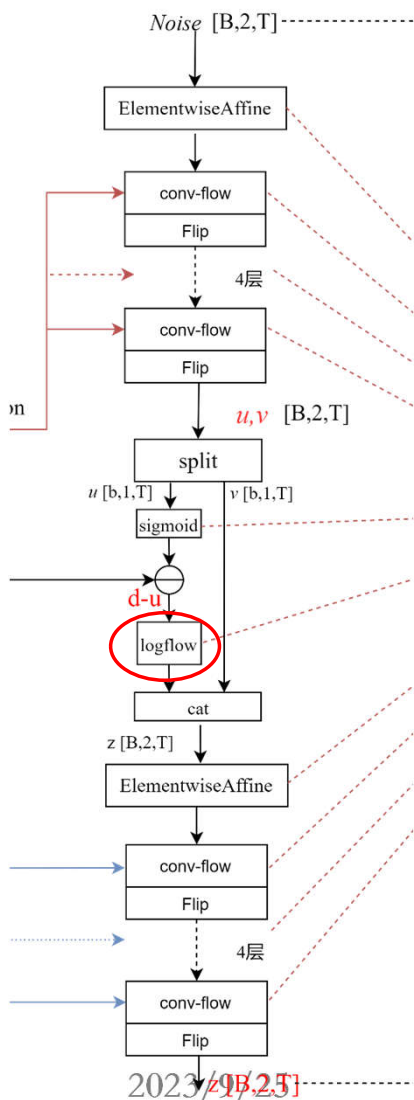
$$c = -s^{(k)}(y - y^{(k)}),$$



```

u = torch.sigmoid(z_u) * x_mask
z0 = (w - u) * x_mask
logdet_tot_q += torch.sum((F.logsigmoid(z_u) + F.logsigmoid(-z_u)) * x_mask, [1,2])

```



```
class Log(nn.Module):
    def forward(self, x, x_mask, reverse=False, **kwargs):
        if not reverse:
            y = torch.log(torch.clamp_min(x, 1e-5)) * x_mask
            logdet = torch.sum(-y, [1, 2])
            return y, logdet
        else:
            x = torch.exp(x) * x_mask
            return x
```

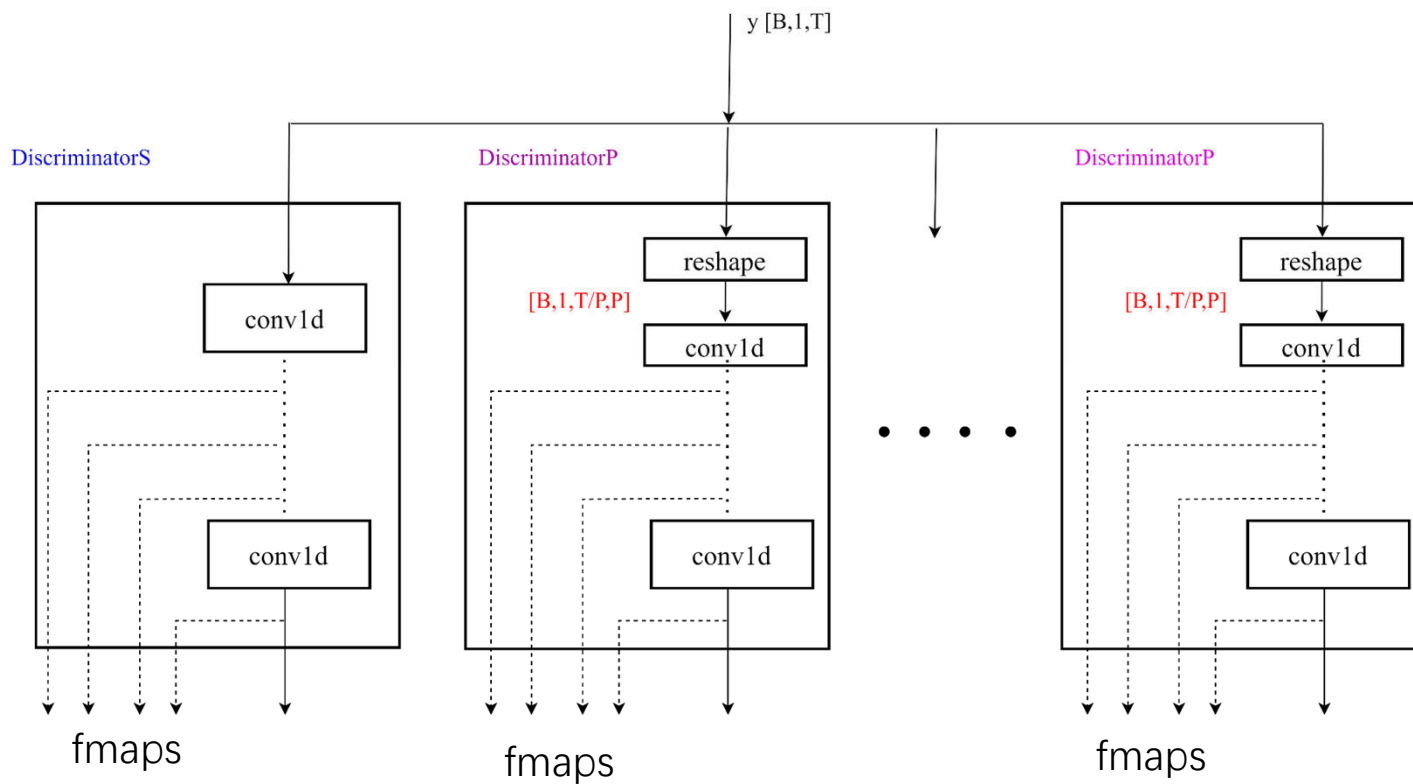
$$y = \log(x)$$

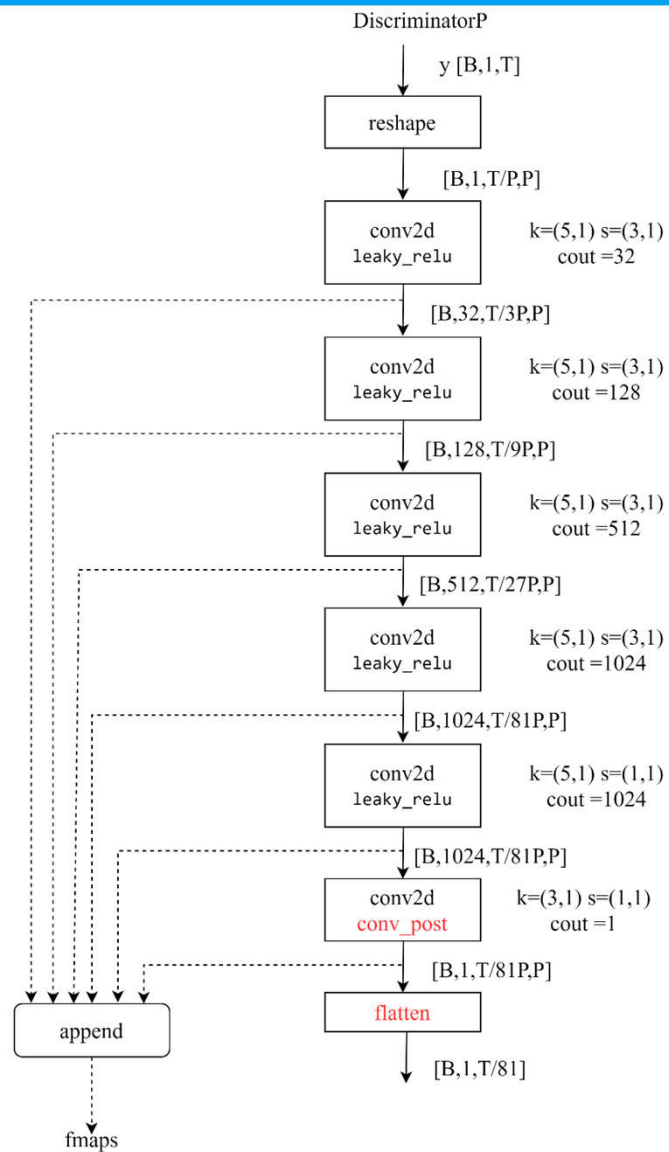
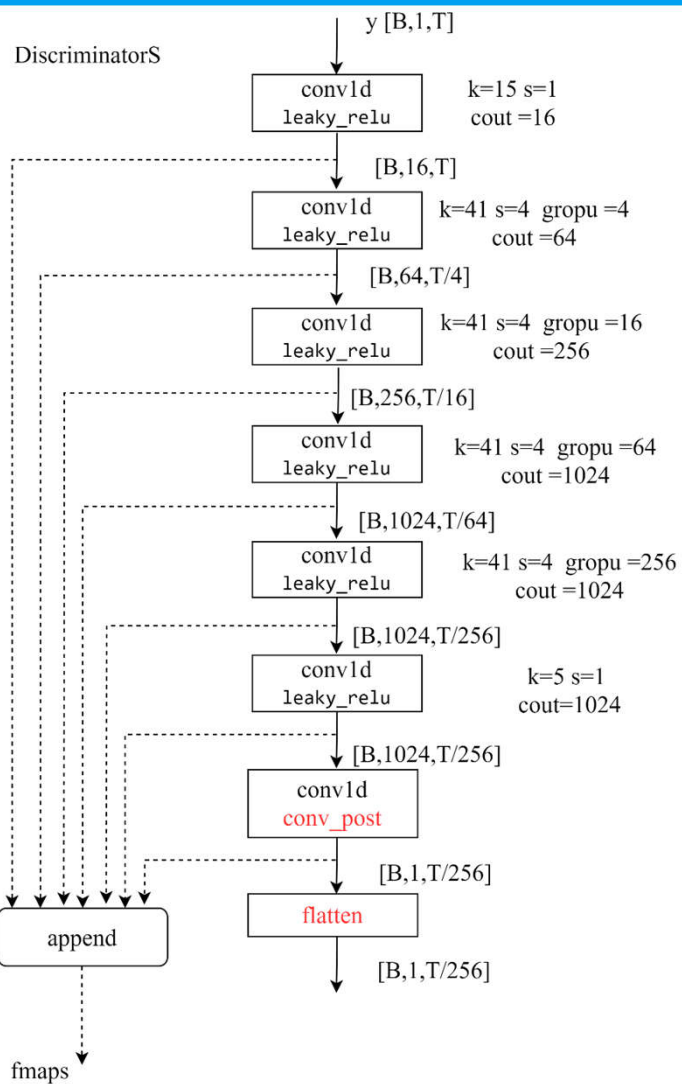
$$\frac{dy}{dx} = \frac{1}{x}$$

$$\log\left(\frac{dy}{dx}\right) = \log\left(\frac{1}{x}\right) = -\log(x) = -y$$

## 鉴别器 (Discriminator)

## MultiPeriodDiscriminator 多尺度鉴别器


$$P=[2,3,5,7,11]$$



2023/9/25

损失函数计算:

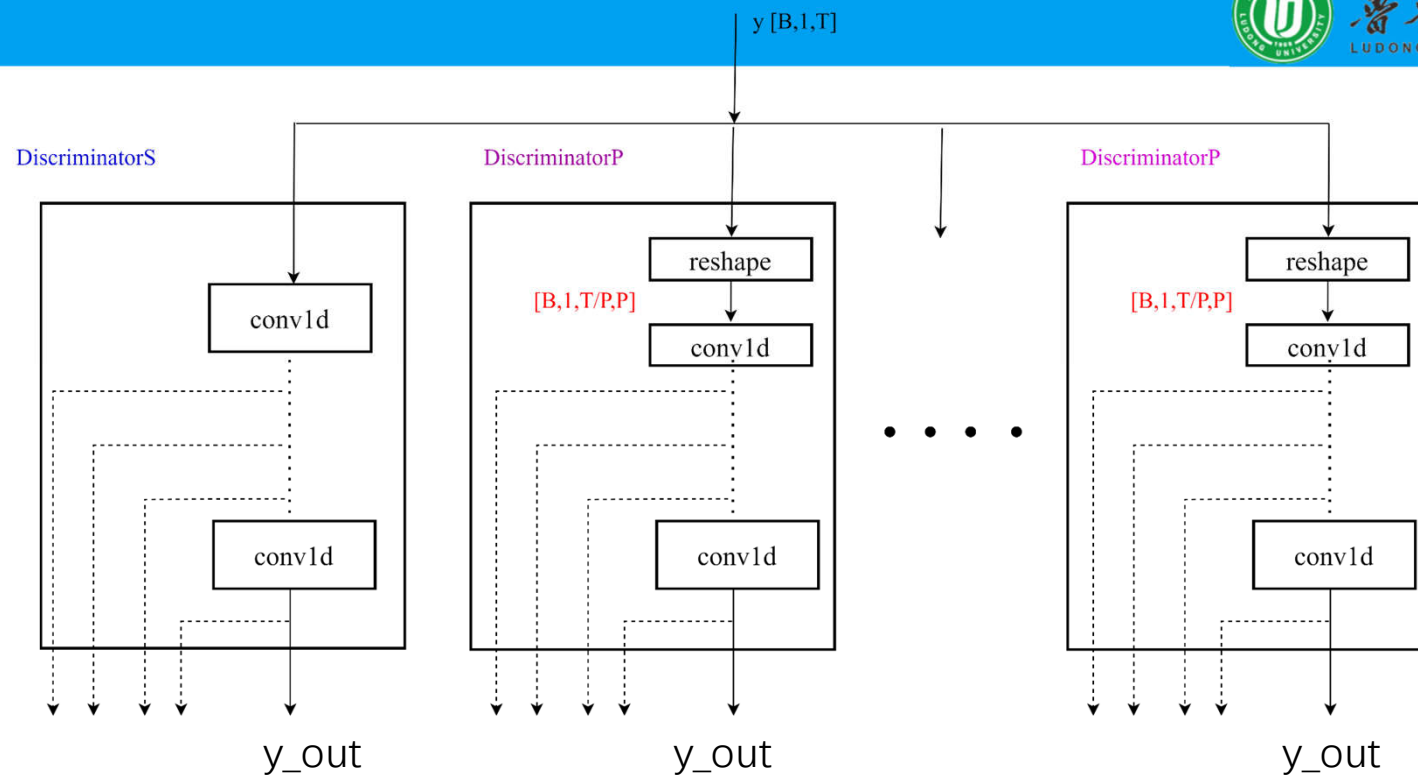
更新D时

$$\text{mean}(y_r - 1)^2 + \text{mean}(y_g)^2$$

更新G时:

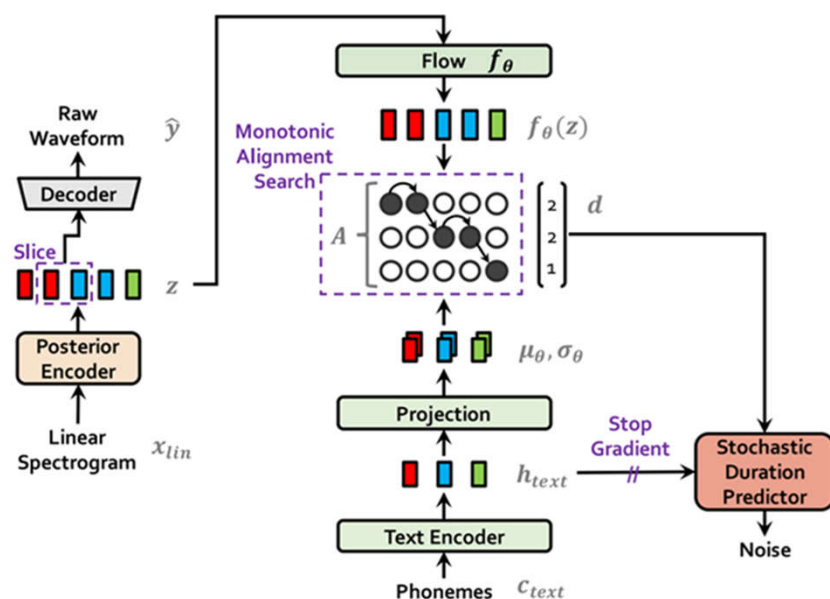
$$\text{mean}(y_g - 1)^2$$

$$\text{mean}(\text{abs}(\text{fmaps}_r - \text{fmaps}_g))$$



输入  $y_{real}$  输出  $y_{out} = y_r$   
 输入  $y_{gen}$  输出  $y_{out} = y_g$

# 生成器工作流程



models.py 390行

```
class SynthesizerTrn(nn.Module):
    """
    Synthesizer for Training
    """

    def __init__(self,
        n_vocab,
        spec_channels,
        segment_size,
        inter_channels,
        hidden_channels,
        filter_channels,
        n_heads,
        n_layers,
        kernel_size,
        p_dropout,
        resblock,
        resblock_kernel_sizes,
        resblock_dilation_sizes,
        upsample_rates,
        upsample_initial_channel,
        upsample_kernel_sizes,
        n_speakers=0,
        gin_channels=0,
        use_sdp=True,
        **kwargs):
        super().__init__()
        self.n_vocab = n_vocab
        self.spec_channels = spec_channels
        self.inter_channels = inter_channels
        self.hidden_channels = hidden_channels
        self.filter_channels = filter_channels
        self.n_heads = n_heads
        self.n_layers = n_layers
        self.kernel_size = kernel_size
        self.p_dropout = p_dropout
        self.resblock = resblock
        self.resblock_kernel_sizes = resblock_kernel_sizes
        self.resblock_dilation_sizes = resblock_dilation_sizes
        self.upsample_rates = upsample_rates
        self.upsample_initial_channel = upsample_initial_channel
        self.upsample_kernel_sizes = upsample_kernel_sizes
        self.segment_size = segment_size
        self.n_speakers = n_speakers
        self.gin_channels = gin_channels
        self.use_sdp = use_sdp
```

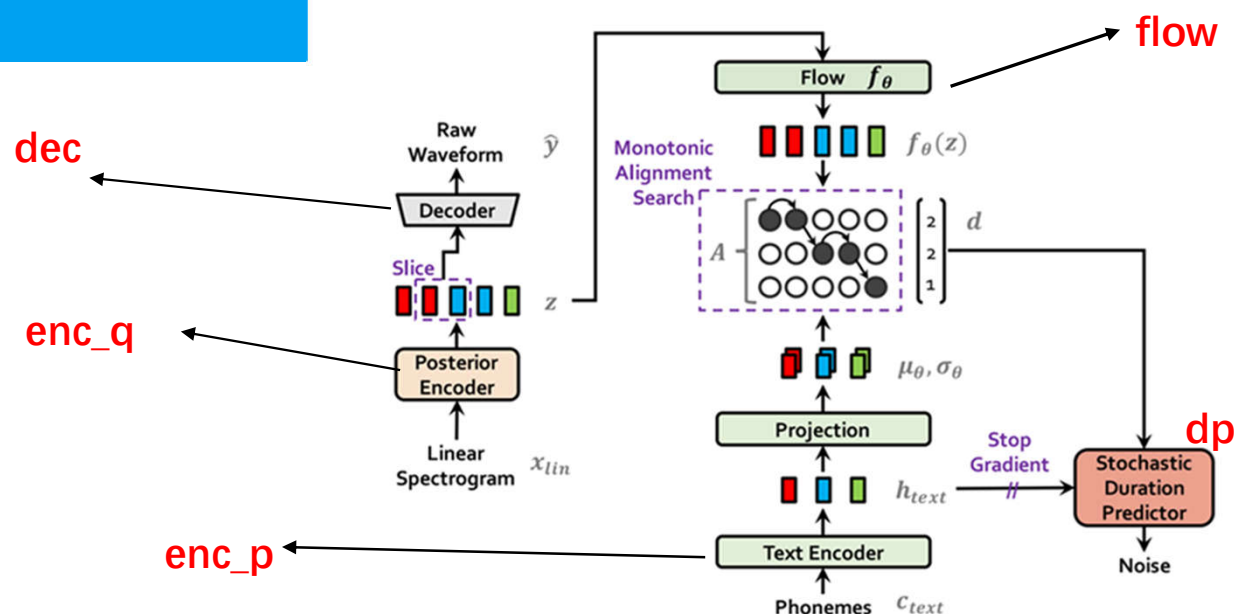
```

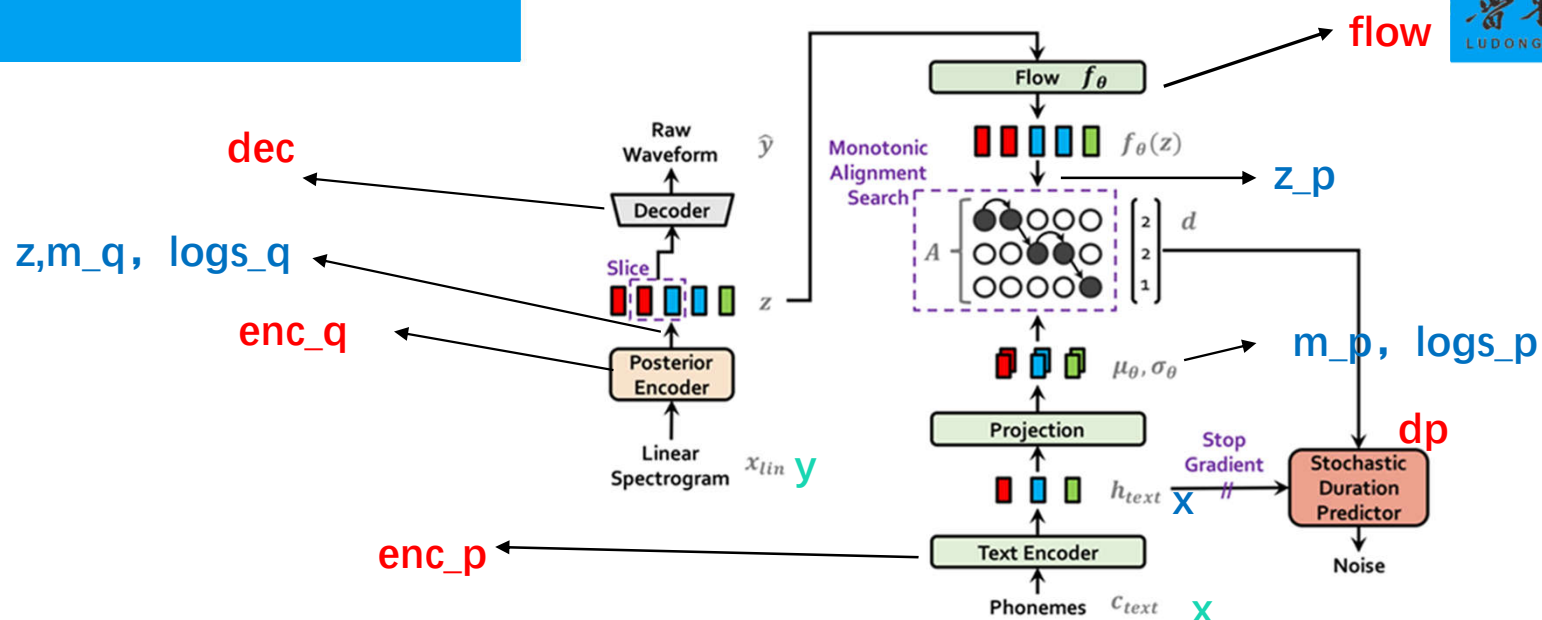
self.enc_p = TextEncoder(n_vocab,
    inter_channels,
    hidden_channels,
    filter_channels,
    n_heads,
    n_layers,
    kernel_size,
    p_dropout)

self.dec = Generator(inter_channels, resblock, resblock_kernel_sizes, resblock_dilation_sizes, upsample_rates, u
self.enc_q = PosteriorEncoder(spec_channels, inter_channels, hidden_channels, 5, 1, 16, gin_channels=gin_channel
self.flow = ResidualCouplingBlock(inter_channels, hidden_channels, 5, 1, 4, gin_channels=gin_channels)

if use_sdp:
    self.dp = StochasticDurationPredictor(hidden_channels, 192, 3, 0.5, 4, gin_channels=gin_channels)
else:
    self.dp = DurationPredictor(hidden_channels, 256, 3, 0.5, gin_channels=gin_channels)

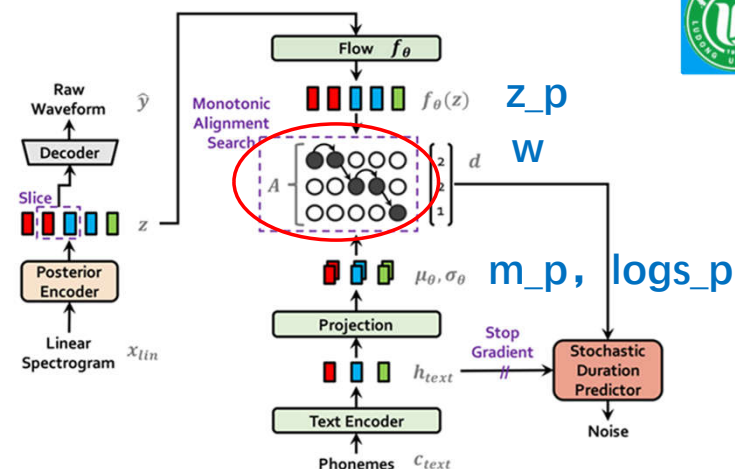
if n_speakers > 1:
    self.emb_g = nn.Embedding(n_speakers, gin_channels)
    
```





```
def forward(self, x, x_lengths, y, y_lengths, sid=None):
    # x, x_lengths 文本 y, y_lengths spec特征
    x, m_p, logs_p, x_mask = self.enc_p(x, x_lengths)
    if self.n_speakers > 0:
        g = self.emb_g(sid).unsqueeze(-1) # [b, h, 1]
    else:
        g = None

    z, m_q, logs_q, y_mask = self.enc_q(y, y_lengths, g=g)
    z_p = self.flow(z, y_mask, g=g)
```



```
attn_mask = torch.unsqueeze(x_mask, 2) * torch.unsqueeze(y_mask, -1)
attn = monotonic_align.maximum_path(neg_cent, attn_mask.squeeze(1)).unsqueeze(1).detach()
```

```
tensor([[[[1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],  
          [0., 1., 0., 0., 0., 0., 0., 0., 0., 0.],  
          [0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],  
          [0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],  
          [0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],  
          [0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],  
          [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],  
          [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],  
          [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],  
          [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],  
          [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],  
          [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.]
```

$$f(x) = \frac{1}{\sqrt{2\pi}\delta} e^{-\frac{(x-m)^2}{2\delta^2}}$$
$$\log(f(x)) = (-0.5\log 2\pi - \log \delta) + \frac{1}{\delta^2}(-0.5x^2 + xm - 0.5m^2)$$

```

if self.use_sdp:
    l_length = self.dp(x, x_mask, w, g=g)
    l_length = l_length / torch.sum(x_mask)
else:
    logw_ = torch.log(w + 1e-6) * x_mask
    logw = self.dp(x, x_mask, g=g)
    l_length = torch.sum((logw - logw_)**2, [1,2]) / torch.sum(x_mask) # for averaging

```

```
# expand prior
```

```

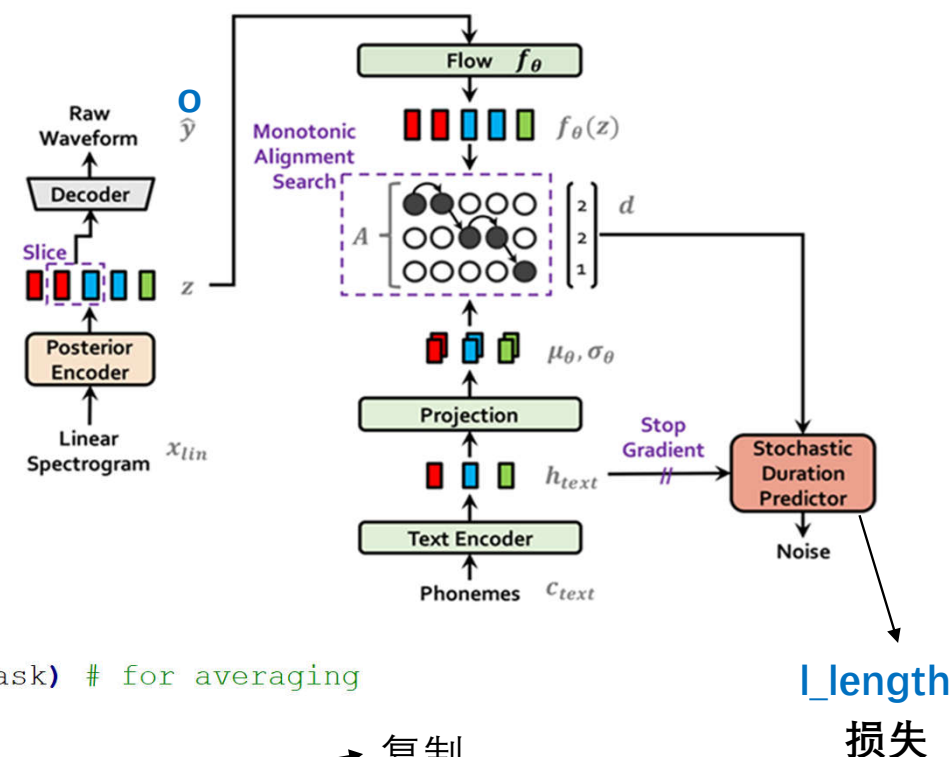
m_p = torch.matmul(attn.squeeze(1), m_p.transpose(1, 2)).transpose(1, 2)
logs_p = torch.matmul(attn.squeeze(1), logs_p.transpose(1, 2)).transpose(1, 2)

```

```

z_slice, ids_slice = commons.rand_slice_segments(z, y_lengths, self.segment_size)
o = self.dec(z_slice, g=g)
return o, l_length, attn, ids_slice, x_mask, y_mask, (z, z_p, m_p, logs_p, m_q, logs_q)

```



复制

随机切片

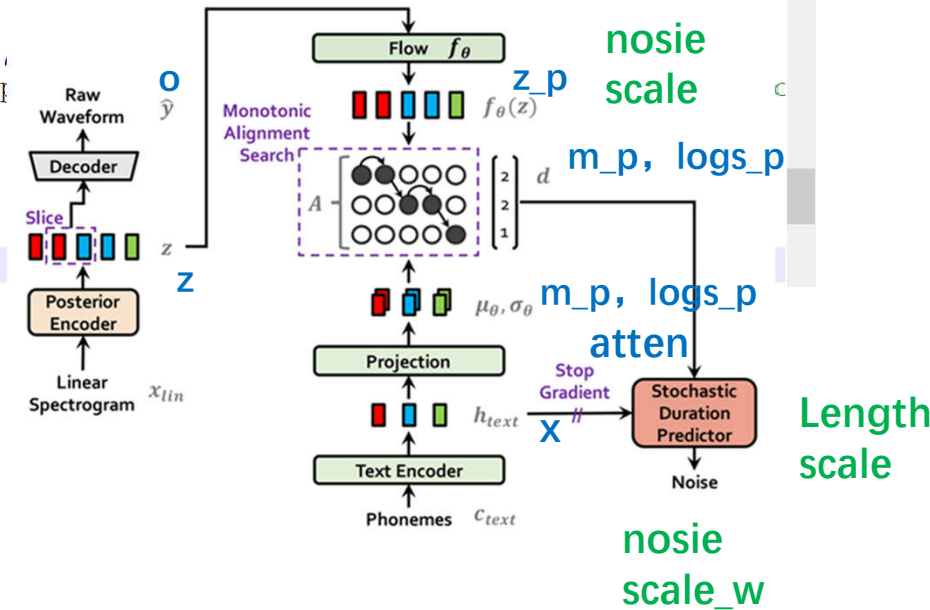
 $l_{length}$   
损失

```
def infer(self, x, x_lengths, sid=None, noise_scale=1, length_scale=1, noise_scale_w=1., max_len=None):
    x, m_p, logs_p, x_mask = self.enc_p(x, x_lengths)
    if self.n_speakers > 0:
        g = self.emb_g(sid).unsqueeze(-1) # [b, h, 1]
    else:
        g = None

    if self.use_sdp:
        logw = self.dp(x, x_mask, g=g, reverse=True, noise_scale=noise_scale_w)
    else:
        logw = self.dp(x, x_mask, g=g)
    w = torch.exp(logw) * x_mask * length_scale
    w_ceil = torch.ceil(w)
    y_lengths = torch.clamp_min(torch.sum(w_ceil, [1, 2]), 1).long()
    y_mask = torch.unsqueeze(common.sequence_mask(y_lengths, None), 1).to(x_mask.dtype)
    attn_mask = torch.unsqueeze(x_mask, 2) * torch.unsqueeze(y_mask, -1)
    attn = commons.generate_path(w_ceil, attn_mask)

    m_p = torch.matmul(attn.squeeze(1), m_p.transpose(1, 2)).transpose(1, 2)
    logs_p = torch.matmul(attn.squeeze(1), logs_p.transpose(1, 2)).transpose(1, 2)

    z_p = m_p + torch.randn_like(m_p) * torch.exp(logs_p) * noise_scale
    z = self.flow(z_p, y_mask, g=g, reverse=True)
    o = self.dec((z * y_mask)[:,:,:max_len], g=g)
    return o, attn, y_mask, (z, z_p, m_p, logs_p)
```



## 训练过程及损失函数

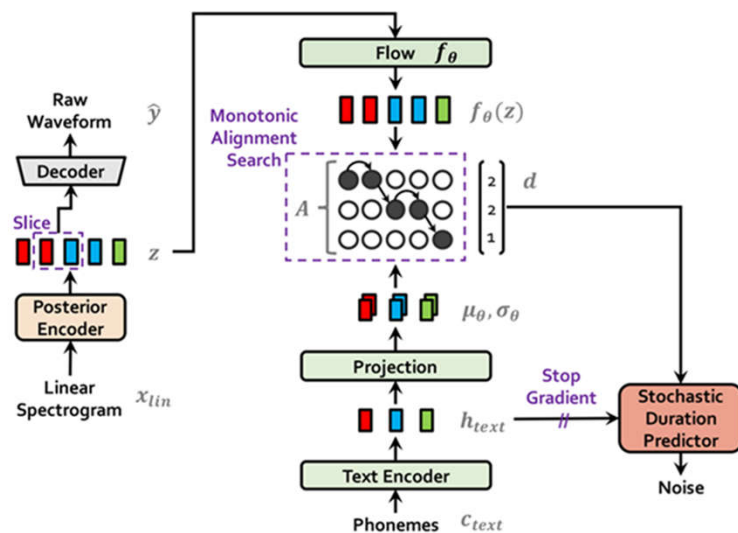
$$\log p_{\theta}(x|c) \geq \mathbb{E}_{q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) - \log \frac{q_{\phi}(z|x)}{p_{\theta}(z|c)} \right]$$

CVAE变分下界，越大越好

重构损失  $L_{recon} = \|x_{mel} - \hat{x}_{mel}\|_1$  由采样值 $z$ 生成的 $x$ ，应当拟合真实数据 $x_{data}$ 的分布

假设拉普拉斯分布

$$f(x|\mu, b) = \frac{1}{2b} e^{-\frac{|x-\mu|}{b}}$$



$$\log p_{\theta}(x|c) \geq \mathbb{E}_{q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) - \log \frac{q_{\phi}(z|x)}{p_{\theta}(z|c)} \right] \quad \text{KL散度}$$

$$L_{kl} = \log q_{\phi}(z|x_{lin}) - \log p_{\theta}(z|c_{text}, A), \quad (3)$$

$$z \sim q_{\phi}(z|x_{lin}) = N(z; \mu_{\phi}(x_{lin}), \sigma_{\phi}(x_{lin}))$$

$$p_{\theta}(z|c) = N(f_{\theta}(z); \mu_{\theta}(c), \sigma_{\theta}(c)) \left| \det \frac{\partial f_{\theta}(z)}{\partial z} \right|,$$

$$c = [c_{text}, A]$$

$$\log p_{\theta}(d|c_{text}) \geq \mathbb{E}_{q_{\phi}(u, \nu|d, c_{text})} \left[ \log \frac{p_{\theta}(d-u, \nu|c_{text})}{q_{\phi}(u, \nu|d, c_{text})} \right] \quad (7) \quad \longrightarrow \quad L_{dur} \text{ 时长预测loss}$$

$$L_{adv}(D) = \mathbb{E}_{(y,z)} \left[ (D(y) - 1)^2 + (D(G(z)))^2 \right], \quad (8)$$

$$L_{adv}(G) = \mathbb{E}_z \left[ (D(G(z)) - 1)^2 \right], \quad (9) \quad \longrightarrow \quad \text{竞争训练loss}$$

$$L_{fm}(G) = \mathbb{E}_{(y,z)} \left[ \sum_{l=1}^T \frac{1}{N_l} \|D^l(y) - D^l(G(z))\|_1 \right] \quad (10)$$

-----

更新D和G的时候选择不通过的loss

**最终：**  $L_{vae} = L_{recon} + L_{kl} + L_{dur} + L_{adv}(G) + L_{fm}(G) \quad (11)$

## 训练过程及损失函数

### train\_ms.py 88行

```
net_g = SynthesizerTrn(  
    len(symbols),  
    hps.data.filter_length // 2 + 1,  
    hps.train.segment_size // hps.data.hop_length,  
    **hps.model).cuda(rank)  
net_d = MultiPeriodDiscriminator(hps.model.use_spectral_norm).cuda(rank)
```

### train\_ms.py 143行 (train\_and\_evaluate 函数内部)

```
for batch_idx, (x, x_lengths, spec, spec_lengths, y, y_lengths, speakers) in enumerate(train_loader):  
    x, x_lengths = x.cuda(rank, non_blocking=True), x_lengths.cuda(rank, non_blocking=True)  
    spec, spec_lengths = spec.cuda(rank, non_blocking=True), spec_lengths.cuda(rank, non_blocking=True)  
    y, y_lengths = y.cuda(rank, non_blocking=True), y_lengths.cuda(rank, non_blocking=True)  
    speakers = speakers.cuda(rank, non_blocking=True)  
  
    with autocast(enabled=hps.train.fp16_run):  
        y_hat, l_length, attn, ids_slice, x_mask, z_mask,\  
        (z, z_p, m_p, logs_p, m_q, logs_q) = net_g(x, x_lengths, spec, spec_lengths, speakers)
```

```
mel = spec_to_mel_torch(  
    spec,  
    hps.data.filter_length,  
    hps.data.n_mel_channels,  
    hps.data.sampling_rate,  
    hps.data.mel_fmin,  
    hps.data.mel_fmax)  
y_mel = commons.slice_segments(mel, ids_slice, hps.train.segment_size // hps.data.hop_length)  
y_hat_mel = mel_spectrogram_torch(  
    y_hat.squeeze(1),  
    hps.data.filter_length,  
    hps.data.n_mel_channels,  
    hps.data.sampling_rate,  
    hps.data.hop_length,  
    hps.data.win_length,  
    hps.data.mel_fmin,  
    hps.data.mel_fmax  
)  
  
y = commons.slice_segments(y, ids_slice * hps.data.hop_length, hps.train.segment_size) # slice
```

stft->mel

Wav->mel

```
# Discriminator
y_d_hat_r, y_d_hat_g, _, _ = net_d(y, y_hat.detach())
with autocast(enabled=False):
    loss_disc, losses_disc_r, losses_disc_g = discriminator_loss(y_d_hat_r, y_d_hat_g)
    loss_disc_all = loss_disc
optim_d.zero_grad()
scaler.scale(loss_disc_all).backward()
scaler.unscale_(optim_d)
grad_norm_d = commons.clip_grad_value_(net_d.parameters(), None)
scaler.step(optim_d)
```

$$L_{adv}(D) = \mathbb{E}_{(y,z)} \left[ (D(y) - 1)^2 + (D(G(z)))^2 \right], \quad (8)$$

```
def discriminator_loss(disc_real_outputs, disc_generated_outputs):
    loss = 0
    r_losses = []
    g_losses = []
    for dr, dg in zip(disc_real_outputs, disc_generated_outputs):
        dr = dr.float()
        dg = dg.float()
        r_loss = torch.mean((1-dr)**2)
        g_loss = torch.mean(dg**2)
        loss += (r_loss + g_loss)
        r_losses.append(r_loss.item())
        g_losses.append(g_loss.item())

    return loss, r_losses, g_losses
```

loss.py 18 行

```
with autocast(enabled=hps.train.fp16_run):
```

```
    # Generator
```

```
    y_d_hat_r, y_d_hat_g, fmap_r, fmap_g = net_d(y, y_hat)
```

```
    with autocast(enabled=False):
```

```
        loss_dur = torch.sum(l_length.float())
```

```
        loss_mel = F.l1_loss(y_mel, y_hat_mel) * hps.train.c_mel
```

```
        loss_kl = kl_loss(z_p, logs_q, m_p, logs_p, z_mask) * hps.train.c_kl
```

```
        loss_fm = feature_loss(fmap_r, fmap_g)
```

```
        loss_gen, losses_gen = generator_loss(y_d_hat_g)
```

```
        loss_gen_all = loss_gen + loss_fm + loss_mel + loss_dur + loss_kl
```

```
    optim_g.zero_grad()
```

```
    scaler.scale(loss_gen_all).backward()
```

```
    scaler.unscale_(optim_g)
```

```
    grad_norm_g = commons.clip_grad_value_(net_g.parameters(), None)
```

```
    scaler.step(optim_g)
```

```
    scaler.update()
```

 $L_{dur}$ 

$$L_{recon} = \|x_{mel} - \hat{x}_{mel}\|_1$$

$$L_{fm}(G) = \mathbb{E}_{(y,z)} \left[ \sum_{l=1}^T \frac{1}{N_l} \|D^l(y) - D^l(G(z))\|_1 \right] \quad (10)$$

$$L_{adv}(G) = \mathbb{E}_z \left[ (D(G(z)) - 1)^2 \right], \quad (9)$$

```
def feature_loss(fmap_r, fmap_g):
    loss = 0
    for dr, dg in zip(fmap_r, fmap_g):
        for rl, gl in zip(dr, dg):
            rl = rl.float().detach()
            gl = gl.float()
            loss += torch.mean(torch.abs(rl - gl))

    return loss * 2
```

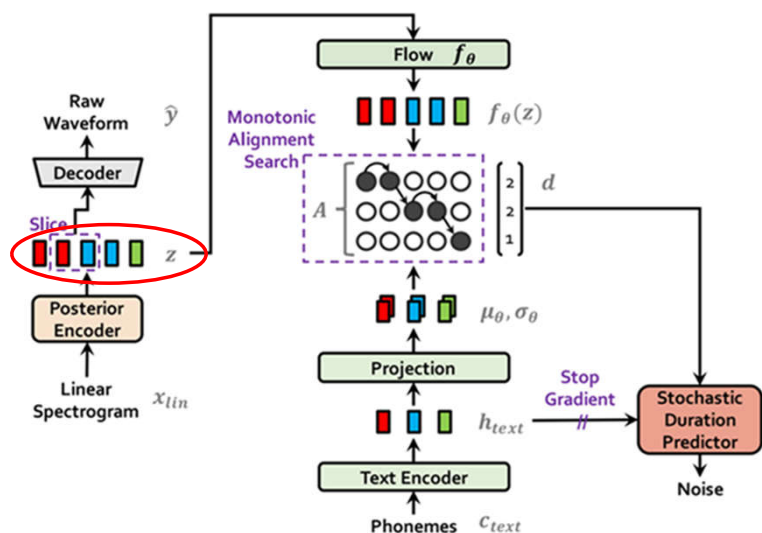
```
def generator_loss(disc_outputs):
    loss = 0
    gen_losses = []
    for dg in disc_outputs:
        dg = dg.float()
        l = torch.mean((1-dg)**2)
        gen_losses.append(l)
        loss += l

    return loss, gen_losses
```

$$\log p_{\theta}(x|c) \geq \mathbb{E}_{q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) - \log \frac{q_{\phi}(z|x)}{p_{\theta}(z|c)} \right]$$

$$L_{kl} = \log q_{\phi}(z|x_{lin}) - \log p_{\theta}(z|c_{text}, A), \quad (3)$$

$$z \sim q_{\phi}(z|x_{lin}) = N(z; \mu_{\phi}(x_{lin}), \sigma_{\phi}(x_{lin}))$$



$$N(z; m_q, \delta_q) = \frac{1}{\sqrt{2\pi}\delta_q} e^{-\frac{(z-m_q)^2}{2\delta_q^2}}$$

$$\mathbb{E}_{q_{\phi}(z|x)} [\log q_{\phi}(z|x)] = \int_{-\infty}^{+\infty} N(z; m_q, \delta_q) \log N(z; m_q, \delta_q) dz$$

$$= -\frac{1}{2} - \frac{1}{2} \log(2\pi) - \log \delta_q$$

证明网址

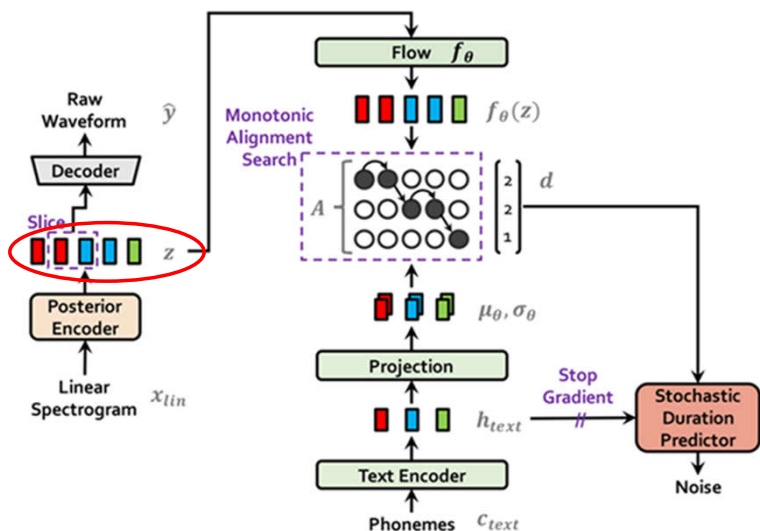
[多元高斯分布以及高斯分布微分熵 - 知乎 \(zhihu.com\)](https://zhuanlan.zhihu.com/p/150114190)

<https://zhuanlan.zhihu.com/p/150114190>

$$\log p_{\theta}(x|c) \geq \mathbb{E}_{q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) - \log \frac{q_{\phi}(z|x)}{p_{\theta}(z|c)} \right]$$

$$L_{kl} = \log q_{\phi}(z|x_{lin}) - \log p_{\theta}(z|c_{text}, A), \quad (3)$$

$$z \sim q_{\phi}(z|x_{lin}) = N(z; \mu_{\phi}(x_{lin}), \sigma_{\phi}(x_{lin}))$$



$$p_{\theta}(z|c) = N(f_{\theta}(z); \mu_{\theta}(c), \sigma_{\theta}(c)) \left| \det \frac{\partial f_{\theta}(z)}{\partial z} \right|, \quad c = [c_{text}, A]$$

语音域的采样值  $z$  在本文域的概率密度函数  $p_{\theta}(z|c)$

表示式未知

但是采样值  $z$  经过 flow 后得到的  $f_{\theta}(z)$  在本文域的概率密度函数为高斯分布

$f_{\theta}(z)$  就是  $z_p$  就是复制后的  $m_p, logs_p$

$$p_{\theta}(z|c) = N(f_{\theta}(z); \mu_{\theta}(c), \sigma_{\theta}(c)) \left| \det \frac{\partial f_{\theta}(z)}{\partial z} \right|, \quad c = [c_{text}, A]$$

$$\log p_{\theta}(z|c) = \log \frac{1}{\sqrt{2\pi}\delta_p} e^{-\frac{(z_p - m_p)^2}{2\delta_p^2}} = -\frac{1}{2} \log 2\pi - \log \delta_p - \frac{(z_p - m_p)^2}{2\delta_p^2}$$

$$\log p_{\theta}(x|c) \geq \mathbb{E}_{q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) - \log \frac{q_{\phi}(z|x)}{p_{\theta}(z|c)} \right] \quad L_{kl} = \log q_{\phi}(z|x_{lin}) - \log p_{\theta}(z|c_{text}, A), \quad (3)$$

$$z \sim q_{\phi}(z|x_{lin}) = N(z; \mu_{\phi}(x_{lin}), \sigma_{\phi}(x_{lin}))$$

$$\mathbb{E}_{q_{\phi}(z|x)} [\log q_{\phi}(z|x)] = \int_{-\infty}^{+\infty} N(z; m_q, \delta_q) \log N(z; m_q, \delta_q) dz = -\frac{1}{2} - \frac{1}{2} \log(2\pi) - \log \delta_q$$

$$\log p_{\theta}(z|c) = \log \frac{1}{\sqrt{2\pi}\delta_p} e^{-\frac{(z_p - m_p)^2}{2\delta_p^2}} = -\frac{1}{2} \log 2\pi - \log \delta_p - \frac{(z_p - m_p)^2}{2\delta_p^2}$$

$$\mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(z|x)] = \text{mean} \left( -\frac{1}{2} \log 2\pi - \log \delta_p - \frac{(z_p - m_p)^2}{2\delta_p^2} \right)$$

```
def kl_loss(z_p, logs_q, m_p, logs_p, z_mask):
    """
    z_p, logs_q: [b, h, t_t]
    m_p, logs_p: [b, h, t_t]
    """
    z_p = z_p.float()
    logs_q = logs_q.float()
    m_p = m_p.float()
    logs_p = logs_p.float()
    z_mask = z_mask.float()

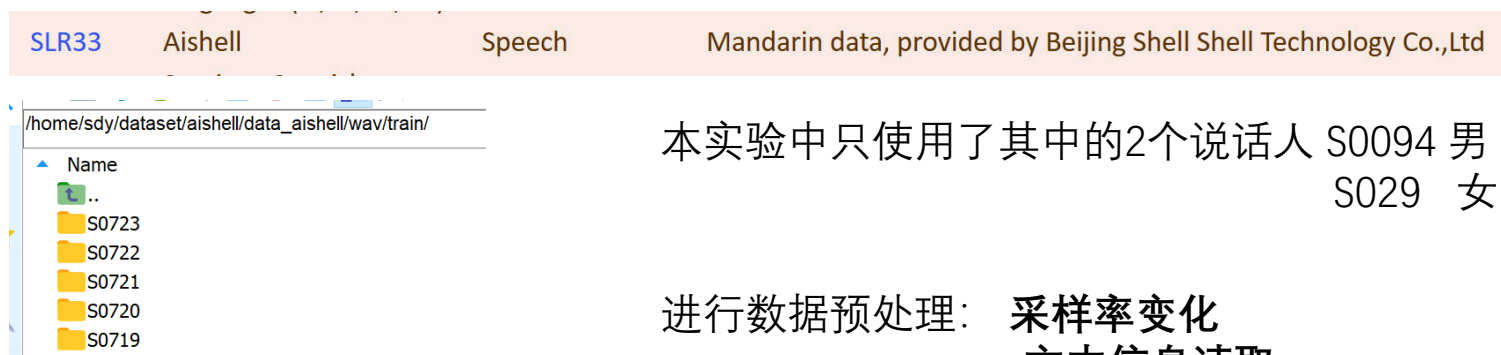
    kl = logs_p - logs_q - 0.5
    kl += 0.5 * ((z_p - m_p)**2) * torch.exp(-2. * logs_p)
    kl = torch.sum(kl * z_mask)
    l = kl / torch.sum(z_mask)
    return l
```

$$\mathbb{E}_{q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) - \log \frac{q_{\phi}(z|x)}{p_{\theta}(z|c)} \right] = \text{mean} \left( -\frac{1}{2} - \log \delta_q + \log \delta_p + \frac{(z_p - m_p)^2}{2\delta_p^2} \right)$$

## 数据集的准备:

### (1) 准备待训练的语音以及其相应的文本

在本实验中使用的是aishell 数据集 <http://openslr.org/resources.php>



本实验中只使用了其中的2个说话人 S0094 男  
S029 女

进行数据预处理：  
 采样率变化  
 文本信息读取  
 文本编码 从 txt->symbol

这里编码使用的台湾省的  
拼音

格式 wav文件路径|sid|symbols      sid 用 0,1,2,3,... 进行编号

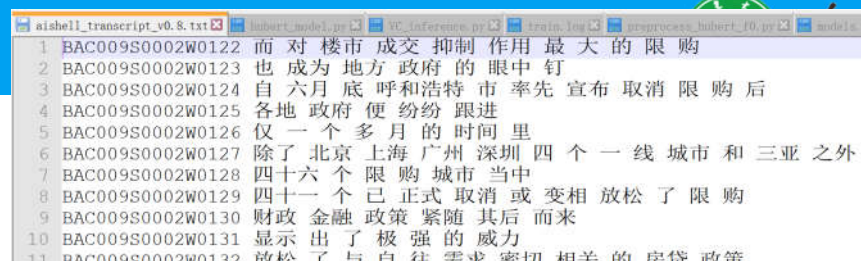
|                             |   |                |
|-----------------------------|---|----------------|
| data/0/BAC009S0094W0164.wav | 0 | 本轮楼市回暖多体现在一线城市 |
| data/0/BAC009S0094W0345.wav | 0 | 选手可以自行免费获取照片   |
| data/0/BAC009S0094W0130.wav | 0 | 领涨全国二手住宅市场     |

data/0/BAC009S0094W0164.wav|0|ㄅㄨˋㄌㄧˊㄉㄜˊ ㄆㄛˊㄇㄣˊㄍㄜˊㄎㄜˊㄏㄜˊㄗㄩˊㄇㄞˊ ㄑㄩˊㄒㄩˊㄇㄞˊ ㄔㄩˊ  
data/0/BAC009S0094W0345.wav|0|ㄊㄩˊㄌㄩˊㄇㄞˊㄆㄛˊㄇㄣˊㄍㄜˊㄐㄜˊㄍㄜˊㄕㄩˊ ㄘㄩˊㄙㄩˊㄇㄞˊ。  
data/0/BAC009S0094W0130.wav|0|ㄌㄠˊㄌㄩˊㄘㄩˊㄍㄩˊㄍㄜˊㄕㄩˊㄆㄛˊㄘㄩˊㄘㄩˊㄆㄞˊㄆㄞˊㄈㄜˊ。  
data/0/BAC009S0094W0174.wav|0|ㄆㄛˊㄆㄞˊㄌㄩˊㄆㄞˊㄑㄩˊㄕㄩˊㄆㄞˊㄆㄞˊㄆㄞˊㄆㄞˊㄆㄞˊㄆㄞˊ。

```
import librosa
import os
import soundfile as sf
import random
```

```
def remove_file_extension(filename):
    parts = filename.split('.')
    new_filename = '.'.join(parts[:-1])
    return new_filename
```

```
dic_tex = {}
file_text = "/home/sdy/dataset/aishell/data_aishell/transcript/aishell_transcript_v0.8.txt"
with open(file_text) as f:
    lines = f.readlines()
    for line in lines:
        pos = line.find(" ")
        id = line[:pos]
        txt = line[pos+1:]
        dic_tex[id]=txt.lstrip()
```



```
1 BAC009S0002W0122 而对楼市成交抑制作用最大的限购
2 BAC009S0002W0123 也成为地方政府的 眼中钉
3 BAC009S0002W0124 自六月底呼和浩特市率先宣布取消限购后
4 BAC009S0002W0125 各地政府便纷纷跟进
5 BAC009S0002W0126 仅一个多月的时间里
6 BAC009S0002W0127 除了北京上海广州深圳四个一线城市和三亚之外
7 BAC009S0002W0128 四十六个限购城市当中
8 BAC009S0002W0129 四十一个已正式取消或变相放松了限购
9 BAC009S0002W0130 财政金融政策紧随其后而来
10 BAC009S0002W0131 显示出了极强的威力
11 BAC009S0002W0132 放松了与以往需求密切相关的房贷政策
```

去掉文件后缀

aishell提供的翻译文件

方便查询把信息存成字典

```
list_ids = ['S0094', 'S0129']
wav_path = '/home/sdy/dataset/aishell/data_aishell/wav/train/'
save_path = "data/"
with open("transcript.txt", 'wt') as f:
    for i, ids in enumerate(list_ids):
        path_wavs = os.path.join(wav_path, ids)
        save_wavs = os.path.join(save_path, str(i))
        os.makedirs(save_wavs, exist_ok=True)
        for roots, dirs, files in os.walk(path_wavs):
            for file in files:
                if file.endswith("wav"):
```

# 读取原始音频

```
file_wav = os.path.join(roots, file)
print("SRC_WAV", file_wav)
```

# 降采样到22k

```
y, fs = librosa.load(file_wav)
```

# 重新保存

```
file_22k = os.path.join(save_wavs, file)
print("22k WAV", file_22k)
sf.write(file_22k, y, fs)
```

# 读取文本信息

```
try:
```

```
    id = remove_file_extension(file)
    txt = dic_tex[id]
    if len(txt) < 5:
        print(id + "too short")
```

```
        continue
```

```
    str_info = file_22k + "|" + str(i) + "|" + txt.replace(" ", "")
    f.write(str_info)
```

```
    # print(str_info)
```

```
except:
```

```
    print("Notfind", id)
```

librosa 默认  
22k采样率

去掉过短的文本

去掉找不到的翻译

## 最终生成

```
data/0/BAC009S0094W0164.wav|0|本轮楼市回暖多体现在一线城市
data/0/BAC009S0094W0345.wav|0|选手可以自行免费获取照片
data/0/BAC009S0094W0130.wav|0|领涨全国二手住宅市场
```

进行翻译/转码

```
import text
from utils import load_filepaths_and_text

filelists = ['transcript.txt']
for filelist in filelists:
    print("START:", filelist)
    filepaths_and_text = load_filepaths_and_text(filelist)
    for i in range(len(filepaths_and_text)):
        original_text = filepaths_and_text[i][2]
        cleaned_text = text.clean_text(original_text, ["chinese_cleaners"])
        filepaths_and_text[i][2] = cleaned_text

new_filelist = filelist + "." + "clean"
with open(new_filelist, "w", encoding="utf-8") as f:
    f.writelines(["|".join(x) + "\n" for x in filepaths_and_text])

f_train = open("train_txt.clean", 'wt')
f_val = open("val_txt.clean", 'wt')

with open("transcript.txt.clean", 'r') as f:
    lines = f.readlines()
    for line in lines:
        a = random.random()
        if a > 0.05:
            f_train.write(line)
        else:
            f_val.write(line)
f_train.close()
f_val.close()
```

第三列式文本

转码

```
# chinese_cleaners
_pad          = '_'
_punctuation  = ', . ! ? ~...'
_letters      = 'ㄅ ㄆ ㄇ ㄏ ㄓ ㄔ ㄕ ㄖ ㄗ ㄘ ㄙ ㄨ ㄩ ㄣ ㄤ ㄛ ㄜ ㄝ ㄞ ㄟ ㄠ ㄡ ㄢ ㄣ ㄤ ㄥ ㄦ ㄷ ㄸ ㄹ ㄺ ㄻ ㄼ ㄽ ㄾ ㄿ ㅁ ㅂ ㅃ ㅅ ㅆ ㅈ ㅊ ㅋ ㆁ ㆂ ㆃ ㆄ ㆅ ㆆ ㆇ ㆈ ㆉ ㆊ ㆋ ㆌ ㆍ ㆎ ㆏ ㆐ ㆑ ㆒ ㆓ ㆔ ㆕ ㆖ ㆗ ㆘ ㆙ ㆚ ㆛ ㆜ ㆝ ㆞ ㆟ ㆠ ㆡ ㆢ ㆣ ㆤ ㆥ ㆦ ㆧ ㆨ ㆩ ㆪ ㆫ ㆬ ㆭ ㆮ ㆯ ㆰ ㆱ ㆲ ㆳ ㆴ ㆵ ㆶ ㆷ ㆸ ㆹ ㆺ ㆻ ㆼ ㆽ ㆾ ㆿ ㆿ'

'''# zh_ja_mixture_cleaner
_pad          = '_'
_punctuation  = ', . ! ? ~...'
```

**注意：**

text/symbols.py 注释掉不需要的symbol  
text/cleaners.py 注释掉不需要的引用

```
def chinese_cleaners(text):
    '''Pipeline for Chinese text'''
    from text.mandarin import number_to_chinese, chinese_to_bopomofo, latin_to_bopomofo
    text = number_to_chinese(text)
    text = chinese_to_bopomofo(text)
    text = latin_to_bopomofo(text)
    text = re.sub(r'([`~!@#$%^&*()-+=\|;:,./? ]+$)', r'\1. ', text)
    return text


# def zh_ja_mixture_cleaners(text):
#     # from text.mandarin import chinese_to_romaji
#     # from text.japanese import japanese_to_romaji_with_accent
#     # text = re.sub(r'[ZH\](.*?)\[ZH\]',
#     #               lambda x: chinese_to_romaji(x.group(1))+ ' ', text)
```

[illegible]



### (3) 训练之前的一些小的tip

#### 1 librosa 的安装

##### requirement.txt

```
Cython==0.29.21
librosa==0.8.0
matplotlib==3.3.1
numpy==1.21.6
scipy==1.5.2
tensorboard==2.3.0
torch==1.6.0
torchvision==0.7.0
unidecode==1.3.4
pyopenjtalk==0.2.0
jamo==0.4.1
pypinyin==0.44.0
jieba==0.42.1
protobuf==3.19.0
cn2an==0.5.17
inflect==6.0.0
eng_to_ipa==0.0.2
ko_pron==1.3
indic_transliteration==2.3.37
num_thai==0.0.5
opencc==1.1.1
```

目前的新版本是 0.10.0 安装过程中可能涉及到 numba 版本与其他安装包的冲突，以及 llvmlite 安装问题（需要管理员权限）

`pip install --ignore-installed llvmlite`

然后再安装最新版的 librosa 就可以

**mel\_processing.py** 73-83行左右 **def spec\_to\_mel\_torch**

`mel=librosa.filters.mel(sr=sampling_rate,n_fft=n_fft,n_mels=num_mels,fmin=fmin,fmax=fmax)`

`# librosa_mel_fn(sampling_rate, n_fft, num_mels, fmin, fmax)` → 旧版函数  
换新版函数

**97行左右 函数 mel\_spectrogram\_torch** 也做同样的替换

原版的 train\_ms.py 采用的是 分布式训练，默认占用所有的GPU

在实际应用中可能没有那么多GPU 并且 0编号的GPU被可能给别人占用，因此 我做了一些修改  
把多GPU分布训练改成了单GPU的训练。

主要修改 1 sampler  
2 evaluate 部分 device 的配置

utils.py 144 行

```
def get_hparams(init=True):  
    parser = argparse.ArgumentParser()  
    parser.add_argument('-c', '--config', type=str, default="./configs/base.json",  
                        help='JSON file for configuration')  
    parser.add_argument('-m', '--model', type=str, required=True,  
                        help='Model name')  
  
    args = parser.parse_args()
```

config 文件的路径

存放模型的路径

可以通过命令行的方法  
来执行，也可以通过  
修改文件来执行

```
model_dir = os.path.join("../drive/MyDrive", args.model)
```

## 训练后的模型存放

/home/sdo/vits-main\_all/drive/MyDrive/yuhong/ 

| Name                                           | Size (KB) |
|------------------------------------------------|-----------|
| ..                                             |           |
| eval                                           |           |
| train.log                                      | 282       |
| G_2000.pth                                     | 465 423   |
| G_1000.pth                                     | 465 423   |
| events.out.tfevents.1694498065.admin2.35757.0  | 47 620    |
| events.out.tfevents.1694495970.admin2.60394.0  | 3 150     |
| events.out.tfevents.1694495815.admin2.2195.0   | 116       |
| events.out.tfevents.1694495616.admin2.58067.0  | 116       |
| events.out.tfevents.1694495502.admin2.16870.0  | 118       |
| events.out.tfevents.1694495397.admin2.107038.0 | 118       |
| D_2000.pth                                     | 547 920   |
| D_1000.pth                                     | 547 920   |
| config.json                                    | 1         |

## 测试 m\_test.py

```
import torch
import utils
from models import SynthesizerTrn
from text import text_to_sequence
import commons
from torch import no_grad, LongTensor
import os
import soundfile as sf
limitation = os.getenv("SYSTEM") == "spaces"

file_model = '/home/sdo/vits-main_all/drive/MyDrive/yuhong/G_2000.pth'
hps_ms = utils.get_hparams_from_file('./configs/my_chinese_base.json')

device = torch.device("cpu")

net_g_ms = SynthesizerTrn(
    len(hps_ms.symbols),
    hps_ms.data.filter_length // 2 + 1,
    hps_ms.train.segment_size // hps_ms.data.hop_length,
    n_speakers=hps_ms.data.n_speakers,
    **hps_ms.model)
_ = net_g_ms.eval().to(device)

speakers = hps_ms.speakers
model, optimizer, learning_rate, epochs = utils.load_checkpoint(file_model, net_g_ms, None)
```

```
def get_text(text, hps):  
    text_norm = text_to_sequence(text, hps.data.text_cleaners)  
    if hps.data.add_blank:  
        text_norm = commons.intersperse(text_norm, 0)  
    text_norm = LongTensor(text_norm)  
    return text_norm  
  
def vits(text, speaker_id, noise_scale, noise_scale_w, length_scale):  
    if not len(text):  
        return "输入文本不能为空!", None, None  
    text = text.replace('\n', ' ').replace('\r', '').replace(" ", "")  
    if len(text) > 100 and limitation:  
        return f"输入文字过长! {len(text)}>100", None, None  
  
    stn_tst = get_text(text, hps_ms)  
    with no_grad():  
        x_tst = stn_tst.unsqueeze(0).to(device)  
        x_tst_lengths = LongTensor([stn_tst.size(0)]).to(device)  
        speaker_id = LongTensor([speaker_id]).to(device)  
        # tensor([132]) 0.1 0.668 1.1  
        audio = net_g_ms.infer(x_tst, x_tst_lengths, sid=speaker_id, noise_scale=noise_scale,  
                               noise_scale_w=noise_scale_w,  
                               length_scale=length_scale)[0][0, 0].data.cpu().float().numpy()  
  
    return 22050, audio
```

```
speed = 1  
fs, audio = vits("开发者的销售压力较大", torch.tensor([0], 0.1, 0.668, 1.0/speed)  
sf.write('male-zzz.wav', audio, samplerate=fs)
```

→ id 值

# VITS-fast-fine-tuning VITS的快速微调:

项目地址: <https://github.com/Plachtaa/VITS-fast-fine-tuning>

步骤: (1) 下载预训练模型

CJE  
中日英  
三语模型

[https://huggingface.co/spaces/Plachta/VITS-Umamusume-voice-synthesizer/resolve/main/pretrained\\_models/D\\_trilingual.pth](https://huggingface.co/spaces/Plachta/VITS-Umamusume-voice-synthesizer/resolve/main/pretrained_models/D_trilingual.pth)

[https://huggingface.co/spaces/Plachta/VITS-Umamusume-voice-synthesizer/resolve/main/pretrained\\_models/G\\_trilingual.pth](https://huggingface.co/spaces/Plachta/VITS-Umamusume-voice-synthesizer/resolve/main/pretrained_models/G_trilingual.pth)

Config的  
Json文件

[https://huggingface.co/spaces/Plachta/VITS-Umamusume-voice-synthesizer/resolve/main/configs/uma\\_trilingual.json](https://huggingface.co/spaces/Plachta/VITS-Umamusume-voice-synthesizer/resolve/main/configs/uma_trilingual.json)

一些预训练的数据 (当用来微调的数据过少时可以使用)

[https://huggingface.co/datasets/Plachta/sampled\\_audio4ft/resolve/main/sampled\\_audio4ft.zip](https://huggingface.co/datasets/Plachta/sampled_audio4ft/resolve/main/sampled_audio4ft.zip)

## (2) 下载源代码

创建文件夹 **pretrained\_models**

将步骤 (1) 中下载模型文件保存并改名

| /home/sdo/VITS-finetune/VITS-fast-fine-tuning-main/ |           |
|-----------------------------------------------------|-----------|
| Name                                                | Size (KB) |
| ..                                                  |           |
| .idea                                               |           |
| __pycache__                                         |           |
| configs                                             |           |
| monotonic_align                                     |           |
| pretrained_models                                   |           |
| utils                                               |           |

| /home/sdo/VITS-finetune/VITS-fast-fine-tuning-main/pretrained_models/ |           |                 |
|-----------------------------------------------------------------------|-----------|-----------------|
| Name                                                                  | Size (KB) | Last modified   |
| ..                                                                    |           |                 |
| D_0.pth                                                               | 182 641   | 2023-09-22 1... |
| G_0.pth                                                               | 156 170   | 2023-09-22 1... |

将下载的config文件，保存到 configs文件夹

| /home/sdo/VITS-finetune/VITS-fast-fine-tuning-main/configs/ |           |        |
|-------------------------------------------------------------|-----------|--------|
| Name                                                        | Size (KB) | Last m |
| ..                                                          |           |        |
| m_speaker_finetune.json                                     | 9         | 2023-0 |
| modified_finetune_speaker.json                              | 2         | 2023-0 |
| modified_m_speaker_finetune....                             | 2         | 2023-0 |
| uma_trilingual.json                                         | 1         | 2023-0 |

### (3) 训练数据准备, 这依旧使用aishell数据集中的 说话人 **S0094 男** **S029 女**

创建数据描述文件 (格式) : **file\_path|id|symbols**

由于这里使用的是 CJE三语模型, 因此也要切换相应的cleaner

变换前

```
import os
import text
import random
import json

if __name__ == "__main__":
    file_old = "transcript.txt"
    path_wav = "/home/sdo/vits-main_all/vits-main"
    cleaner_name = "cjke_cleaners2"
```

```
data/0/BAC009S0094W0164.wav|0|本轮楼市回暖多体现在一线城市
data/0/BAC009S0094W0345.wav|0|选手可以自行免费获取照片
data/0/BAC009S0094W0130.wav|0|领涨全国二手住宅市场
data/0/BAC009S0094W0174.wav|0|带动商品房销售额大幅回升
data/0/BAC009S0094W0162.wav|0|易居研究所研究员亢亚娟分析认为
data/0/BAC009S0094W0493.wav|0|看过射雕英雄传的人都知道这件宝贝
```

变换后

```
with open(file_old, 'rt') as f:
    lines = f.readlines()

with open("transcript.txt.clean", 'wt') as f:
    for line in lines:
        file, spk_id, txt = line.split("|")

        file_wav = os.path.join(path_wav, file)
        txt = '[ZH]' + txt[:-1] + '[ZH]'
        clean_txt = text._clean_text(txt, cleaner_names=[cleaner_name])

        if os.path.exists(file_wav):
            f.write("%s| %s| %s\n" % (file_wav, spk_id, clean_txt))
        else:
            print("file wav %s not find" % (file_wav))
```

```
/home/sdo/vits-main_all/vits-main/data/0/BAC009S0094W0164.wav|0|p'æn;lwən; lóu's'x', xwér;nwán; t'wo- t'hi;fjén; ts'ar; i-fjén; ts'həŋ's'x'.
/home/sdo/vits-main_all/vits-main/data/0/BAC009S0094W0345.wav|0|f'uan;s'ou; k'a; i; ts'xi;fjén; mjen;fer; xwo;t'hq; ts'ou;p'hjen-.
/home/sdo/vits-main_all/vits-main/data/0/BAC009S0094W0130.wav|0|lin;ts'ən; t'f'uan;k'wo; ə's'ou; ts'u'ts'ar; s'x'ts'hən;.
/home/sdo/vits-main_all/vits-main/data/0/BAC009S0094W0174.wav|0|t'ar;t'ən; s'ən-p'hin;fən; fiau-s'ou; t'a;fu; xwér;s'ən-.
/home/sdo/vits-main_all/vits-main/data/0/BAC009S0094W0162.wav|0|i;t'f' - jen;t'f'joo-s'wo; jen;t'f'joo-uan; k'ən;j'a;t'f'uan; fən-fi- x'an;weir;.
/home/sdo/vits-main_all/vits-main/data/0/BAC009S0094W0493.wav|0|k'hən;k'wo; s'a;t'iau; in-fjón;ts'hwan; t'a x'an t'oo- ts'x'-t'ow; ts'ə;t'fjén; p'
/home/sdo/vits-main_all/vits-main/data/0/BAC009S0094W0485.wav|0|s'ən;ts'x'; in;fa; uən;j'joo; t'wei; t'ən-s'x';x'an; t'f'in;fjén; x'an x'ou; sou-
```

加  
[ZH]  
标记  
中文

## 文件切分

```
f_train = open("train_txt.clean", 'wt')
f_val = open("val_txt.clean", 'wt')

with open("transcript.txt.clean", 'r') as f:
    lines = f.readlines()
for line in lines:
    a = random.random()
    if a > 0.05:
        f_train.write(line)
    else:
        f_val.write(line)
f_train.close()
f_val.close()
```

```
with open("./configs/uma_trilingual.json", 'r', encoding='utf-8') as f:
    hps = json.load(f)
```

```
hps['data']['n_speakers'] = len(speaker2id)
```

```
# overwrite speaker names
```

```
hps['speakers'] = speaker2id
```

```
hps['train']['log_interval'] = 10
```

```
hps['train']['eval_interval'] = 100
```

```
hps['train']['batch_size'] = 16
```

```
hps['data']['training_files'] = "train_txt.clean.txt"
```

```
hps['data']['validation_files'] = "val_txt.clean.txt"
```

```
# save modified config
```

```
with open("./configs/modified_m_speaker_finetune.json", 'w', encoding='utf-8') as f:
    json.dump(hps, f, indent=2)
```

## 修改config文件

#### (4) 模型微调的核心部分 在于预训练模型的加载      utils.py (178行) 函数 **load\_checkpoint**

```
def load_checkpoint(checkpoint_path, model, optimizer=None, drop_speaker_emb=False):
    assert os.path.isfile(checkpoint_path)
    checkpoint_dict = torch.load(checkpoint_path, map_location='cpu')
    iteration = checkpoint_dict['iteration']
    learning_rate = checkpoint_dict['learning_rate']
    if optimizer is not None:
        optimizer.load_state_dict(checkpoint_dict['optimizer'])
    saved_state_dict = checkpoint_dict['model']
    if hasattr(model, 'module'):
        state_dict = model.module.state_dict()
    else:
        state_dict = model.state_dict()
    new_state_dict = {}
    for k, v in state_dict.items():
        try:
            if k == 'emb_g.weight':
                if drop_speaker_emb:
                    new_state_dict[k] = v
                    continue
                v[:, saved_state_dict[k].shape[0], :] = saved_state_dict[k]
                new_state_dict[k] = v
            else:
                new_state_dict[k] = saved_state_dict[k]
        except:
            logger.info("%s is not in the checkpoint" % k)
            new_state_dict[k] = v
    if hasattr(model, 'module'):
        model.module.load_state_dict(new_state_dict)
    else:
        model.load_state_dict(new_state_dict)
    logger.info("Loaded checkpoint '{}' (iteration {})".format(
        checkpoint_path, iteration))
    return model, optimizer, learning_rate, iteration
```

预训练模型

新创建的模型

把预训练模型的  
参数，填入到  
新创建模型中

## (5) 训练部分 `m_finetune_speaker_v2.py`

```
def main():
    """Assume Single Node Multi GPUs Training Only"""
    assert torch.cuda.is_available(), "CPU training is not allowed."

    n_gpus = torch.cuda.device_count()
    os.environ['MASTER_ADDR'] = 'localhost'
    os.environ['MASTER_PORT'] = '8000'

    # 读取config信息

    config_file = os.path.join("configs", "modified_m_speaker_finetune.json")
    hps = readconfig(config_file)

    # 定义模型存储位置
    hps.model_dir = os.path.join("./yuhong2")
    if not os.path.exists(hps.model_dir):
        os.makedirs(hps.model_dir)
```

```
# 定义迭代的轮次
max_epochs = 50
hps.max_epochs = max_epochs

# 是否继续上一次停止的地方开始重新训练
hps.cont = False

# 是否去掉预训练模型中的speaker_embed
hps.drop_speaker_embed = True

# 是否使用预训练模型
hps.train_with_pretrained_model = True

# 保存模型的数目
hps.preserved = 4

# mp.spawn(run, nprocs=n_gpus, args=(n_gpus, hps,))
run(1, 1, hps)
```

/home/sdo/VITS-finetune/VITS-fast-fine-tuning-main/yuhong2/

| Name                             | Size (KB) | Last modified  |
|----------------------------------|-----------|----------------|
| eval                             |           | 2023-09-22 1.. |
| D_1800.pth                       | 182 643   | 2023-09-22 1.. |
| D_1900.pth                       | 182 643   | 2023-09-22 1.. |
| D_2000.pth                       | 182 643   | 2023-09-22 1.. |
| D_2100.pth                       | 182 643   | 2023-09-22 1.. |
| D_latest.pth                     | 182 643   | 2023-09-22 1.. |
| events.out.tfevents.169535343... | 28 693    | 2023-09-22 1.. |
| G_1800.pth                       | 155 184   | 2023-09-22 1.. |
| G_1900.pth                       | 155 184   | 2023-09-22 1.. |
| G_2000.pth                       | 155 184   | 2023-09-22 1.. |
| G_2100.pth                       | 155 184   | 2023-09-22 1.. |
| G_latest.pth                     | 155 184   | 2023-09-22 1.. |
| train.log                        | 65        | 2023-09-22 1.. |

## (6) 测试部分 `m_test.py`

```
def get_text(text, hps, is_symbol):
    text_norm = text_to_sequence(text, hps.symbols, [] if is_symbol else hps.data.text_cleaners)
    if hps.data.add_blank:
        text_norm = commons.intersperse(text_norm, 0)
    text_norm = LongTensor(text_norm)
    return text_norm

def tts_fn(model, text, speaker_id, language, speed, hps):
    if language is not None:
        text = language + text + language

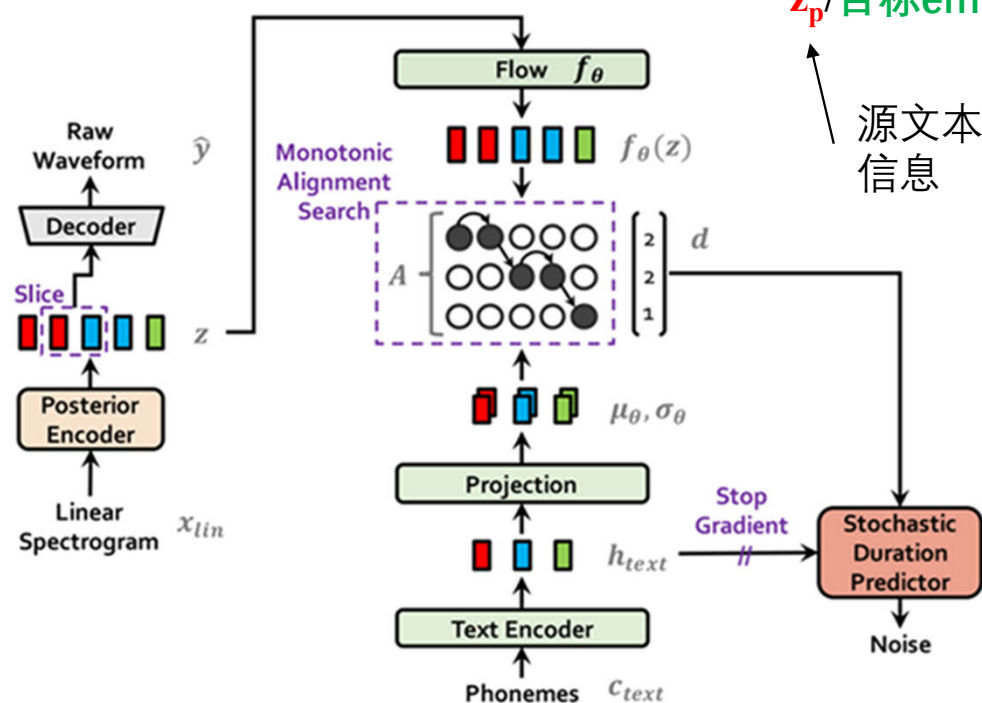
    stn_tst = get_text(text, hps, False)
    with no_grad():
        x_tst = stn_tst.unsqueeze(0).to(device)
        x_tst_lengths = LongTensor([stn_tst.size(0)]).to(device)
        sid = LongTensor([speaker_id]).to(device)
        audio = model.infer(x_tst, x_tst_lengths, sid=sid, noise_scale=.667, noise_scale_w=0.8,
                           length_scale=1.0 / speed)[0][0, 0].data.cpu().float().numpy()
    del stn_tst, x_tst, x_tst_lengths, sid
    return hps.data.sampling_rate, audio
```

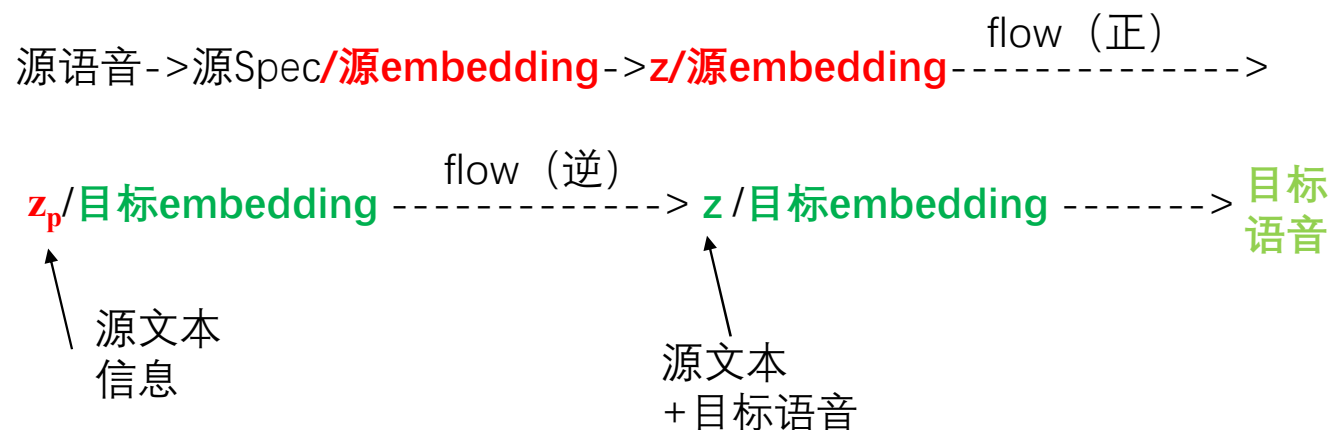
```
if __name__ == "__main__":  
    # 通过config 读取 hps  
    file_config = os.path.join('configs','modified_m_speaker_finetune.json')  
    hps = utils.get_hparams_from_file(file_config)  
  
    # 读取模型  
    file_model = os.path.join("yuhong2","G_latest.pth")  
    net_g = SynthesizerTrn(  
        len(hps.symbols),  
        hps.data.filter_length // 2 + 1,  
        hps.train.segment_size // hps.data.hop_length,  
        n_speakers=hps.data.n_speakers,  
        **hps.model).to(device)  
    _ = net_g.eval()  
  
    _ = utils.load_checkpoint(file_model, net_g, None)  
  
    text= "你好,欢迎来到鲁东大学"  
    lang = "[ZH]"  
    fs, audio = tts_fn(net_g, text, speaker_id=0, language=lang, speed=1, hps=hps)  
  
    sf.write('female-zzz.wav', audio, samplerate=fs)
```

# VITS 用于 Voice Conversion

源语音  $\rightarrow$  源Spec/**源embedding**  $\rightarrow$  **z/源embedding**  $\xrightarrow{\text{flow (正)}}$

**z<sub>p</sub>/目标embedding**  $\xrightarrow{\text{flow (逆)}}$  **z/目标embedding**  $\rightarrow$  目标语音





class SynthesizerTrn(nn.Module):  
内部函数 models.py 390行

```
def voice_conversion(self, y, y_lengths, sid_src, sid_tgt):
    assert self.n_speakers > 0, "n_speakers have to be larger than 0."
    if sid_src is None:
        g_src = None
    else:
        g_src = self.emb_g(sid_src).unsqueeze(-1)
    g_tgt = self.emb_g(sid_tgt).unsqueeze(-1)
    z, m_q, logs_q, y_mask = self.enc_q(y, y_lengths, g=g_src)
    z_p = self.flow(z, y_mask, g=g_src)
    z_hat = self.flow(z_p, y_mask, g=g_tgt, reverse=True)
    o_hat = self.dec(z_hat * y_mask, g=g_tgt)
    return o_hat, y_mask, (z, z_p, z_hat)
```

```
def vc_fn(audio, fs, org_id, tgt_id, model):  
    # audio = (audio / np.iinfo(audio.dtype).max).astype(np.float32)  
    # audio = (audio / np.finfo(audio.dtype).max).astype(np.float32)  
    if len(audio.shape) > 1:  
        audio = librosa.to_mono(audio.transpose(1, 0))  
    if fs != hps.data.sampling_rate:  
        audio = librosa.resample(audio, orig_sr=fs, target_sr=hps.data.sampling_rate)  
    with no_grad():  
        y = torch.FloatTensor(audio)  
        y = y / max(-y.min(), y.max()) / 0.99  
        y = y.to(device)  
        y = y.unsqueeze(0)  
        spec = spectrogram_torch(y, hps.data.filter_length,  
                                hps.data.sampling_rate, hps.data.hop_length, hps.data.win_length,  
                                center=False).to(device)  
        spec_lengths = LongTensor([spec.size(-1)]).to(device)  
        if org_id is None:  
            sid_src = None  
        else:  
            sid_src = LongTensor([org_id]).to(device)  
        sid_tgt = LongTensor([tgt_id]).to(device)  
        audio = model.voice_conversion(spec, spec_lengths, sid_src=sid_src, sid_tgt=sid_tgt)[0][  
            0, 0].data.cpu().float().numpy()  
    del y, spec, spec_lengths, sid_src, sid_tgt
```